Integer multiplication in $O(n \log n)$ time on a single two-dimensional grid, with a formally verified proof*

Piaoyang Cui†
Google DeepMind

Yiming Liu†
Google DeepMind

David P. Woodruff†
Google Research
Carnegie Mellon University

30 September 2026

Abstract

We give an algorithm that multiplies two n-bit integers in $O(n \log n)$ steps on a deterministic multitape Turing machine, and we prove this bound in Lean 4 for an explicit machine with eight stacks. The algorithm of Harvey and van der Hoeven reaches the same bound by routing a one-dimensional Fourier transform through pairwise coprime lengths. That route requires the Chinese remainder theorem and resampling between grids of different sizes. Our algorithm needs neither, working on a single two-dimensional power-of-two grid of the transform's size. We read the twiddle factors of the fast Fourier transform as fractional row shifts, realized by spreading each row with a data-independent smooth kernel. A short, well-conditioned recurrence in each column then undoes the spreading to working precision. The two-dimensional transform is computed by a classical polynomial transform, which needs no multiplications, together with recursive one-dimensional transforms along the other axis. Because the kernel is flat where needed, working precision grows only additively across recursive levels. A single-threaded C implementation confirms the algorithm's correctness bit for bit against GMP on inputs up to a billion bits.

*This work was carried out with substantial assistance from AI agents (primarily Gemini 4), including proof development, auditing, and the Lean 4 formalization; see Section 1.4.

†Authors are listed in alphabetical order.

1 Introduction

1.1 Background

Multiplying two n-bit integers on a deterministic multitape Turing machine means reading the $2n$ input bits from an input tape and writing the $2n$-bit product to an output tape, counting every head move and state transition. In 1971, Schönhage and Strassen gave an $O(n \log n \log\log n)$ algorithm and conjectured that the complexity of integer multiplication is of order $n \log n$ [SS71]. Fifty years later, Harvey and van der Hoeven proved the upper bound of that conjecture with an $O(n \log n)$ algorithm [HvdH21]. The extra $\log\log n$ factor in classical Fourier multipliers comes from recursive butterfly multiplications on logarithmic-sized chunks; Section 2 gives the parameter accounting.

Before $O(n \log n)$ was reached, Fürer replaced the $\log\log n$ factor by $2^{O(\log^* n)}$ using synthetic roots of unity that act by cyclic shifts in quotient rings, as in Section 2.3 [Für07, Für09]. Harvey, van der Hoeven, and Lecerf sharpened that bound to $O(n \log n \cdot 8^{\log^* n})$ [HvdHL16]. A classical polynomial transform due to Nussbaumer eliminates butterfly multiplications on a two-dimensional power-of-two grid [Nus80], whereas integer multiplication starts from a one-dimensional transform. Harvey and van der Hoeven bridged that gap by routing the transform through 1,729 pairwise coprime lengths via Chinese remaindering and resampling each axis onto a larger power-of-two grid [HvdH21]. A recent preprint gave an $o(n \log n)$ bound by combining a Harvey–van der Hoeven-style architecture whose dimension grows with n with finite linear networks [OAI26]; the present paper does not use that work.

1.2 Main results

In this paper, we show that $O(n \log n)$ integer multiplication on a multitape Turing machine needs neither Chinese remaindering nor resampling between grids of different sizes, working instead on a single two-dimensional grid of power-of-two size. Throughout, an n-bit integer means a nonnegative integer in $[0, 2^n)$; signed n-bit integers reduce immediately in $O(n)$ steps.

Theorem 1. *There is a deterministic multitape Turing machine and a constant $C > 0$ such that, for every $n \geq 2$ and any two n-bit integers A and B, given the binary representations of A and B on the input tape, the machine writes the $2n$-bit binary product on the output tape in at most $Cn \log_2 n$ steps.*

In addition to the written proof of Theorem 1, we construct one explicit eight-stack binary machine and verify its bit-exactness and $O(n \log n)$ step bound in Lean 4.

Theorem 2 (formally verified)**.** *A fixed stack machine `tmIntMul` (Mathlib's `Turing.FinTM2`: eight binary stacks, 32 register states, a fixed finite control) satisfies the following, verified in Lean 4 [dMU21, Mat20]. For every $n \geq 2$ and all $A, B < 2^n$, started with the $2n$ input bits it halts with the $2n$ bits of $A \cdot B$ on its output stack within $C \cdot n\lfloor \log_2 n \rfloor$ steps, for a constant C (the formal statement is existential; its proof exhibits the explicit witness `C_tm_total` $\approx 5 \cdot 10^{92}$, Section 8, `IntMul.tmIntMul_evalsToInTime_n_log_n`). Every step executes $O(1)$ elementary stack operations (`IntMul.tmIntMul_elementary_stack_ops_n_log_n`). The proof uses no axioms beyond `propext`, `Classical.choice`, and `Quot.sound`.*

Remarks.

- **What the algorithm avoids and what it uses.** Every Fourier transform in our algorithm has power-of-two length. The algorithm uses no Chinese remainder map, no prime or coprime transform lengths, no resampling between grids of different sizes, and no grid expansion across

recursive levels. Instead, it combines Nussbaumer's multiplication-free polynomial transform on a two-dimensional grid with a smooth row-shifting step that is undone column by column to working precision; Section 2 and Table 1 compare each component against [HvdH21]. Our written proof imports no external algorithmic subroutine as a black box and relies only on standard real-analysis estimates.

- **Scope of this paper.** We present this self-contained two-dimensional multiplier in full, using a real-valued smoothing kernel. At every recursion depth, the active two-dimensional sub-grids together have the same number of entries as the top-level array; no grid ever grows. Smaller integer products of polylogarithmic bit-length that arise inside a level are handled by the same algorithm one stage down, resting at the base stage on a one-dimensional radix-2 Fourier multiplier with schoolbook arithmetic.
- **Executability.** We execute the algorithm end to end in a single-threaded C and GMP program, `intmul_fastdft` [CGMP26]. On one core, it multiplies two 10^8-bit integers in 92.5 seconds of end-to-end time (one-time table setup excluded; Section 9 gives the setup-inclusive figures) versus 0.606 seconds for GMP's `mpz_mul`, about 153 times slower. It agrees bit for bit with GMP on all tested inputs, one random operand pair per size, up to 10^9 bits. It is a reference test of the algorithm rather than a replacement for production multipliers; Section 9 and Table 4 report the full measurements and caveats.
- **Formal verification and trust base.** Theorem 2 is formally verified in Lean 4 for the explicit eight-stack binary machine `tmIntMul` from Lean's three standard logical axioms alone. Its finite control has about $1.04 \cdot 10^9$ labels, a size that Section 8 discloses and checks. Accepting the formal theorem requires trusting the Lean 4 kernel, the Mathlib definitions named in the statement, and our input-output bit-encoding definitions, quoted verbatim in Section 8. The constant-factor simulation between finite-alphabet multi-stack machines and multitape Turing machines is a standard fact of complexity theory and is not part of the formalization.
- **Remaining bottleneck.** All scalar and integer multiplications across the recursion take $o(n \log n)$ steps in total. The sole remaining $O(n \log n)$ work comes from the multiplication-free polynomial transform and tape routing, the sequential scans that transpose and permute records across tapes; Section 10 discusses this term.

1.3 Comparison with related work

Harvey and van der Hoeven [HvdH21] route a one-dimensional Fourier transform through a high-dimensional transform of pairwise coprime lengths via Chinese remaindering and resample each axis onto a larger power-of-two grid by Gaussian convolution. Because the source and target grid sizes differ on every axis, their grid expands at each step and uses a high fixed dimension. Our algorithm works on a single two-dimensional power-of-two grid of the same size as the transform at every recursion depth. The active subproblems at each depth partition the original array with zero grid expansion, and the smoothing kernel keeps the working precision at $O(\log n)$ bits across all $O(\log \log n)$ recursive levels.

A companion manuscript [CLW26] extends this same-grid approach to growing dimension and claims an $o(n \log n)$ step bound on a deterministic multitape Turing machine. It obtains that bound by combining the growing-dimension grid with the finite linear network primitives of [OAI26]. Nothing in the present paper depends on [CLW26]. That manuscript's main theorem treats the inner multiplier on polylogarithmic-bit operands as a black box. Its Chinese-remainder-free instantiation and its formally verified proof use a formalization of our two-dimensional multiplier in that role. The present paper gives the self-contained two-dimensional multiplier that is executed up to 10^9

bits in C and GMP and formally verified for one concrete eight-stack machine.

1.4 Verification and executability

The formal proof of Theorem 2 is organized as the Lean 4 library `IntMul` [LeanMS26] of 27 files and 82,377 lines, described in Section 8. Built on Mathlib [Mat20], the library proves bit-exactness and the $O(n \log n)$ bounds on steps and elementary stack operations, such as single pushes, pops, peeks, and branches, for `tmIntMul`, together with all underlying analytic and fixed-point estimates. Table 3 records the correspondence between the paper's statements and the Lean declarations, including the places where the formal constants are looser than the printed ones. Compile-time axiom checks confirm that the headline theorems depend only on Lean's three standard logical axioms: propositional extensionality `propext`, classical choice `Classical.choice`, and quotient soundness `Quot.sound`.

Section 9 and Table 4 report single-core measurements and one multi-threaded run of the C and GMP program `intmul_fastdft` [CGMP26] across inputs from 10^7 to 10^9 bits. Those measurements cover core transform times, one-time table setup, memory footprint, and a comparison against a plain-Gaussian baseline. Section 9 also documents where the C program differs from the Turing machine specification, including a smaller base-case threshold under which the second recursion level at 10^9 bits falls below the minimum size assumed in our recursive proof.

Use of AI tools. This work was developed with substantial assistance from AI agents (primarily Gemini 4), working under the direction of the authors. That assistance spanned the algorithm, the proofs in this paper, the C/GMP implementation and its benchmark harness, and the Lean 4 formalization. The authors have checked the written proofs in this paper. No author has checked the formal Lean proof line by line; the correctness of the formal development rests on verification by Lean's kernel.

The authors have checked the formal statements of the machine, its uniformity, and the step bound of Theorem 2 in `IntMul/TMEndToEnd.lean` and the encoding conventions in `IntMul/Defs.lean`. The further definitions and the elementary-operation theorem that fix the meaning of Theorem 2 (the `IsUniformFinTM2` predicate and the `CoeDep` instance in `IntMul/TMPrimitives.lean`, and `stmtMaxElemOps`, `FinTM2BoundedElemOps`, and `tmIntMul_elementary_stack_ops_n_log_n` in `IntMul.lean`) are quoted verbatim in Section 8 so that readers can inspect them. Agreement among AI systems was used only as a working heuristic during exploration, never as mathematical evidence. Every theorem and lemma stated in this paper is established by a complete written proof, an explicit citation to the literature, or a kernel-checked formal theorem. Statements about the companion manuscript report its claims, standard facts about models of computation (such as the constant-factor simulation between stack machines and multitape Turing machines) are stated as such, and the measurements of Section 9 are read from the logs of the runs described there.

1.5 Organization

Section 2 explains how the algorithm works; the full statements and proofs begin in Section 3. Specifically, Section 3 fixes notation, the Turing machine and fixed-point model, shared table generation, tape routing, and the polynomial transform; readers familiar with fixed-point arithmetic and Nussbaumer's transform may skim it after checking Table 2. Section 4 proves the two-dimensional transform reduction and the fractional-shift identity; readers who trust Proposition 4.1 and Theorem 4.2 can skip to Section 5. Section 5 analyzes the smoothing kernel and the column solvers; readers who trust Theorem 5.1 and Lemma 5.2 can move on to Section 6.

Section 6 bounds the row-spreading step and closes the inductive error across recursive levels, and Section 7 states the full algorithm and proves Theorem 1. Section 8 presents the Lean 4 formalization of Theorem 2 without assuming Lean background, Section 9 reports the C and GMP measurements, and Section 10 isolates the remaining $O(n \log n)$ routing term. Appendices A–C collect deferred proofs for series evaluation, tape scheduling, tape routing, and rounding error.

2 Technical overview

Splitting two n-bit integers A and B into $N = \lceil n/b \rceil$ digits of $b = \Theta(\log n)$ bits reduces multiplication to a cyclic convolution of power-of-two length $R_0 = \Theta(n/\log n) \geq 2N$ at fixed-point precision $p_0 = \Theta(\log n)$ bits. A standard one-dimensional radix-2 discrete Fourier transform of length R_0 performs $\frac{1}{2} R_0 \log_2 R_0$ complex butterfly multiplications on p_0-bit operands; even when each p_0-bit product costs $\Theta(p_0 \log p_0)$ steps, those twiddle multiplications cost $\Theta(n \log n \log\log n)$ steps [HvdH21]. Nussbaumer's polynomial transform [Nus80] eliminates butterfly multiplications on a two-dimensional power-of-two grid by letting roots of unity act as cyclic shifts of polynomial coefficients, but a two-dimensional transform $F_{t_1,r}$ lacks the twiddle factors of a one-dimensional transform $F_{t_1 r}$. To avoid those twiddles, Harvey and van der Hoeven map the one-dimensional transform onto 1,729 pairwise coprime axis lengths via the Chinese remainder theorem and resample each coprime axis onto a larger power-of-two grid by convolving with a Gaussian [HvdH21].

We no longer use coprime lengths, Chinese remaindering, or resampling between grids of different sizes. Instead, we observe that on a $t_1 \times r$ power-of-two grid of total size $R = t_1 r$, the Cooley–Tukey twiddle factor in row j_1 shifts every entry of that row by the same fraction j_1/t_1 of a grid cell. We realize that fractional shift directly on the same $t_1 \times r$ grid by convolving each row cyclically with a data-independent smooth kernel. Next, we evaluate the two-dimensional transform $F_{t_1,r}$ without butterfly multiplications via Nussbaumer's polynomial transform and t_1 recursive transforms of length r. Finally, we undo the kernel spreading to working precision in each frequency column by a linear-time two-term recurrence.

Under this single-grid recursion, only the multiplication-free polynomial transform and sequential tape routing take $\Theta(n \log n)$ steps; all scalar and integer multiplications total $o(n \log n)$. Because the grid never expands, the active subproblems at every recursion depth total R_0 complex entries at $\Theta(\log n)$-bit precision, keeping the total bit volume $\Theta(n)$ per level. Multiplying the Gaussian kernel by a unit-width sinc function forms a windowed sinc-Gaussian kernel that stays flat up to the half-frequency boundary, so that undoing the spreading costs only $O(\log R_\ell)$ extra bits of precision per level. Slicing each short row convolution into kernel-width blocks by overlap-add and packing each block into one $O((\log n)^2)$-bit integer via Kronecker substitution makes the kernel width cancel out of the multiplication cost. Sections 3–7 formalize each piece in turn; readers familiar with this three-paragraph outline may move directly to Section 3.

2.1 The algorithm in one box

Listing 1 in Section 7 specifies the top-level multiplier `Multiply` and the recursive power-of-two transform `FastDFT`. At the root, `Multiply` splits two n-bit integers A and B into $N = \lceil n/b \rceil$ radix-2^b digits $a_j, b_j \in [0, 2^b)$ with $b = \max(4, \lfloor \log_2 n \rfloor)$. It pads the digits to a power of two $R_0 = 2^{K_0} \geq 2N$ and packs both inputs into one normalized complex vector $\tilde{x}_j = 2^{-b}(a_j + i b_j)$ at root precision $p_0 = 2b + 2K_0 + 6$.

Intermediate values and products use faithful fixed-point rounding $\mathcal{Q}_{p_0+2}$ and rounded multiplication RM_{p_0+2}, each accurate to within one unit in the last place. The classical two-call real convolution identity of Lemma 3.4 packs two real vectors into one complex vector so that two forward

transforms recover their real cyclic convolution. Accordingly, two calls to `FastDFT` evaluate the digit convolution before nearest-integer rounding and carry propagation produce $A \cdot B$. Inside `FastDFT`, non-leaf levels with $R_\ell > 4p_\ell^2$ factor $R_\ell = t_1 r$ and recurse, whereas leaf levels with $R_\ell \le 4p_\ell^2$ stop the recursion and call the direct radix-2 transform `BaseRadix2FFT`. Algorithm 1 summarizes the pipeline.

Algorithm 1 Integer multiplication on a single two-dimensional grid

Require: n-bit integers A, B
Ensure: $A \cdot B$

Parameters: $b = \max(4, \lfloor \log_2 n \rfloor)$, $N = \lceil n/b \rceil$, least power of two $R_0 = 2^{K_0} \ge 2N$, $p_0 = 2b + 2K_0 + 6$

if $n \le 64$ **then return** $A \cdot B$ by schoolbook multiplication
$\tilde{x} \leftarrow \mathrm{Pack}(A, B)$ ▷ $\tilde{x}_j = 2^{-b}(a_j + ib_j)$, zero-padded to R_0
$\hat{y} \leftarrow \texttt{FastDFT}(\tilde{x}, R_0, p_0)$
$\hat{w} \leftarrow \overline{\mathrm{RM}_{p_0+2}(\hat{y}, \hat{y})}$ ▷ pointwise square and conjugate, Lemma 3.4
$\hat{z} \leftarrow \texttt{FastDFT}(\hat{w}, R_0, p_0)$
return $\mathrm{Recover}(\hat{z})$ ▷ round $-2^{2b-1} R_0^2 \,\mathrm{Im}(\hat{z}_j)$ to integers, propagate radix-2^b carries
procedure $\texttt{FastDFT}(x, R_\ell, p_\ell)$ ▷ $R_\ell = 2^{K_\ell}$; Listing 1 gives the exact specification
  if $R_\ell \le 4p_\ell^2$ **then return** $\texttt{BaseRadix2FFT}(x)$ at precision $q_b = p_\ell + K_\ell + 4$
  factor $R_\ell = t_1 r$ with $t_1 = 2^{\lfloor K_\ell/2 \rfloor}$, $r = 2^{\lceil K_\ell/2 \rceil}$; set $p_{\mathrm{work},\ell}, p_{\ell+1}, q_{\mathrm{child}}$ by the schedule (34)
  $A_{\mathrm{N}} \leftarrow$ alternating sum of row $t_1/2$ ▷ (25)
  Step 1. Spread rows: for each row j_1, convolve with the windowed sinc-Gaussian kernel at offset j_1/t_1 by overlap-add and Kronecker substitution ▷ Theorem 6.3
  Step 2. 2-D DFT: $Y \leftarrow$ (2a) pre-twist by ζ^{k_2}, (2b) G_{t_1} over $\mathbb{C}[y]/(y^r + 1)$, (2c) post-twist by ζ^{-k_2}, (2d) t_1 `FastDFT` calls of length r at $p_{\ell+1}$, odd-stride routing ▷ Lemma 3.5, Prop. 4.1, Lemma 3.3
  Step 3. Deconvolve columns: solve $c_0(u_2) Z_{u_1} + c_1(u_2) Z_{(u_1-1) \bmod t_1} = Y_{u_1}$ on each column $u_2 \ne r/2$, solve $u_2 = r/2$ with A_{N}, quantize to $\widetilde{\mathbb{C}}_{p_\ell+2}$, transpose ▷ Theorem 5.1, Lemma 5.2
  return Z

Inner $O((\log n)^2)$-bit products are delegated to the same algorithm one stage down, backed at the bottom by a radix-2 Fourier multiplier with schoolbook arithmetic (Section 7).

2.2 Why one grid suffices

This subsection shows how fractional row shifts reduce a one-dimensional transform F_R to a two-dimensional transform $F_{t_1,r}$ on the same $t_1 \times r$ power-of-two grid. Fix $R = t_1 r$ with $t_1 \mid r$ and $t_1 \in \{r, r/2\}$, write the input index in strided form $j = j_1 + t_1 j_2$, and write the output frequency in mixed-radix form $u = u_2 + r u_1$, where $0 \le j_1, u_1 < t_1$ and $0 \le j_2, u_2 < r$. Expanding the normalized one-dimensional transform $(F_R x)_u = \frac{1}{R} \sum_{j=0}^{R-1} x_j e^{2\pi i j u / R}$ gives the Cooley–Tukey decomposition

$$(F_R x)_{u_2 + r u_1} = \frac{1}{t_1} \sum_{j_1=0}^{t_1-1} e^{2\pi i j_1 u_1 / t_1} \left(\frac{1}{r} \sum_{j_2=0}^{r-1} x(j_1 + t_1 j_2)\, e^{2\pi i (j_2 + j_1/t_1) u_2 / r} \right). \tag{1}$$

Without the offset j_1/t_1 in the inner exponential, the right-hand side of (1) would equal the two-dimensional transform $(F_{t_1,r} U)_{u_1,u_2}$ of the array $U_{j_1,j_2} = x(j_1 + t_1 j_2)$. With that offset present, the inner sum is still a length-r Fourier sum along row j_1, except that every sample in row j_1 undergoes a uniform fractional shift: its position is displaced by $j_1/t_1 \in [0, 1)$ of a grid cell.

To move those fractionally shifted samples back onto the integer grid nodes without changing the grid size, we convolve each row j_1 cyclically modulo r with a smooth, rapidly decaying kernel g

evaluated at offset j_1/t_1:

$$U^{\circ}_{j_1,k_2} := \sum_{j_2=0}^{r-1} x(j_1 + t_1 j_2) \sum_{a\in\mathbb{Z}} g\Big(j_2 + \frac{j_1}{t_1} - k_2 - ar\Big).$$

We analyze this cyclic convolution by Poisson summation, the classical identity that equates a periodized spatial sum of a smooth function with a sum of its continuous Fourier transform $\hat{g}$ over integer frequency aliases $m \in \mathbb{Z}$. Applying Poisson summation along the row coordinate k_2 supplies both the Cooley–Tukey twiddle $e^{2\pi i j_1 u_2/R}$ and a row-frequency shift $-m$ modulo t_1 through the phase $(j_1/t_1)(u_2/r - m)$. In this way, the m-th alias of the kernel's transform moves the output by m rows, yielding the exact bridge identity of Theorem 4.2:

$$\left(F_{t_1,r} U^{\circ}\right)_{u_1,u_2} = \sum_{m\in\mathbb{Z}} \hat{g}\Big(\frac{u_2}{r} - m\Big) \left(F_R x\right)_{u_2 + r((u_1 - m) \bmod t_1)}. \tag{2}$$

For $y = u_2/r \in [0,1)$, the two arguments $y - m$ with $m \in \{0,1\}$ lie in $[-1,1)$, whereas every other alias $m \notin \{0,1\}$ satisfies $|y - m| \geq 1$. When $\hat{g}$ is concentrated on $[-1,1]$ up to exponentially small tails, each column u_2 of $F_{t_1,r}U^{\circ}$ reduces to working precision to a two-term cyclic combination $c_0(u_2)Z_{u_1} + c_1(u_2)Z_{(u_1-1) \bmod t_1}$ of the target values $Z_{u_1} = (F_R x)_{u_2 + r u_1}$ on the same $t_1 \times r$ grid. Here the retained weights are $c_0(u_2) = \hat{g}(u_2/r)$ and $c_1(u_2) = \hat{g}(u_2/r - 1)$.

Table 1: Comparison of transform components in [HvdH21] and in this paper.

Component	[HvdH21]	This paper
CRT map	Good–Thomas embedding onto coprime axes	None
Coprime lengths	Pairwise coprime axis lengths	None (powers of two t_1, r only)
Resampling	Gaussian resampling between grids of different sizes	Same-grid row spreading by windowed sinc-Gaussian kernel
Multidimensional transform	Nussbaumer polynomial transforms	Nussbaumer polynomial transform over $\mathbb{C}[y]/(y^r + 1)$ (one two-dimensional level)
Dimension	High-dimensional ($d = 1729$ in main proof)	Two-dimensional ($t_1 \times r$ grid, $t_1 r = R$)
Working precision	$O(\log n)$ bits, fixed per transform; resampling conditioned by oversampling the target grid	$O(\log n)$ bits at every level, additive schedule $p_{\ell+1} = p_\ell + 2K_\ell + 12$; two-term solvers conditioned by at most $2.36\,t_1$, no oversampling

Table 1 and Figure 1 summarize what this single-grid pipeline removes relative to [HvdH21] and how it differs from nonuniform fast Fourier transform gridding [DR93, PST01, GL04, BMK19]. General gridding interpolates arbitrary sample nodes onto an oversampled grid and divides diagonally by $\hat{g}$. Here the shift j_1/t_1 is constant along each row, so spreading is a cyclic convolution of length r and oversampling is replaced by a two-term cyclic solve per column. All analytic inputs in our proof are elementary real-analysis facts: Poisson summation for Schwartz functions, the convolution theorem, Parseval's identity, and standard series and Gaussian tail bounds.

A tiny worked grid. To see the reduction concretely, take $t_1 = 2$ and $r = 4$, so $R = 8$, as illustrated in Figure 2. The eight inputs $x = (x_0, \ldots, x_7)$ sit on a 2×4 grid: row $j_1 = 0$ holds the even

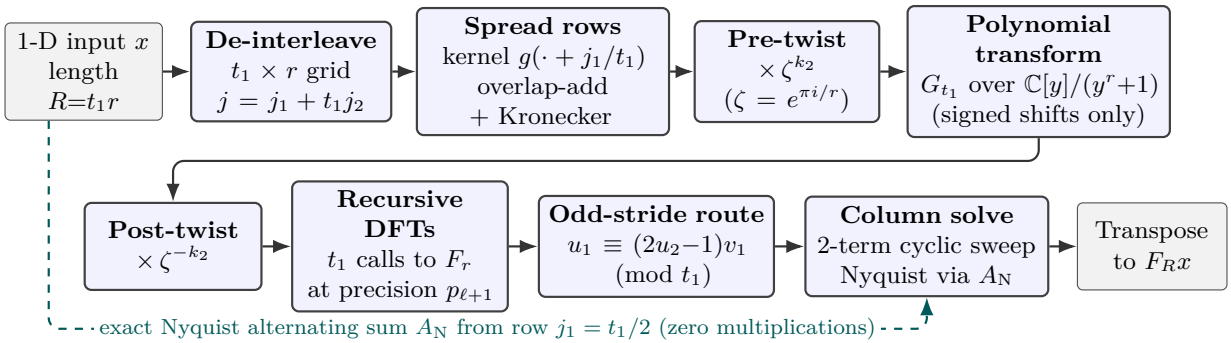


Figure 1: One recursive step of `FastDFT` on a single two-dimensional power-of-two grid of size $R = t_1 r$. Spreading each row with a smooth kernel turns the fractional row shifts into a two-term mixture down each frequency column. A multiplication-free polynomial transform and t_1 recursive length-r transforms evaluate $F_{t_1,r}$, after which per-column solvers recover $F_R x$ to working precision.

entries (x_0, x_2, x_4, x_6) at integer cells with shift 0, and row $j_1 = 1$ holds the odd entries (x_1, x_3, x_5, x_7) at half-cell shift $1/2$. Convolving row 0 with g at offset 0 and row 1 with $g(\cdot + 1/2)$, each cyclically modulo 4, and applying $F_{2,4}$ produces two values Y_0, Y_1 in each column $u_2 \in \{0, 1, 2, 3\}$. Because sinc vanishes at every nonzero integer, g at offset 0 is the identity on the grid, so row 0 passes through unchanged and only the half-shifted row 1 is actually spread. Dropping the exponentially small outer aliases $m \notin \{0, 1\}$ in (2) gives $Y_0 \approx c_0(u_2)Z_0 + c_1(u_2)Z_1$ and $Y_1 \approx c_0(u_2)Z_1 + c_1(u_2)Z_0$ for $Z_0 = (F_8 x)_{u_2}$ and $Z_1 = (F_8 x)_{u_2+4}$.

On columns $u_2 \in \{0, 1, 3\}$, $c_0(u_2) \neq c_1(u_2)$ and this 2×2 system inverts directly to working precision. At the midpoint Nyquist column $u_2 = r/2 = 2$, corresponding to half the sampling rate $u_2/r = 1/2$, symmetry of an even $\hat{g}$ gives $c_0(2) = \hat{g}(1/2) = \hat{g}(-1/2) = c_1(2)$. There $Y_1/c_0(2)$ gives $Z_0 + Z_1$ to working precision, while row 1 supplies the exact difference $A_N = Z_0 - Z_1 = \frac{i}{4}(x_1 - x_3 + x_5 - x_7)$ without multiplications.

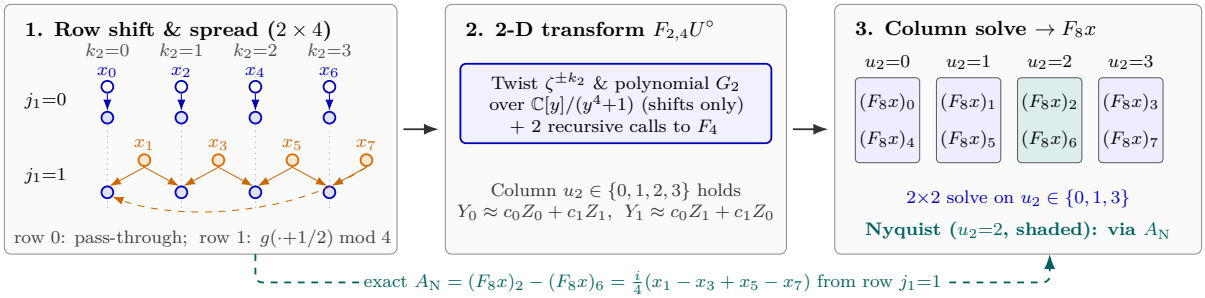


Figure 2: How the single-grid reduction computes an eight-point transform on a 2×4 grid with $t_1 = 2$ and $r = 4$. Convolving row j_1 with $g(\cdot + j_1/2)$ modulo 4 passes row 0 through unchanged and spreads the half-shifted row 1 onto the four integer grid nodes before $F_{2,4}$ runs without butterfly multiplications. Each column $u_2 \in \{0, 1, 3\}$ then recovers $(F_8 x)_{u_2}$ and $(F_8 x)_{u_2+4}$ to working precision from a 2×2 system, while the Nyquist column $u_2 = 2$ combines $Y_1/c_0(2)$ with the exact input alternating sum A_N.

2.3 Computing $F_{t_1,r}$ without multiplications

This subsection explains how the classical polynomial transform of Nussbaumer and Quandalle [NQ79, Nus80, Nus82] reduces the two-dimensional transform $F_{t_1,r}$ to t_1 one-dimensional transforms F_r without butterfly multiplications. Evaluating $F_{t_1,r}U$ directly by row and column transforms would perform $\Theta(R \log t_1)$ scalar multiplications along the first axis. Instead, we work in the negacyclic polynomial quotient ring $\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1)$, where r is a power of two and the indeterminate y satisfies $y^r \equiv -1$. Thus y acts as a synthetic root of unity of order $2r$ whose powers shift r-coefficient polynomials cyclically with sign flips. Because $t_1 \mid r$ with $\nu = r/t_1 \in \{1, 2\}$, the monomial $\omega_{t_1} := y^{2\nu}$ satisfies $\omega_{t_1}^{t_1/2} = y^r \equiv -1$ and has exact order t_1 in $\mathcal{R}_r$. The Nussbaumer polynomial transform of length t_1,

$$(G_{t_1} P)_{v_1}(y) := \frac{1}{t_1} \sum_{j_1=0}^{t_1-1} P_{j_1}(y)\, y^{2\nu j_1 v_1} \pmod{y^r + 1},$$

therefore runs in $\log_2 t_1$ radix-2 butterfly stages over $\mathcal{R}_r$ using only additions, halvings, and signed cyclic coefficient shifts, with zero scalar multiplications, as proved in Lemma 3.5.

To connect $\mathcal{R}_r$ to the row transforms of $F_{t_1,r}$, we apply the classical negacyclic-to-cyclic twist [SS71, CF94], formalized in Proposition 4.1. Let $\zeta = e^{\pi i/r}$, so $\zeta^r = -1$; the r roots of $y^r + 1 = 0$ are the odd powers ζ^{2u_2-1} for $u_2 \in \{0, \dots, r-1\}$, defining r evaluation homomorphisms $\phi_{u_2} : \mathcal{R}_r \to \mathbb{C}$ via $y \mapsto \zeta^{2u_2-1}$. Pre-twisting column k_2 of row j_1 by ζ^{k_2} forms $P_{j_1}(y) = \sum_{k_2=0}^{r-1} \zeta^{k_2} U_{j_1,k_2} y^{k_2}$, whose evaluation $\phi_{u_2}(P_{j_1}) = \sum_{k_2=0}^{r-1} U_{j_1,k_2} e^{2\pi i k_2 u_2/r}$ is the unnormalized length-r Fourier transform along row j_1. Conversely, for each output polynomial $Q_{v_1} = (G_{t_1} P)_{v_1}$, post-twisting coefficient k_2 by ζ^{-k_2} and calling a recursive length-r transform F_r evaluates $\frac{1}{r}\phi_{u_2}(Q_{v_1})$ across all u_2 at once. Finally, $\phi_{u_2}(y^{2\nu j_1 v_1}) = e^{2\pi i j_1 (2u_2-1) v_1/t_1}$, and because $2u_2 - 1$ is odd while t_1 is a power of two, the map $v_1 \mapsto u_1 := (2u_2 - 1)v_1 \bmod t_1$ permutes $\{0, \dots, t_1 - 1\}$ bijectively in every column u_2. This multiplication-free polynomial transform, together with per-level tape transposes and odd-stride routing, is the sole $\Theta(n \log n)$ work term in the algorithm.

2.4 The kernel and two-term deconvolution

The choice of spreading kernel g governs how fixed-point precision grows with recursion depth. Suppose first that one spreads with a plain Gaussian $g_{\text{plain}}(t) = e^{-\pi t^2/\alpha_{\text{plain}}^2}$ scaled by $\pi\alpha_{\text{plain}}^2 = p_{\text{work},\ell} \ln 2$ so excluded aliases decay to $O(2^{-p_{\text{work},\ell}})$. Its Fourier transform $\widehat{g_{\text{plain}}}(\xi) = \alpha_{\text{plain}} 2^{-p_{\text{work},\ell}\xi^2}$ peaks at $\xi = 0$ but drops at the half-frequency Nyquist boundary $\xi = 1/2$, where $u_2 = r/2$, to $\alpha_{\text{plain}} 2^{-p_{\text{work},\ell}/4}$, as plotted on the left of Figure 3. Inverting the column system near $u_2 = r/2$ then divides by $O(2^{-p_{\text{work},\ell}/4})$; in a natural heuristic estimate, not used in our proofs, this loses about $\frac{1}{4} p_{\text{work},\ell}$ bits of accuracy per level and forces a multiplicative precision schedule $p_{\ell+1} \gtrsim \frac{5}{4} p_{\text{work},\ell}$.

We remove this attenuation by multiplying the Gaussian bell curve by $\operatorname{sinc}(t) = \frac{\sin(\pi t)}{\pi t}$ to form the windowed sinc-Gaussian kernel of Theorem 4.2, whose Fourier transform is a flat-topped smoothed boxcar:

$$g(t) := \operatorname{sinc}(t)\, e^{-\pi t^2/\alpha^2}, \qquad \alpha := 2\sqrt{\frac{p_{\text{work},\ell} \ln 2}{\pi}}. \tag{3}$$

In frequency space, multiplying by $\operatorname{sinc}(t)$ convolves $\alpha e^{-\pi\alpha^2\xi^2}$ with the unit indicator on $[-1/2, 1/2]$, giving

$$\hat{g}(\xi) = \frac{1}{2}\left[\operatorname{erf}\left(\sqrt{\pi}\alpha\left(\xi + \frac{1}{2}\right)\right) - \operatorname{erf}\left(\sqrt{\pi}\alpha\left(\xi - \frac{1}{2}\right)\right)\right].$$

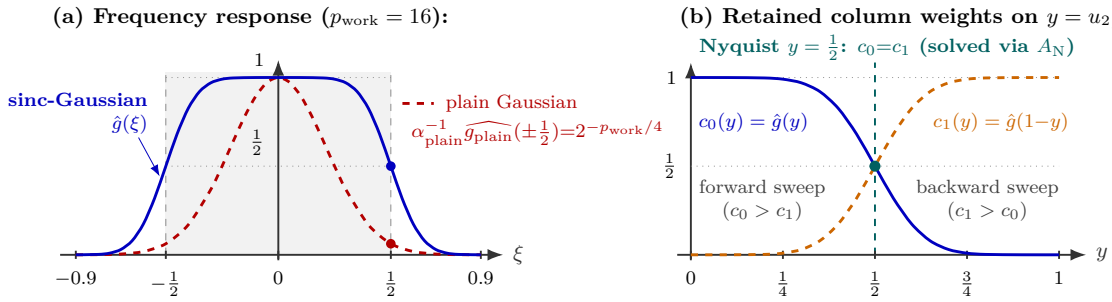


Figure 3: Why multiplying a Gaussian by a sinc function keeps the column recurrence well conditioned. Left: at the half-frequency boundary $\xi = \pm 1/2$, a plain Gaussian drops exponentially with the working precision, whereas the windowed sinc-Gaussian kernel stays near $1/2$. Right: across columns $y = u_2/r \in [0,1)$, the two retained weights $c_0(y)$ and $c_1(y)$ cross only at the Nyquist column $y = 1/2$, where the exact input alternating sum A_{N} closes the system.

Because the shifted intervals $[y-m-1/2, y-m+1/2]$ tile $\mathbb{R}$, the weights satisfy the exact partition of unity $\sum_{m\in\mathbb{Z}} \hat{g}(y-m) = 1$. For every $y = u_2/r \in [0,1)$ and $p_{\text{work},\ell} \geq 8$, the two retained weights $c_0(u_2) = \hat{g}(y)$ and $c_1(u_2) = \hat{g}(y-1) = \hat{g}(1-y)$ satisfy $\max(c_0, c_1) \geq \hat{g}(1/2) = \frac{1}{2}\operatorname{erf}(\sqrt{\pi}\alpha) > 0.49999999$, while all omitted aliases $m \notin \{0,1\}$ telescope in closed form to $1 - c_0(u_2) - c_1(u_2) < 0.120 \cdot 2^{-p_{\text{work},\ell}}$ by Theorem 5.1.

Two-term deconvolution solves the cyclic two-entry linear system $Y_{u_1} \approx c_0(u_2)Z_{u_1} + c_1(u_2)Z_{(u_1-1) \bmod t_1}$ in each column $u_2 \in \{0,\ldots,r-1\}$ to recover $Z_{u_1} = (F_R x)_{u_2+ru_1}$ to working precision, as shown on the right of Figure 3. Whenever $u_2 \neq r/2$, the separation $|1/2-y| \geq 1/r \geq 1/(2t_1)$ ensures that the damping power $(\min(c_0,c_1)/\max(c_0,c_1))^{t_1}$ lies strictly below 0.14 by Theorem 5.1. The procedure `CyclicSolve` of Section 5 sweeps forward when $u_2 < r/2$, where $c_0 > c_1$, or backward when $u_2 > r/2$, where $c_1 > c_0$, inverting the cyclic bidiagonal operator B in $O(t_1)$ scalar operations with ℓ_∞ inverse norm $1/|c_0(u_2) - c_1(u_2)| < 2.36\,t_1$.

At the single Nyquist column $u_2 = r/2$, corresponding to the half-sampling frequency $y = 1/2$, symmetry gives $c_0(r/2) = c_1(r/2) = c \approx 1/2$. Because $t_1 \geq 2$ is even, the cyclic equations $c(Z_{u_1} + Z_{(u_1-1) \bmod t_1}) = Y_{u_1}$ sum with alternating signs $(-1)^{u_1}$ to zero and leave the alternating mode $\sum_{u_1=0}^{t_1-1}(-1)^{u_1}Z_{u_1}$ undetermined. By character orthogonality on $\mathbb{Z}_{t_1}$, however, that alternating sum collapses directly in the spatial domain to a single row of the input vector, proved in Theorem 5.1 and (25):

$$A_{\text{N}} := \sum_{u_1=0}^{t_1-1}(-1)^{u_1}Z_{u_1} = \frac{i}{r}\sum_{j_2=0}^{r-1}(-1)^{j_2}x\left(\frac{t_1}{2} + t_1 j_2\right).$$

Accumulating A_{N} from row $j_1 = t_1/2$ before spreading uses only additions, subtractions, and a power-of-two shift by r^{-1}, with zero scalar multiplications. The procedure `NyquistSolve` replaces the wrap-around $u_1 = 0$ equation by A_{N} and solves the resulting non-cyclic two-term chain, an open bidiagonal system, at $u_2 = r/2$ with inverse norm below $1.0000001\,t_1 < 2.36\,t_1$.

2.5 Additive precision schedule

An additive precision schedule increases the fixed-point working precision by a small additive number of bits at each recursive call rather than by a constant factor. Uniform $O(t_1)$ conditioning across all r columns gives such an additive recurrence along the recursion tree, proved in Lemma 6.5. At depth ℓ, a node of length $R_\ell = 2^{K_\ell} = t_{1,\ell}r_\ell$ receives an input $x \in \widetilde{\mathbb{C}}^{R_\ell}_{p_\ell+2}$ and approximates $F_{R_\ell}x$

within ℓ_∞ error $2^{-p_\ell}\max(1,\|x\|_\infty)$. Two error groups enter before final quantization to $\widetilde{\mathbb{C}}_{p_\ell+2}$:

(1) *Excluded Poisson aliases.* Dropping modes $m \notin \{0,1\}$ perturbs the ideal column right-hand side by at most $0.120 \cdot 2^{-p_{\mathrm{work},\ell}}\max(1,\|x\|_\infty)$. Multiplication by B^{-1}, whose ℓ_∞ norm is below $2.36\,t_{1,\ell}$, amplifies this tail to at most $\frac{1}{2}t_{1,\ell}2^{-p_{\mathrm{work},\ell}}\max(1,\|x\|_\infty)$.

(2) *Spreading, polynomial transform, recursive calls, and column solvers.* Spatial window truncation and tap rounding in Step 1, together with rounding in the twists and $\log_2 t_{1,\ell}$ polynomial-transform stages of Steps 2a–2c, perturb the intermediate arrays at precision $p_{\ell+1}$. The inductive error $2^{-p_{\ell+1}}$ of the $t_{1,\ell}$ child calls in Step 2d on inputs of norm $O(r_\ell\max(1,\|x\|_\infty))$ then perturbs the column right-hand sides by $O(r_\ell 2^{-p_{\ell+1}}\max(1,\|x\|_\infty))$. Combined with fixed-point rounding across the $O(t_{1,\ell})$-step column sweeps of Step 3, these terms sum after deconvolution to at most $200R_\ell 2^{-p_{\ell+1}}\max(1,\|x\|_\infty)$.

Choosing the additive precision schedule (34),

$$p_{\mathrm{work},\ell} := p_\ell + K_\ell + 6, \qquad p_{\ell+1} := p_{\mathrm{work},\ell} + K_\ell + 6 = p_\ell + 2K_\ell + 12,$$

makes $\frac{1}{2}t_{1,\ell}2^{-p_{\mathrm{work},\ell}} \le 2^{-p_\ell-7}$ and $200R_\ell 2^{-p_{\ell+1}} \le \frac{25}{2048}2^{-p_\ell}$. Adding the final $\widetilde{\mathbb{C}}_{p_\ell+2}$ rounding error $\frac{\sqrt{2}}{8}2^{-p_\ell}$ keeps the total error below $0.197\cdot 2^{-p_\ell}\max(1,\|x\|_\infty) < \frac{1}{4}2^{-p_\ell}\max(1,\|x\|_\infty)$, closing the induction at every non-leaf node, while leaves $R_L \le 4p_L^2$ close at precision $q_b = p_L + K_L + 4$.

Because $K_{\ell+1} = \lceil K_\ell/2\rceil$, the log-length halves at each level and reaches the leaf threshold $R_L \le 4p_L^2$ in $L = O(\log K_0) = O(\log\log n)$ levels. Summing $2K_j + 12$ from 0 to $\ell-1$ bounds the precision at every level by $p_\ell \le p_0 + 4K_0 + 14\ell = \Theta(\log n)$, while the integer-part bit-width H_ℓ before the binary point, called the scalar height, satisfies $H_\ell \le 1 + K_0 + 2\ell = O(\log n)$, as shown in Section 3. Every level therefore stores R_0 fixed-width complex records of $w_\ell = \Theta(\log n)$ bits, maintaining the invariant bit volume $R_0 p_\ell = \Theta(n)$ throughout the recursion.

2.6 Cost on a multitape Turing machine

Because every depth $\ell < L$ holds R_0 active entries of $\Theta(\log n)$ bits, the multitape Turing machine cost proved in Section 7 splits into one $\Theta(n\log n)$ movement-and-shift term and $o(n\log n)$ arithmetic and table work:

- *Polynomial transform and tape routing.* At depth ℓ, each of the R_0/R_ℓ nodes performs $\log_2 t_{1,\ell} = \lfloor K_\ell/2\rfloor$ radix-2 polynomial-transform stages over $\mathcal{R}_{r_\ell}$ by Lemma 3.5, two matrix transposes, and one odd-stride merge-sort column permutation $u_1 = (2u_2 - 1)v_1 \bmod t_{1,\ell}$ by Lemma 3.3. Those steps cost $O(R_\ell p_{\ell+1}(1+\log_2 t_{1,\ell}))$ operations per node, or $O(n(1+K_0/2^{\ell+1}))$ across all R_0/R_ℓ nodes at depth ℓ, summing geometrically over $\ell = 0,\dots,L-1$ to $O(nK_0 + nL) = O(n\log n)$.
- *Kronecker spreading and window cancellation.* Truncating $g(k_2 + j_1/t_{1,\ell})$ at radius $m_{1,\ell} = \Theta(p_{\ell+1})$ and folding modulo r_ℓ gives a filter of effective width $\tilde{W}_\ell = \min(2m_{1,\ell}+1, r_\ell)$. We evaluate each row convolution by overlap-add, slicing the row into blocks of width $\tilde{W}_\ell$ and summing their overlapping products. Each short block product is computed by signed two's-complement Kronecker substitution [Sch82, Har09], which packs the fixed-point entries of a block into bitfields of width $B_{\mathrm{slot}} = \Theta(p_{\ell+1})$ inside one large integer. By Theorem 6.3, this reduces row spreading across a node to $O(R_\ell/\tilde{W}_\ell)$ integer products of bit-length $\Theta(\tilde{W}_\ell p_{\ell+1}) = O((\log n)^2)$.
- *Total multiplication volume across levels.* Because the window width $\tilde{W}_\ell$ cancels between the block count $O(R_\ell/\tilde{W}_\ell)$ and the packed operand bit-length $\Theta(\tilde{W}_\ell p_{\ell+1})$, all multiplications in spreading, scalar twists, column sweeps, and leaf transforms at $R_L \le 4p_L^2$ act on $O((\log n)^2)$-bit integers. Their total bit volume is $O(n)$ per non-leaf level $\ell < L$ and $O(n\log\log n)$ across the $K_L = O(\log\log n)$ butterfly stages of the leaf level.

- *Shared tables and non-circular bootstrap.* All per-level filter, twist, solver, and leaf-twiddle tables are evaluated from truncated Taylor series and Machin's arctangent formula for π at guard precision in $O(\sqrt{n}(\log n)^{7/2}\log\log n) = o(n)$ steps by (8). Alternating between two data tapes in a breadth-first sweep runs the recursion on $O(1)$ tapes in Appendix A.
  To multiply $O((\log n)^2)$-bit integers on $O(1)$ tapes without circularity in the asymptotic constant, the inner multiplier is a separately bounded procedure rather than the outer algorithm itself, as detailed in Section 7. Specifically, a three-stage bootstrap uses a schoolbook-backed radix-2 Fourier multiplier running in $O(N(\log 2N)^2)$ steps to feed a first stage of our multiplier running in $O(N\log 2N)$ steps. That first stage then multiplies all inner operands of the outer call in $O(n(\log\log n)^2)$ total steps.

2.7 What is verified and what is executed

Both the Lean 4 stack machine `tmIntMul` [LeanMS26] in Section 8 and the C/GMP program `intmul_fastdft` [CGMP26] in Section 9 execute the windowed sinc-Gaussian pipeline of Algorithm 1 under the identical additive precision schedule (34). Inside the Lean 4 library, `tmIntMul` realizes the breadth-first tape schedule on eight binary pushdown stacks with a two-tier self-recursive bootstrap, and Table 3 maps every analytic and fixed-point bound above to its formal theorem. By contrast, `intmul_fastdft` delegates the $O((\log n)^2)$-bit inner products to GMP and lowers the default leaf threshold from $4p_\ell^2$ to p_0^2, as analyzed in Section 9.

3 Preliminaries

This section sets up the computational model and basic primitives used throughout the proof of Theorem 1. Section 3.1 fixes notation, and Section 3.2 defines our multitape Turing machine model, dyadic fixed-point grids, and uniform record widths. Section 3.3 and Section 3.4 then account for shared elementary-function tables and sequential tape-routing sweeps, while Section 3.5 records our discrete Fourier transform and polynomial-transform conventions. We state these details explicitly because Theorem 1 charges every head move, rounding step, and transcendental evaluation on a Turing machine without unit-cost real-RAM oracles. Readers comfortable with fixed-point rounding and Nussbaumer's polynomial transform may check Table 2 and move directly to Section 4.

3.1 Notation

We first fix our logarithm and norm conventions. Logarithms are natural, written ln, unless base 2 is specified as $\log_2$. For an integer $R \geq 1$, a vector $x = (x_0, \dots, x_{R-1}) \in \mathbb{C}^R$ carries the Euclidean supremum norm $\|x\|_\infty := \max_{0\leq j<R} |x_j|$, where $|a+ib| = \sqrt{a^2+b^2}$ is the complex modulus. We also use the coordinatewise supremum norm $\|x\|_{\infty,\mathrm{coord}} := \max_{0\leq j<R} \max(|\operatorname{Re}(x_j)|, |\operatorname{Im}(x_j)|)$. For a linear map $B\colon \mathbb{C}^m \to \mathbb{C}^m$, $\|B\|_{\infty\to\infty}$ denotes the induced ℓ_∞ operator norm, namely the maximum absolute row sum of B. Table 2 collects the notation used throughout the paper.

3.2 Multitape Turing machine and fixed-point grids

All time bounds in this paper count single steps of a deterministic multitape Turing machine. Following Schönhage and Strassen [SS71] and Harvey and van der Hoeven [HvdH21, Section 2.1], the machine has a fixed finite alphabet and a fixed finite number of tapes. Every bound proved in this model also holds *a fortiori* on the standard $\Theta(\log n)$-bit Word RAM. Throughout the paper, a "bit operation" means one Turing machine step: a single state transition, one tape-symbol read

and write, and a head move in $\{-1, 0, +1\}$ on each of the $O(1)$ tapes. Sequential tape scans, head movements, rewinds, and counter updates are therefore charged explicitly.

For multiplication of two n-bit integers A and B, the input is the concatenation of their fixed-width n-bit binary strings on a designated input tape. That input has length $2n$ bits, so n is determined directly from the input. The output is the fixed-width $2n$-bit binary string of $A \cdot B$ written on a designated output tape, where both the input tape and the output tape belong to the fixed set of $O(1)$ tapes.

To approximate complex Fourier transforms on this machine, we represent real and complex scalars as integer multiples of 2^{-q} on a dyadic fixed-point grid. Whenever a subroutine targets p bits of accuracy, we work on the grid with $q = p + 2$ fractional bits so that a single coordinatewise rounding step costs less than a quarter of the target error budget.

Definition 3.1 (Fixed-point grids and rounding)**.** For an integer $q \geq 0$, define the dyadic fixed-point real and complex grids

$$\widetilde{\mathbb{R}}_q := \{2^{-q}a : a \in \mathbb{Z}\}, \qquad \widetilde{\mathbb{C}}_q := \{2^{-q}(a + ib) : a, b \in \mathbb{Z}\}.$$

Here q counts fractional bits after the binary point, while the integer part is represented in signed two's-complement binary without modular wrap-around. A target accuracy parameter $p \in \mathbb{Z}_{\geq 1}$ uses the working grid $\widetilde{\mathbb{C}}_{p+2}$ ($q = p+2$). For $x \in \mathbb{R}$, let $\mathcal{Q}_q(x) := 2^{-q}\,\mathrm{round}(2^q x) \in \widetilde{\mathbb{R}}_q$ denote nearest-grid rounding under any fixed deterministic tie-breaking rule on half-integers (such as ties to even, or $\lfloor 2^q x + 1/2 \rfloor$), and for $z = x + iy \in \mathbb{C}$, define coordinatewise rounding $\mathcal{Q}_q(z) := \mathcal{Q}_q(x) + i\mathcal{Q}_q(y) \in \widetilde{\mathbb{C}}_q$.

For every $x \in \mathbb{R}$ and $z \in \mathbb{C}$, regardless of which tie-breaking rule on half-integers is fixed in Definition 3.1, coordinatewise nearest rounding satisfies

$$|\mathcal{Q}_q(x) - x| \leq \frac{1}{2}2^{-q}, \qquad |\mathcal{Q}_q(z) - z| \leq \frac{\sqrt{2}}{2}2^{-q}. \tag{4}$$

On the working grid $\widetilde{\mathbb{C}}_{p+2}$ with $q = p+2$, real coordinate rounding error is at most $\frac{1}{8}2^{-p} = 0.125 \cdot 2^{-p}$ and complex Euclidean rounding error is at most $\frac{\sqrt{2}}{8}2^{-p} < 0.177 \cdot 2^{-p}$.

Scalar height and uniform fixed-width records. Every complex number $z \in \mathbb{C}$ satisfies $\|z\|_{\infty,\mathrm{coord}} \leq |z| \leq \sqrt{2}\|z\|_{\infty,\mathrm{coord}}$. Suppose a grid scalar $z \in \widetilde{\mathbb{C}}_q$ satisfies $\|z\|_{\infty,\mathrm{coord}} \leq 2^H$ for an integer $H \geq 0$. This bound holds whenever $|z| \leq 2^H$, and by Lemma 3.5 it is preserved by the polynomial transform G_{t_1}. Then the scaled integer coordinates $2^q \operatorname{Re}(z), 2^q \operatorname{Im}(z) \in \mathbb{Z}$ have absolute value at most 2^{q+H}. Along any recursive call tree of `FastDFT` starting from a normalized root input $\|x^{(0)}\|_\infty < \sqrt{2}$ with $H_0 := 1$, Lemma 6.5 shows that the child input vectors $\widehat{V}_{v_1,\cdot}$ at depth ℓ satisfy $\max(1, \|\widehat{V}_{v_1,\cdot}\|_\infty) \leq 2r_\ell \max(1, \|x^{(\ell)}\|_\infty)$, where $r_\ell = 2^{K_{\ell+1}}$. Consequently, the integer height bound $H_\ell := 1 + \sum_{j=0}^{\ell-1}(K_{j+1} + 1)$ satisfies $\log_2 \max(1, \|x^{(\ell)}\|_\infty) \leq H_\ell$ and

$$H_\ell = 1 + \sum_{j=0}^{\ell-1}(K_{j+1} + 1) \leq 1 + K_0 + 2\ell = O(\log n). \tag{5}$$

Because $K_{\ell+1} = \lceil K_\ell/2 \rceil$, the recursion reaches a leaf $R_L \leq 4p_L^2$ in $L \leq \lceil \log_2 K_0 \rceil$ levels. Moreover, $H_\ell < p_0 + \sum_{j=0}^{\ell-1}(2K_j + 12) = p_\ell$ holds for all $K_0 \geq 2$ and $\ell \geq 0$, so $p_\ell > H_\ell$ at every level under schedule (34). Every routed record at depth ℓ therefore has coordinatewise magnitude at most $2r_\ell 2^{H_\ell} = 2^{H_{\ell+1}} \leq 2^{p_{\ell+1}}$ and Euclidean modulus at most $\sqrt{2} \cdot 2^{H_{\ell+1}} < 2^{H_{\ell+1}+1}$.

The intermediate scalars inside the column solvers obey matching height bounds. Let $\mathcal{M}_Y := \max(1, \|\widehat{Y}\|_\infty)$ denote the routed column scale, for which the proof of Lemma 6.5 establishes

Table 2: Notation for dimensions, precision parameters, fixed-point grids, transforms, and kernel weights. These symbols are used throughout Sections 3–7 and Appendices A–C.

Symbol	Meaning
$n,\ b,\ N$	bit length of input integers $A, B \geq 0$, radix-2^b chunk width $b = \max(4, \lfloor \log_2 n \rfloor)$, and chunk count $N = \lceil n/b \rceil$
$R_0 = 2^{K_0},\ p_0$	root transform length ($2N \leq R_0 < 4N$) with $K_0 = \log_2 R_0$, and root target precision $p_0 = 2b + 2K_0 + 6$
$R_\ell = 2^{K_\ell} = t_{1,\ell} r_\ell$	transform length at recursion depth ℓ, with $K_\ell = \log_2 R_\ell$, $t_{1,\ell} = 2^{\lfloor K_\ell/2 \rfloor}$, $r_\ell = 2^{\lceil K_\ell/2 \rceil}$, and $\nu_\ell = r_\ell / t_{1,\ell} \in \{1, 2\}$
$k_{1,\ell},\ k_{2,\ell}$	$\log_2 t_{1,\ell}$ and $\log_2 r_\ell$ ($k_{1,\ell} + k_{2,\ell} = K_\ell$)
L	leaf recursion depth (first depth ℓ where $R_\ell \leq 4p_\ell^2$)
$p_\ell,\ p_{\mathrm{work},\ell},\ p_{\ell+1}$	target precision at depth ℓ, working cutoff $p_{\mathrm{work},\ell} = p_\ell + K_\ell + 6$, and child target precision $p_{\ell+1} = p_\ell + 2K_\ell + 12$
$\widetilde{\mathbb{R}}_q,\ \widetilde{\mathbb{C}}_q,\ \mathcal{Q}_q$	real and complex dyadic grids $2^{-q}\mathbb{Z}$ and $2^{-q}(\mathbb{Z} + i\mathbb{Z})$ ($q = p + 2$ for target precision p), and coordinatewise nearest-grid rounding
$q_{\mathrm{child}},\ u = 2^{-p_{\ell+1}}$	child fractional grid parameter $q_{\mathrm{child}} = p_{\ell+1} + 2$ and unit accuracy scale of child calls (the letter u also denotes the one-dimensional frequency index $u = u_2 + r u_1$ of (9); the meaning is fixed by context: u_2 indexes columns, $u = 2^{-p_{\ell+1}}$ is the accuracy scale)
$H_\ell,\ w_{\mathrm{coord},\ell},\ w_\ell$	integer coordinate height bound at depth ℓ, signed coordinate bit width, and complex record width $w_\ell = 2w_{\mathrm{coord},\ell}$
$\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1),\ G_{t_1}$	negacyclic polynomial ring of rank r and normalized length-t_1 polynomial transform over $\mathcal{R}_r$
$P_{j_1}(y),\ Q_{v_1}(y)$	input and output polynomials in $\mathcal{R}_r$ of the polynomial transform G_{t_1}
$F_R,\ F_{t_1,r}$	normalized positive one-dimensional DFT of length R and two-dimensional DFT of shape $t_1 \times r$
$g(t),\ \hat{g}(\xi),\ \alpha_\ell$	windowed sinc-Gaussian kernel $\mathrm{sinc}(t) e^{-\pi t^2/\alpha_\ell^2}$, its continuous Fourier transform, and $\alpha_\ell = 2\sqrt{p_{\mathrm{work},\ell} \ln 2/\pi}$
$m_{1,\ell},\ W_\ell,\ \tilde{W}_\ell$	spatial window half-width, raw tap count $W_\ell = 2m_{1,\ell} + 1$, and modulo-r_ℓ folded filter length $\tilde{W}_\ell = \min(W_\ell, r_\ell)$
$c_0(u_2),\ c_1(u_2),\ M(u_2)$	retained Poisson alias weights $\hat{g}(u_2/r)$ and $\hat{g}(1 - u_2/r)$ in column u_2, and $M(u_2) = \max(c_0(u_2), c_1(u_2))$
$\rho(u_2),\ \theta(u_2),\ A_{\mathrm{N}}$	non-Nyquist recurrence ratio $-\min(c_0, c_1)/M(u_2)$, cyclic closure scalar $(1 - \rho(u_2)^{t_1})^{-1}$, and Nyquist spatial alternating sum
$\mathcal{M}_Y$	$\max(1, \|\widehat{Y}\|_\infty)$ for the routed child output $\widehat{Y}$ of a column (Lemma 5.2)

$\mathcal{M}_Y \leq 2\max(1, \|x^{(\ell)}\|_\infty) \leq 2^{H_\ell+1}$. Including the rounding terms of Lemma 5.2, the prefix-sweep scalars $\tilde{P}_i$ and Z_i^{comp} inside `CyclicSolve` have magnitude strictly below $2.37\, t_{1,\ell}\mathcal{M}_Y$. Since $t_{1,\ell} \leq r_\ell$, these scalars are bounded by $4.74\, r_\ell 2^{H_\ell} < 4 \cdot 2^{H_{\ell+1}} = 2^{H_{\ell+1}+2}$. Inside `NyquistSolve`, the prefix sums satisfy $|\tilde{P}_i| \leq 2.0002\, t_{1,\ell}\mathcal{M}_Y$ and the correction satisfies $|\tilde{\delta}| \leq 1.0001\, t_{1,\ell}\mathcal{M}_Y + |A_{\mathrm{N}}|/t_{1,\ell} + 2^{-p_{\ell+1}}$. Hence the outputs $Z_i^{\mathrm{comp}} = \tilde{P}_i + (-1)^i \tilde{\delta}$ satisfy $|Z_i^{\mathrm{comp}}| \leq 3.0003\, t_{1,\ell}\mathcal{M}_Y + |A_{\mathrm{N}}|/t_{1,\ell} + 2^{-p_{\ell+1}}$. Because $|A_{\mathrm{N}}| \leq \|x^{(\ell)}\|_\infty \leq 2^{H_\ell}$ by (25), we obtain $|Z_i^{\mathrm{comp}}| < 6.1\, r_\ell 2^{H_\ell} < 2^{H_{\ell+1}+2}$ as well.

Tape routing, matrix transposition, and merge sort require uniform fixed-width records. At recursion depth ℓ, every complex scalar on $\widetilde{\mathbb{C}}_q$ with $q \leq q_{\mathrm{child}} = p_{\ell+1} + 2$ is padded by two's-complement sign extension and low-bit zero-filling to the uniform record width

$$w_\ell := 2w_{\mathrm{coord},\ell}, \qquad w_{\mathrm{coord},\ell} := q_{\mathrm{child}} + H_{\ell+1} + 3 = \Theta(p_{\ell+1}) = \Theta(\log n) \tag{6}$$

bits. Each record consists of two $w_{\mathrm{coord},\ell}$-bit signed two's-complement integers, where the $+3$ headroom bits accommodate the sign bit, the $\sqrt{2}$ Euclidean-to-coordinatewise margin, and the $2^{H_{\ell+1}+2}$ prefix-sweep bound. Re-padding the R_0 active records when moving between depth ℓ and depth $\ell + 1$ is a single sequential scan costing $O(R_0 w_{\ell+1}) = O(n)$ Turing machine steps per level. Explicit bit-width bounds for all intermediate scratch and accumulator registers are listed in

Appendix A.2.

Rounded arithmetic primitives. Addition and subtraction on $\widetilde{\mathbb{C}}_q$ are exact and take $O(q+H+1)$ bit operations. For $a, b \in \widetilde{\mathbb{C}}_q$ or for $a \in \widetilde{\mathbb{R}}_q$ and $b \in \widetilde{\mathbb{C}}_q$, the rounded product $\mathrm{RM}_q(a,b) := \mathcal{Q}_q(ab)$ first multiplies the scaled integer numerators $N_a = 2^q a$ and $N_b = 2^q b$ exactly. This step uses four real integer multiplications and two additions for complex-by-complex products, or two real integer multiplications for real-by-complex products. It then rounds each coordinate once from fractional precision $2q$ to q, incurring error $|\mathrm{RM}_q(a,b) - ab| \le \frac{\sqrt{2}}{2} 2^{-q}$. For $z \in \widetilde{\mathbb{C}}_q$ and $d \in \widetilde{\mathbb{R}}_q \setminus \{0\}$, the rounded quotient $\mathrm{RD}_q(z,d) := \mathcal{Q}_q(z/d)$ shifts $N_z := 2^q z \in \mathbb{Z}[i]$ by 2^q to form $2^{2q} z$. Integer Euclidean division with remainder by $N_d := 2^q d \in \mathbb{Z} \setminus \{0\}$ then rounds $2^q(z/d)$ onto $\widetilde{\mathbb{C}}_q$ with error $|\mathrm{RD}_q(z,d) - z/d| \le \frac{\sqrt{2}}{2} 2^{-q}$.

3.3 Elementary function evaluation and shared level tables

On a Turing machine we cannot assume unit-cost real-RAM oracles for constants and functions such as π, $\ln 2$, $\sqrt{\cdot}$, exp, sin, cos, and erf. Lemma 3.2 shows that every shared table entry can be evaluated deterministically in polynomial-in-precision time in two steps: first sum truncated elementary series on a high-precision guard grid, and then round once onto the target grid. That extra guard margin makes every real and complex table entry faithful to within one unit in the last place (1 ulp); Appendix A.1 proves the series and composite error bounds.

Lemma 3.2 (Fixed-point evaluation of elementary functions and faithful tables)**.** *Let $s \ge 8$ be a target fractional precision and let $D := \max(s, K_0) = O(\log n)$. Work on the internal dyadic grid $\widetilde{\mathbb{R}}_S$ at precision $S := s + 15D + 64 = \Theta(D)$ with truncated series of $N_{\mathrm{terms}} := 2S + 36D = \Theta(D)$ terms. Assume both the exact real argument x and the grid argument $\tilde{x} \in \widetilde{\mathbb{R}}_S$ lie in the stated domain, with $2^S \tilde{x}$ a signed integer of bit-length at most $S + 2D + 8 = O(D)$. Assume further that $|\tilde{x} - x| \le \delta_{\mathrm{in}} \le 2^{-s-20}$, with $\delta_{\mathrm{in}} = 0$ for exact dyadic rational arguments $t = d + j_1/t_{1,\ell} \in \widetilde{\mathbb{R}}_{K_0} \subseteq \widetilde{\mathbb{R}}_S$. Each primitive (i)–(v) uses intermediate integers of bit-length at most $2S + 4D + 16 = O(D)$ and outputs $\tilde{y} \in \widetilde{\mathbb{R}}_S$ satisfying $|\tilde{y} - f(\tilde{x})| < 2^{-s-D-23} < 2^{-s-30}$ and $|\tilde{y} - f(x)| < 2^{-s-D-23} + L_f \delta_{\mathrm{in}}$ (where $L_{\sqrt{\cdot}} \le 2^{D/2-1}$ on $[2^{-D}, 2^{2D}]$, $L_{\exp}, L_{\sin}, L_{\cos} \le 1$, $L_{\mathrm{erf}} = 2/\sqrt{\pi} < 1.13$, and $L_{\mathrm{sinc}} = 4/\pi < 1.28$ on $[-1/2, 1/2]$) in $O(D^3)$ schoolbook bit operations. Every $\sqrt{\cdot}$ evaluation of the algorithm has its argument in $[1, 2^{2D}]$, where $L_{\sqrt{\cdot}} \le \frac{1}{2}$ (Appendix A.1). To evaluate any real or complex table expression of our algorithm, we work at target precision $s := q + 10$. Here $q = q_{\mathrm{child}} \ge p_{\mathrm{work},\ell}$ for the α_ℓ-dependent tables of a non-leaf level, so $p_{\mathrm{work},\ell} < s \le D$ there. The leaf twiddle tables use $q = q_b$ and involve no α_ℓ. This evaluation produces $\tilde{v} \in \widetilde{\mathbb{R}}_S$ (resp. $\widetilde{\mathbb{C}}_S$) within $< 2^{-s-20} < 2^{-q-10}$ of the exact value v. Rounding once via $\bar{v} := \mathcal{Q}_q(\tilde{v})$ therefore satisfies the 1-ulp faithful table rounding contract:*

$$|\bar{v} - v| \le \left(\frac{1}{2} + 2^{-10}\right) 2^{-q} < 2^{-q} \quad (\textit{real}), \qquad |\bar{v} - v| \le \sqrt{2}\left(\frac{1}{2} + 2^{-10}\right) 2^{-q} < \sqrt{2} \cdot 2^{-q} \quad (\textit{complex}). \tag{7}$$

On $\widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ ($q_{\mathrm{child}} = p_{\ell+1} + 2$, $u = 2^{-p_{\ell+1}}$), (7) gives $|\bar{v} - v| \le \frac{1}{4} u = 0.25u$ for real table entries and $|\bar{v} - v| \le \frac{\sqrt{2}}{4} u < 0.354u$ for complex table entries. The procedures are:

- *(i) Evaluate π via Machin's identity $\frac{\pi}{4} = 4\arctan(1/5) - \arctan(1/239)$ with alternating remainder $\le 5(2N_{\mathrm{terms}} + 1)^{-1} 5^{-(2N_{\mathrm{terms}}+1)}$ for $\pi/4$ (both tails), hence $\le 20(2N_{\mathrm{terms}} + 1)^{-1} 5^{-(2N_{\mathrm{terms}}+1)} < 2^{-S-16}$ for π, and $\ln 2 = 2\sum_{k=0}^{\infty} \frac{1}{(2k+1)3^{2k+1}}$ with geometric remainder $\le \frac{3}{4(2N_{\mathrm{terms}}+1)} 3^{-2N_{\mathrm{terms}}} < 2^{-S-16}$. Let $\tilde{\pi}, \tilde{\ell}_2 \in \widetilde{\mathbb{R}}_S$ denote these approximations and $\tilde{h} := \mathcal{Q}_S(\tilde{\pi}/2)$.*

(ii) Evaluate square roots $\sqrt{x}$ for $x \in [2^{-D}, 2^{2D}]$ via exact integer square root $\lfloor \sqrt{2^{2S}\tilde{x}} \rfloor$.

(iii) Evaluate $\exp(x)$ *for* $x \in [-\pi(D+2)^2, 0]$*, returning* 0 *when* $\tilde{x} < -(S+16)\tilde{\ell}_2$ *(since then* $e^{\tilde{x}} < 2^{-S-15}$*) and otherwise dividing* $-\tilde{x} = m\tilde{\ell}_2 + \tilde{r}$ *with* $m \in \{0, \ldots, S+16\}$*,* $\tilde{r} \in [0, \tilde{\ell}_2)$ *and evaluating* $2^{-m} \sum_{k=0}^{N_{\text{terms}}-1} (-\tilde{r})^k/k!$ *(remainder* $\le \tilde{r}^{N_{\text{terms}}}/N_{\text{terms}}! < 2^{-S-16}$*).*

(iv) Evaluate $\sin(x), \cos(x)$ *for* $|x| \le \pi(D+2)$ *via range reduction* $\tilde{x} = m\tilde{h} + \tilde{r}$ *with integer* $m := \lfloor \tilde{x}/\tilde{h} + 1/2 \rfloor$ *(*$|m| \le 2D+5$*) and* $|\tilde{r}| \le \tilde{h}/2 < 0.79 < 1$ *and alternating Taylor series with remainder* $\le |\tilde{r}|^{2N_{\text{terms}}}/(2N_{\text{terms}})! < 2^{-S-16}$ *(and for* $|t| \le 1/2$*, evaluate* $\operatorname{sinc}(t) = \sum_{k=0}^{\infty} (-1)^k \frac{(\pi t)^{2k}}{(2k+1)!}$ *directly by its* N_{terms}*-term truncation with alternating remainder* $\le \frac{(\pi/2)^{2N_{\text{terms}}}}{(2N_{\text{terms}}+1)!} < 2^{-S-16}$*, avoiding division by small* πt*).*

(v) Evaluate $\operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} e^{-z^2} \sum_{n=0}^{\infty} \frac{2^n z^{2n+1}}{1 \cdot 3 \cdots (2n+1)}$ *on* $z \in [0, 3\sqrt{D}]$*, returning* 1 *when* $\mathrm{RM}_S(\tilde{z}, \tilde{z}) > (s+D+24)\tilde{\ell}_2$ *(since then* $\operatorname{erfc}(\tilde{z}) < 2^{-s-D-23}$*) and otherwise summing the first* N_{terms} *series terms on* $\widetilde{\mathbb{R}}_S$*.*

Shared level tables and breadth-first tape layout. For a top-level multiplication of n-bit integers, the dimensions $R_\ell = 2^{K_\ell}$ and precisions $(p_\ell, p_{\text{work},\ell}, p_{\ell+1})$ at each depth $\ell \in \{0, \ldots, L\}$ depend only on (R_0, p_0) and ℓ. Before the recursive transforms begin, the algorithm precomputes one shared set of tables per level ℓ on a table tape T_{tab}. For each non-leaf level it writes the folded filter taps, their Kronecker-packed integers, the twist roots, and the column solver tuples $(\mu(u_2), \hat{\rho}(u_2), \hat{\theta}(u_2)) \in \widetilde{\mathbb{R}}^3_{q_{\text{child}}}$; at $\ell = L$ it writes the per-stage leaf twiddle lists. We have $t_{1,\ell}, r_\ell \le \sqrt{2R_0}$, $R_L \le 4p_L^2$, and $m_{1,\ell} \le p_L$ by Lemma 6.5. Applying Lemma 3.2 with $D = O(p_L) = O(\log n)$ therefore builds all shared level tables in

$$\mathcal{C}_{\text{tables}}(n) = O\big((L+1)(\sqrt{R_0}\, p_L^4 + p_L^5)\big) = O\big(\sqrt{n}(\log n)^{7/2} \log\log n\big) = o(n) \tag{8}$$

Turing machine steps.

The recursion tree is then executed breadth-first on $O(1)$ tapes. Besides the input and output tapes, the machine uses two alternating data tapes $T^{(0)}_{\text{data}}$ and $T^{(1)}_{\text{data}}$, four scratch tapes $T_1, \ldots, T_4$, and the table tape T_{tab}. It also maintains a level-segmented stack tape T_{stack} for each node's Nyquist alternating sum, together with dedicated work tapes for the nested inner multiplier. Total table replay and level-sweep coordination across all $L = O(\log\log n)$ levels take $O(n \log\log n) = o(n \log n)$ steps, as detailed in Appendix A.2.

3.4 Matrix transposition and odd-stride column routing

Sequential tape heads cannot jump across a two-dimensional array in unit time, so moving records between row order and column order must be scheduled explicitly. At each recursion level we use two routing primitives: transposing a $t_1 \times r$ matrix between row-major and column-major layouts, and permuting each column by an odd stride via bottom-up merge sort. Both primitives cost $O(t_1 r q(1 + \log_2 t_1))$ steps, which together with the polynomial transform forms the dominant $O(n \log n)$ movement work; Appendix B gives the proof by multi-pass split-and-interleave scans and column-parallel merge sort.

Lemma 3.3 (Multitape transposition and odd-stride column routing)**.** *Let* $t_1 \mid r$ *be powers of two* ($t_1 = 2^{k_1}$*,* $r = 2^{k_2}$*), and let each matrix entry be a uniform fixed-width record of* $w = \Theta(q)$ *bits with* $q \ge k_2 \ge k_1$*. On a deterministic multitape Turing machine with a constant number of tapes:*

(i) Transposing a $t_1 \times r$ matrix between row-major order (j_1, j_2) and column-major order (j_2, j_1), including de-interleaving $x \in \widetilde{\mathbb{C}}_q^{t_1 r}$ indexed by $j = j_1 + t_1 j_2$ into t_1 contiguous rows of length r or transposing $(\widehat{Z}_{u_1,u_2})$ from column-major order (u_2, u_1) back to natural frequency order $u = u_2 + r u_1$, takes $O\big(t_1 r q (1 + \log_2 t_1)\big)$ Turing machine steps.

(ii) Given t_1 child DFT output vectors $\widehat{C}_{v_1} \in \widetilde{\mathbb{C}}_q^r$ in row order $v_1 \in \{0, \dots, t_1 - 1\}$, routing all entries according to the odd-stride column permutation

$$(v_1, u_2) \longmapsto (u_1, u_2), \qquad u_1 := (2u_2 - 1)v_1 \bmod t_1,$$

into column-major order (u_2, u_1) takes $O\big(t_1 r q (1 + \log_2 t_1)\big)$ Turing machine steps.

3.5 Discrete Fourier transforms, two-call convolution, and polynomial transforms

We finally fix our Fourier transform normalizations and the multiplication-free polynomial transform. For $R \in \mathbb{Z}_{\geq 1}$, the normalized one-dimensional discrete Fourier transform $F_R \colon \mathbb{C}^R \to \mathbb{C}^R$ is defined for $u \in \{0, \dots, R-1\}$ by

$$(F_R x)_u := \frac{1}{R} \sum_{j=0}^{R-1} x_j \, e^{2\pi i j u / R}. \tag{9}$$

By the triangle inequality, $\|F_R x\|_\infty \leq \frac{1}{R} \sum_{j=0}^{R-1} |x_j| \leq \|x\|_\infty$. Under this normalization, $\big(F_R(\overline{F_R w})\big)_j = \frac{1}{R} \overline{w_j}$ holds for every $w \in \mathbb{C}^R$. Moreover, the cyclic convolution $(a \circledast b)_j := \sum_{k=0}^{R-1} a_k b_{(j-k) \bmod R}$ satisfies $F_R(a \circledast b) = R \cdot (F_R a) \odot (F_R b)$, where $\odot$ denotes pointwise multiplication.

Because the digit chunks of A and B are real, we pack them as the real and imaginary parts of a single complex vector $z = a + ib$, following a classical identity [Nus82]. Squaring $F_R z$ pointwise, conjugating, and applying F_R once more recovers $a \circledast b$ from the imaginary part, using two forward transforms instead of three.

Lemma 3.4 (Two-call real cyclic convolution)**.** *For any two real vectors $a, b \in \mathbb{R}^R$, set $z := a + ib \in \mathbb{C}^R$. Then the real cyclic convolution $a \circledast b \in \mathbb{R}^R$ is given in terms of two evaluations of F_R by*

$$a \circledast b = -\frac{R^2}{2} \operatorname{Im}\Big(F_R\big(\overline{(F_R z)^2}\big) \Big), \tag{10}$$

where $(F_R z)^2 := (F_R z) \odot (F_R z)$ denotes the pointwise square.

Proof. By linearity of F_R and the convolution theorem,

$$(F_R z)^2 = (F_R a + i F_R b)^2 = (F_R a \odot F_R a - F_R b \odot F_R b) + 2i (F_R a \odot F_R b) = \frac{1}{R} F_R(d - e + 2ic),$$

where $d := a \circledast a \in \mathbb{R}^R$, $e := b \circledast b \in \mathbb{R}^R$, and $c := a \circledast b \in \mathbb{R}^R$. Applying $F_R(\overline{F_R w}) = \frac{1}{R} \overline{w}$ to $w := \frac{1}{R}(d - e + 2ic)$ yields

$$F_R\big(\overline{(F_R z)^2}\big) = \frac{1}{R^2} \overline{d - e + 2ic} = \frac{1}{R^2}(d - e - 2ic),$$

because $d, e, c \in \mathbb{R}^R$ are real. Taking the imaginary part and multiplying by $-R^2/2$ recovers $c = a \circledast b$. □

For dimensions (t_1, r), the normalized two-dimensional discrete Fourier transform $F_{t_1,r}\colon \mathbb{C}^{t_1\times r} \to \mathbb{C}^{t_1\times r}$ is defined for $(u_1, u_2) \in \{0,\dots,t_1-1\}\times\{0,\dots,r-1\}$ by

$$(F_{t_1,r}U)_{u_1,u_2} := \frac{1}{t_1 r}\sum_{j_1=0}^{t_1-1}\sum_{j_2=0}^{r-1} U_{j_1,j_2}\, e^{2\pi i j_1 u_1/t_1}\, e^{2\pi i j_2 u_2/r}. \tag{11}$$

Let $r = 2^{k_2}$ and $t_1 = 2^{k_1}$ be powers of two with $t_1 \mid r$, and set $\nu := r/t_1 \in \{1, 2\}$. In the negacyclic quotient ring $\mathcal{R}_r := \mathbb{C}[y]/(y^r+1)$, the monomial $\omega_{t_1} := y^{2\nu}$ satisfies $\omega_{t_1}^{t_1/2} = y^{\nu t_1} = y^r \equiv -1 \pmod{y^r+1}$, so ω_{t_1} is a primitive t_1-th root of unity in $\mathcal{R}_r$. Following Nussbaumer [Nus80, Nus82], for $P = (P_0(y),\dots,P_{t_1-1}(y)) \in (\mathcal{R}_r)^{t_1}$ we define the normalized polynomial transform $G_{t_1}\colon (\mathcal{R}_r)^{t_1} \to (\mathcal{R}_r)^{t_1}$ for $v_1 \in \{0,\dots,t_1-1\}$ by

$$(G_{t_1}P)_{v_1}(y) := \frac{1}{t_1}\sum_{j_1=0}^{t_1-1} P_{j_1}(y)\, y^{2\nu j_1 v_1} \pmod{y^r+1}. \tag{12}$$

The next lemma shows that G_{t_1} evaluates this length-t_1 transform over $\mathcal{R}_r$ in $\log_2 t_1$ radix-2 butterfly stages, while keeping coordinate magnitudes bounded and accumulating at most $\log_2 t_1$ half-ulp roundings. Because every twiddle power $y^{2m} \bmod (y^r+1)$ merely rotates and negates polynomial coefficients, the butterflies perform zero scalar multiplications.

Lemma 3.5 (Radix-2 polynomial transform G_{t_1})**.** *Let $t_1 = 2^{k_1} \geq 2$ ($k_1 \geq 1$) and $r = 2^{k_2}$ with $t_1 \mid r$. The radix-2 decimation-in-time Cooley–Tukey factorization of G_{t_1} over $\mathcal{R}_r$ bit-reverses the t_1 input polynomials and executes $k_1 = \log_2 t_1$ butterfly stages. In stage $s \in \{1,\dots,k_1\}$, each butterfly updates a pair of polynomials $A(y), B(y) \in \widetilde{\mathbb{C}}_q[y]/(y^r+1)$ via*

$$A^{\mathrm{new}}(y) := \mathcal{Q}_q\!\left(\frac{1}{2}\big(A(y) + y^{2m}B(y)\big)\right), \qquad B^{\mathrm{new}}(y) := \mathcal{Q}_q\!\left(\frac{1}{2}\big(A(y) - y^{2m}B(y)\big)\right) \pmod{y^r+1}.$$

On a deterministic multitape Turing machine, for inputs on $\widetilde{\mathbb{C}}_q$ ($q = p_{\ell+1}+2 \geq k_2$) with coordinatewise magnitude $\|P\|_{\infty,\mathrm{coord}} \leq 2^H$ stored as uniform fixed-width records of width $w = \Theta(q+H+1)$, computing G_{t_1} performs zero scalar multiplications, takes $O(t_1 r(q+H+1)(1+\log_2 t_1))$ Turing machine steps, preserves the coordinatewise magnitude bound $\|\cdot\|_{\infty,\mathrm{coord}} \leq 2^H$ at every stage (while for any Euclidean input bound $\|P\|_\infty \leq V$, stage s satisfies $\|P^{(s)}\|_\infty \leq V + s\frac{\sqrt{2}}{2}2^{-q}$), and outputs $\widehat{Q} = (\widehat{Q}_0,\dots,\widehat{Q}_{t_1-1}) \in (\widetilde{\mathbb{C}}_q[y]/(y^r+1))^{t_1}$ in natural index order $v_1 \in \{0,\dots,t_1-1\}$ satisfying

$$\|\widehat{Q} - G_{t_1}P\|_\infty \leq k_1\frac{\sqrt{2}}{2}2^{-q} = k_1\frac{\sqrt{2}}{8}2^{-p_{\ell+1}} < 0.177\, k_1\, 2^{-p_{\ell+1}}. \tag{13}$$

Proof. First, apply the bit-reversal permutation to the $t_1 = 2^{k_1}$ input polynomials $P_0,\dots,P_{t_1-1}$, each stored as a contiguous block of r records of width w, in $k_1 = \log_2 t_1$ sequential split-and-concatenate passes. In pass $s \in \{1,\dots,k_1\}$, within each of the 2^{s-1} segments of $t_1/2^{s-1}$ polynomials, copy even-indexed polynomials to auxiliary tape T_1 and odd-indexed polynomials to T_2, and then concatenate T_1 and T_2. This costs $O(t_1 r w)$ steps per pass and $O(t_1 r w \log_2 t_1)$ steps in total.

Next, in stage $s \in \{1,\dots,k_1\}$ of decimation-in-time Cooley–Tukey, the tape consists of $t_1/2^s$ contiguous blocks of 2^s polynomials. Within each block, the first 2^{s-1} polynomials $(A_j(y))_{j=0}^{2^{s-1}-1}$ are paired with the second 2^{s-1} polynomials $(B_j(y))_{j=0}^{2^{s-1}-1}$ using twiddle exponent

$$\delta_j := 2m_j = 2\nu j \cdot 2^{k_1-s} \in \{0, 2, \dots, r-2\}, \qquad 0 \leq 2m_j \leq r - 2\nu 2^{k_1-s} < r,$$

since $j < 2^{s-1}$ and $\nu t_1 = r$. One linear scan of the block copies $(A_j)_{j=0}^{2^{s-1}-1}$ to T_1 and $(B_j)_{j=0}^{2^{s-1}-1}$ to T_2 in $O(2^s rw)$ steps. As j increments, we maintain $\delta_j \in \{0, \dots, r-2\}$ by adding the stage stride $2\nu \cdot 2^{k_1-s}$, costing $O(\log r) \le O(w)$ steps per polynomial.

For each pair j, multiplying $B_j(y) = \sum_{k_2=0}^{r-1} b_{k_2} y^{k_2}$ by $y^{\delta_j} \pmod{y^r+1}$ yields $\sum_{k_2=0}^{r-1} \tilde{b}_{k_2} y^{k_2}$ with

$$\tilde{b}_{k_2} = \begin{cases} -b_{k_2+r-\delta_j} & \text{for } 0 \le k_2 < \delta_j, \\ +b_{k_2-\delta_j} & \text{for } \delta_j \le k_2 < r. \end{cases}$$

To stream $(\tilde{b}_{k_2})_{k_2=0}^{r-1}$ in natural order on a Turing machine, we scan the first $r-\delta_j$ coefficient records $b_0, \dots, b_{r-\delta_j-1}$ of B_j on T_2 and copy them to T_3. A k_2-bit counter initialized to $r-\delta_j$ tracks this prefix in $O(r + \log r)$ total counter steps, leaving T_2 positioned at $b_{r-\delta_j}$, after which T_3 is rewound to b_0. We then scan $a_0, \dots, a_{r-1}$ on T_1 in lockstep with the δ_j negated records $-b_{r-\delta_j}, \dots, -b_{r-1}$ on T_2 followed by the $r-\delta_j$ records on T_3. During this scan we write $A_j^{\text{new}}(y)$ to the upper half of the block on the main tape and $B_j^{\text{new}}(y)$ to T_4, and finally copy T_4 to the lower half of the block. Each polynomial pair in stage s takes $O(rw + \log r) = O(rw)$ steps, and all k_1 stages take $O(t_1 rw(1 + \log_2 t_1))$ steps.

For the norm and error bounds, the signed cyclic shift $y^{2m_j} B_j(y) \pmod{y^r+1}$ is exact on $\widetilde{\mathbb{C}}_q$. For each real or imaginary coordinate $x, y \in \widetilde{\mathbb{R}}_q \cap [-2^H, 2^H]$, the exact average $\frac{1}{2}(x \pm y)$ lies in $[-2^H, 2^H]$. Since $\pm 2^H \in \widetilde{\mathbb{R}}_q$ are exact grid points and $\mathcal{Q}_q$ is monotone on $\mathbb{R}$, $\mathcal{Q}_q(\frac{1}{2}(x \pm y))$ stays in $[-2^H, 2^H]$, preserving $\|\cdot\|_{\infty,\text{coord}} \le 2^H$. Simultaneously, for each complex coefficient k_2, rounding $\frac{1}{2}(a_{k_2} \pm \tilde{b}_{k_2})$ via $\mathcal{Q}_q$ incurs Euclidean error at most $\frac{\sqrt{2}}{2} 2^{-q}$ by (4). Because the exact butterfly $(A, B) \mapsto \frac{1}{2}(A + y^{2m}B, A - y^{2m}B)$ is a contraction in the complex Euclidean ℓ_∞ norm, the stage-s array $P^{(s)}$ and forward error e_s relative to the exact stage-s array satisfy

$$\|P^{(s)}\|_\infty \le \|P^{(s-1)}\|_\infty + \frac{\sqrt{2}}{2} 2^{-q}, \qquad e_s \le e_{s-1} + \frac{\sqrt{2}}{2} 2^{-q},$$

giving $e_{k_1} \le k_1 \frac{\sqrt{2}}{2} 2^{-q} < 0.177\, k_1\, 2^{-p_{\ell+1}}$. □

4 The two-dimensional bridge on a power-of-two grid

This section establishes the two exact identities that reduce a one-dimensional Fourier transform F_R of power-of-two length $R = t_1 r$ to t_1 recursive transforms F_r on the same $t_1 \times r$ grid. Section 4.1 reduces the two-dimensional transform $F_{t_1,r}$ to one multiplication-free polynomial transform G_{t_1} and t_1 calls to F_r. Section 4.2 then proves the fractional-shift Poisson identity, which expresses $F_{t_1,r}$ of a row-spread input array as a column-wise alias sum of $F_R x$. Together, these identities eliminate both Chinese remaindering and grid-expanding resampling from the recursion of Theorem 1. Readers who saw the overview in Sections 2.2–2.3 and trust Proposition 4.1 and Theorem 4.2 may skip directly to Section 5.

4.1 Polynomial-transform reduction ($F_{t_1,r} \to t_1 F_r$)

We first show how to evaluate the two-dimensional transform $F_{t_1,r}$ without complex butterfly multiplications along the t_1-axis. The construction is due to Nussbaumer and Quandalle [NQ79, Nus82] and uses the standard negacyclic-to-cyclic twist $y \mapsto \zeta^{2u_2-1}$ of Section 2.3 [SS71, CF94].

Proposition 4.1 states that twisting each row by powers of $\zeta = e^{\pi i/r}$, applying the multiplication-free polynomial transform G_{t_1} over $\mathbb{C}[y]/(y^r+1)$, untwisting, running t_1 recursive transforms F_r, and permuting each column by the odd stride $2u_2 - 1$ recovers $F_{t_1,r}U$ exactly. We need this reduction

so that the t_1-axis costs zero scalar multiplications, leaving only the $2R$ twist multiplications of Steps 1 and 3 per call, plus the t_1 recursive calls of length r.

Proposition 4.1 (Polynomial-transform reduction; Nussbaumer–Quandalle [NQ79, Nus82])**.** *Let $t_1 \mid r$ be powers of two, $\nu = r/t_1 \in \{1, 2\}$, $R = t_1 r$, and $\zeta := e^{\pi i/r}$ (so $\zeta^r = -1$). Given any $U \in \mathbb{C}^{t_1 \times r}$, perform the following five steps:*

1. ***Pre-twist:*** *For $(j_1, k_2) \in \{0, \dots, t_1 - 1\} \times \{0, \dots, r - 1\}$, set $\tilde{U}_{j_1,k_2} := \zeta^{k_2} U_{j_1,k_2}$ and form $P_{j_1}(y) := \sum_{k_2=0}^{r-1} \tilde{U}_{j_1,k_2} y^{k_2} \in \mathcal{R}_r$.*
2. ***Polynomial transform:*** *Compute $Q = G_{t_1} P \in (\mathcal{R}_r)^{t_1}$ via (12), and write $Q_{v_1}(y) = \sum_{k_2=0}^{r-1} q_{v_1,k_2} y^{k_2}$ for its unique representative of degree $< r$.*
3. ***Post-twist:*** *For $(v_1, k_2) \in \{0, \dots, t_1 - 1\} \times \{0, \dots, r - 1\}$, set $\tilde{q}_{v_1,k_2} := \zeta^{-k_2} q_{v_1,k_2}$.*
4. ***Recursive 1D DFTs:*** *For each $v_1 \in \{0, \dots, t_1 - 1\}$, compute the 1D DFT $\hat{q}_{v_1} := F_r(\tilde{q}_{v_1,\cdot}) \in \mathbb{C}^r$.*
5. ***Odd-stride permutation:*** *For each $v_1 \in \{0, \dots, t_1 - 1\}$ and $u_2 \in \{0, \dots, r - 1\}$, set*

$$u_1 := (2u_2 - 1)v_1 \bmod t_1, \qquad Y_{u_1,u_2} := (\hat{q}_{v_1})_{u_2}. \tag{14}$$

Then $Y_{u_1,u_2} = (F_{t_1,r} U)_{u_1,u_2}$ for all $(u_1, u_2) \in \{0, \dots, t_1 - 1\} \times \{0, \dots, r - 1\}$.

Proof. We include the short proof in our normalization. Fix any $u_2 \in \{0, \dots, r-1\}$. Because $2u_2 - 1$ is odd and $\zeta^r = e^{\pi i} = -1$, the complex number $\xi_{u_2} := \zeta^{2u_2-1}$ satisfies

$$\xi_{u_2}^r = (\zeta^r)^{2u_2-1} = (-1)^{2u_2-1} = -1, \quad \text{so} \quad \xi_{u_2}^r + 1 = 0.$$

Therefore, the polynomial evaluation map

$$\phi_{u_2} \colon \mathbb{C}[y]/(y^r + 1) \longrightarrow \mathbb{C}, \qquad H(y) \bmod (y^r + 1) \longmapsto H(\xi_{u_2})$$

is a well-defined ring homomorphism.

Fix $v_1 \in \{0, \dots, t_1 - 1\}$ and $u_2 \in \{0, \dots, r - 1\}$. Since $\zeta^{-k_2} e^{2\pi i k_2 u_2/r} = e^{-\pi i k_2/r} e^{2\pi i k_2 u_2/r} = \zeta^{(2u_2-1)k_2} = \xi_{u_2}^{k_2}$, Step 4 evaluates to

$$(\hat{q}_{v_1})_{u_2} = \frac{1}{r} \sum_{k_2=0}^{r-1} \tilde{q}_{v_1,k_2} e^{2\pi i k_2 u_2/r} = \frac{1}{r} \sum_{k_2=0}^{r-1} q_{v_1,k_2} \xi_{u_2}^{k_2} = \frac{1}{r} Q_{v_1}(\xi_{u_2}) = \frac{1}{r} \phi_{u_2}(Q_{v_1}(y)).$$

Because ϕ_{u_2} is a ring homomorphism on $\mathcal{R}_r$, applying ϕ_{u_2} directly to the sum defining $Q_{v_1}(y)$ in Step 2 gives

$$(\hat{q}_{v_1})_{u_2} = \frac{1}{t_1 r} \sum_{j_1=0}^{t_1-1} \phi_{u_2}\big(P_{j_1}(y)\big)\, \phi_{u_2}\big(y^{2\nu j_1 v_1}\big) = \frac{1}{t_1 r} \sum_{j_1=0}^{t_1-1} \left(\sum_{k_2=0}^{r-1} \zeta^{k_2} U_{j_1,k_2}\, \zeta^{(2u_2-1)k_2} \right) \zeta^{2\nu(2u_2-1)j_1 v_1}.$$

Noting that $\zeta^{k_2} \zeta^{(2u_2-1)k_2} = \zeta^{2u_2 k_2} = e^{2\pi i k_2 u_2/r}$ and $\zeta^{2\nu} = e^{2\pi i (r/t_1)/r} = e^{2\pi i/t_1}$, this becomes

$$(\hat{q}_{v_1})_{u_2} = \frac{1}{t_1 r} \sum_{j_1=0}^{t_1-1} \sum_{k_2=0}^{r-1} U_{j_1,k_2}\, e^{2\pi i k_2 u_2/r}\, e^{2\pi i j_1 ((2u_2-1)v_1)/t_1}.$$

Since $e^{2\pi i/t_1}$ is t_1-periodic in the exponent, setting $u_1 := (2u_2 - 1)v_1 \bmod t_1$ gives $(\hat{q}_{v_1})_{u_2} = (F_{t_1,r} U)_{u_1,u_2}$. Finally, since $2u_2 - 1$ is odd and t_1 is a power of two, $\gcd(2u_2 - 1, t_1) = 1$, so $v_1 \mapsto (2u_2 - 1)v_1 \bmod t_1$ is a bijection on $\{0, \dots, t_1 - 1\}$ for every u_2. □

4.2 Fractional-shift Poisson identity ($F_{t_1 r} \leftrightarrow F_{t_1,r}$)

We next connect the one-dimensional transform $F_{t_1 r}x$ to the two-dimensional transform $F_{t_1,r}$ on the same $t_1 \times r$ grid. Convolving shifted samples with a localized smooth kernel and expanding the resulting alias sum via Poisson summation is the classical gridding mechanism of the nonuniform discrete Fourier transform [DR93, PST01, GL04, BMK19]. The same Poisson-summation principle underlies the Gaussian resampling step of Harvey and van der Hoeven [HvdH21]. Because $F_{t_1 r}$ and $F_{t_1,r}$ act on the same $t_1 \times r$ grid and the fractional shift j_1/t_1 is constant across row j_1, spreading each row requires no grid oversampling.

Theorem 4.2 gives an exact formula for $F_{t_1,r}U^\circ$ when each row j_1 of the input is convolved cyclically modulo r with the shifted sinc-Gaussian kernel $g(\cdot + j_1/t_1)$. For every column u_2, the two-dimensional transform equals an infinite sum of shifted entries of the target one-dimensional transform $F_R x$ weighted by $\hat{g}(u_2/r - m)$. We need this identity because Section 5 shows that all weights with $m \notin \{0, 1\}$ are negligibly small, turning each column into a two-term linear system for $F_R x$. That system is invertible on every column except the Nyquist column $u_2 = r/2$, where the exact alternating sum of the input closes it.

Theorem 4.2 (Fractional-shift Poisson identity)**.** *Let $R = t_1 r$ with $t_1, r \in \mathbb{Z}_{\geq 1}$, and for $\alpha > 0$ define the windowed sinc-Gaussian kernel*

$$g(t) := \operatorname{sinc}(t)\, e^{-\frac{\pi}{\alpha^2}t^2} = \frac{\sin(\pi t)}{\pi t}\, e^{-\frac{\pi}{\alpha^2}t^2} \quad (g(0) := 1), \tag{15}$$

whose continuous Fourier transform $\hat{g}(\xi) := \int_{-\infty}^{\infty} g(t) e^{-2\pi i \xi t}\, dt$ is

$$\hat{g}(\xi) = \int_{-1/2}^{1/2} \alpha\, e^{-\pi\alpha^2(\xi-\xi')^2}\, d\xi' = \frac{1}{2}\left[\operatorname{erf}\left(\sqrt{\pi}\alpha\left(\xi + \frac{1}{2}\right)\right) - \operatorname{erf}\left(\sqrt{\pi}\alpha\left(\xi - \frac{1}{2}\right)\right)\right]. \tag{16}$$

For any input vector $x \in \mathbb{C}^R$, define the ideal spread array $U^\circ \in \mathbb{C}^{t_1 \times r}$ by convolving each row $j_1 \in \{0, \ldots, t_1 - 1\}$ cyclically along $j_2 \in \{0, \ldots, r-1\}$ with the real r-periodic filter $h^{(j_1),\mathrm{per}}$:

$$U^\circ_{j_1,k_2} := \sum_{j_2=0}^{r-1} x(j_1 + t_1 j_2)\, h^{(j_1),\mathrm{per}}_{(j_2-k_2) \bmod r}, \qquad h^{(j_1),\mathrm{per}}_d := \sum_{a\in\mathbb{Z}} g\left(d + \frac{j_1}{t_1} - ar\right). \tag{17}$$

Then for all $(u_1, u_2) \in \{0, \ldots, t_1 - 1\} \times \{0, \ldots, r-1\}$, the 2D DFT $F_{t_1,r}U^\circ$ and the 1D DFT $F_R x$ satisfy the exact identity

$$\left(F_{t_1,r}U^\circ\right)_{u_1,u_2} = \sum_{m\in\mathbb{Z}} \hat{g}\left(\frac{u_2}{r} - m\right) \left(F_R x\right)_{u_2 + r((u_1-m) \bmod t_1)}. \tag{18}$$

Proof idea. Unfolding the modulo-r periodization turns each row's cyclic filter sum into a single sum over all integers $k \in \mathbb{Z}$. Applying one-dimensional Poisson summation along each row and combining the resulting phase factors modulo 1 recovers the mixed-radix one-dimensional transform $F_R x$.

Proof. The function sinc is smooth with $|\operatorname{sinc}^{(k)}| \leq \pi^k$ on $\mathbb{R}$ for every k, being the inverse Fourier transform of the bounded, compactly supported indicator $\mathbf{1}_{[-1/2,1/2]}$. Multiplying sinc by the Gaussian $e^{-\pi t^2/\alpha^2}$ therefore yields a Schwartz function $g \in \mathcal{S}(\mathbb{R})$, so the periodized sum defining $h^{(j_1),\mathrm{per}}_d$ converges absolutely. Expanding $F_{t_1,r}U^\circ$ gives

$$\left(F_{t_1,r}U^\circ\right)_{u_1,u_2} = \frac{1}{t_1 r} \sum_{j_1=0}^{t_1-1} e^{2\pi i j_1 u_1/t_1} \sum_{j_2=0}^{r-1} x(j_1 + t_1 j_2) \left(\sum_{k_2=0}^{r-1} \sum_{a\in\mathbb{Z}} g\left(j_2 + \frac{j_1}{t_1} - (k_2 + ar)\right) e^{2\pi i k_2 u_2/r}\right).$$

Because $e^{2\pi i(k_2+ar)u_2/r} = e^{2\pi i k_2 u_2/r}$ for all $a \in \mathbb{Z}$, as k_2 ranges over $\{0, \dots, r-1\}$ and a ranges over $\mathbb{Z}$, the integer $k := k_2 + ar$ ranges over $\mathbb{Z}$ bijectively. Hence the inner sum equals

$$S(j_1, j_2, u_2) := \sum_{k\in\mathbb{Z}} g(z-k)e^{2\pi i k u_2/r}, \qquad \text{where } z := j_2 + \frac{j_1}{t_1}.$$

In $L^2(\mathbb{R})$ we have $\mathcal{F}\{\mathrm{sinc}\}(\xi) = \mathbf{1}_{[-1/2,1/2]}(\xi)$, while $\mathcal{F}\{e^{-\pi t^2/\alpha^2}\}(\xi) = \alpha e^{-\pi\alpha^2\xi^2}$. Although sinc lies in $L^2(\mathbb{R})$ rather than $L^1(\mathbb{R})$, the convolution identity $\mathcal{F}(\mathrm{sinc}\cdot\phi) = \mathcal{F}(\mathrm{sinc}) * \mathcal{F}(\phi)$ holds for every Schwartz function ϕ. Both sides of (16) are continuous, so (16) holds pointwise. Fix $z \in \mathbb{R}$ and $\xi := u_2/r$, and define $f(t) := g(z-t)e^{2\pi i\xi t} \in \mathcal{S}(\mathbb{R})$. Substituting $s = z - t$ gives the continuous Fourier transform of f at frequency $m \in \mathbb{Z}$:

$$\hat{f}(m) = \int_{-\infty}^{\infty} g(z-t)e^{2\pi i(\xi-m)t}\,dt = e^{2\pi i(\xi-m)z}\int_{-\infty}^{\infty} g(s)e^{-2\pi i(\xi-m)s}\,ds = \hat{g}(\xi-m)e^{2\pi i(\xi-m)z}.$$

By the one-dimensional Poisson summation formula $\sum_{k\in\mathbb{Z}} f(k) = \sum_{m\in\mathbb{Z}} \hat{f}(m)$, we obtain

$$S(j_1, j_2, u_2) = \sum_{m\in\mathbb{Z}} \hat{g}\Big(\frac{u_2}{r} - m\Big)\, e^{2\pi i\left(j_2+\frac{j_1}{t_1}\right)\left(\frac{u_2}{r}-m\right)}.$$

Since $j_2, m \in \mathbb{Z}$, $e^{-2\pi i j_2 m} = 1$. For each $m \in \mathbb{Z}$, let $\tilde{u}_1 := (u_1 - m) \bmod t_1 \in \{0, \dots, t_1 - 1\}$ so that $u_1 - m \equiv \tilde{u}_1 \pmod{t_1}$ and $e^{2\pi i j_1(u_1-m)/t_1} = e^{2\pi i j_1\tilde{u}_1/t_1}$. Then

$$\frac{j_1\tilde{u}_1}{t_1} + \frac{\left(j_2+\frac{j_1}{t_1}\right)u_2}{r} = \frac{j_1 r\tilde{u}_1 + (j_1+t_1j_2)u_2}{t_1 r} \equiv \frac{(j_1+t_1j_2)(u_2+r\tilde{u}_1)}{R} \pmod 1,$$

because $t_1 j_2 r\tilde{u}_1/(t_1 r) = j_2\tilde{u}_1 \in \mathbb{Z}$. As (j_1, j_2) ranges over $\{0, \dots, t_1-1\} \times \{0, \dots, r-1\}$, $j := j_1 + t_1 j_2$ ranges over $\{0, \dots, R-1\}$, so the sum over (j_1, j_2) equals $(F_R x)_{u_2+r\tilde{u}_1}$, proving (18). □

Theorem 4.2 holds for arbitrary positive integers $t_1, r \geq 1$; for the remainder of the paper, $t_1 \mid r$ are powers of two with $t_1 \geq 2$.

5 Sinc-Gaussian deconvolution and column solvers

This section shows how to invert the column-wise alias mixture produced by the fractional-shift Poisson identity (18). Section 5.1 proves that $\hat{g}$, the Fourier transform of the windowed sinc-Gaussian kernel, concentrates almost all of its mass on the two adjacent terms $m \in \{0, 1\}$. Retaining those two terms yields a well-conditioned cyclic bidiagonal system on every column $u_2 \neq r/2$, and a rank-deficient sum at the Nyquist column $u_2 = r/2$ that is closed by an exact alternating sum of the input. Section 5.2 specifies the linear-time fixed-point solvers `CyclicSolve` and `NyquistSolve` and records their forward rounding bound. Readers who saw the overview in Section 2.4 and trust Theorem 5.1 and Lemma 5.2 may skip directly to Section 6.

5.1 Passband, stopband, conditioning, and the Nyquist identity

We begin by analyzing the continuous Fourier transform $\hat{g}$ of the windowed sinc-Gaussian kernel across all columns $u_2 \in \{0, \dots, r-1\}$.

Theorem 5.1 establishes three properties for any working cutoff $p_{\mathrm{work}} \geq 8$. In filter language, the two retained weights $c_0(u_2), c_1(u_2)$ form the passband and the dropped aliases $m \notin \{0, 1\}$ form the

stopband. First, the dominant weight $M(u_2) = \max(c_0(u_2), c_1(u_2))$ stays above $1/2 - 0.08 \cdot 2^{-4p_{\text{work}}}$, while all dropped aliases $m \notin \{0, 1\}$ sum to less than $0.120 \cdot 2^{-p_{\text{work}}}$. Second, every non-Nyquist column $u_2 \neq r/2$ has inverse ℓ_∞ norm below $2.36\, t_1$. Third, at $u_2 = r/2$ an exact multiplication-free input sum A_{N} restores invertibility with norm below $1.0000001\, t_1$. We need these bounds so that undoing the row spread to working precision loses only $O(\log t_1)$ bits per recursion level rather than a constant fraction of the working precision. What later sections use from Theorem 5.1 are the rounded-up stopband bound $0.15 \cdot 2^{-p_{\text{work}}}$ (in the proof of Lemma 6.5), the uniform inverse norm bound $2.36\, t_1$, and the Nyquist identity (25).

Theorem 5.1 (Two-term deconvolution and the Nyquist identity)**.** *Let $t_1 \mid r$ be powers of two with $t_1 \geq 2$ (so t_1 is even) and $t_1 \in \{r, r/2\}$, and let $R = t_1 r$. For a working cutoff parameter $p_{\text{work}} \in \mathbb{Z}_{\geq 8}$, set $\alpha := 2\sqrt{\frac{p_{\text{work}} \ln 2}{\pi}}$ (so $\pi\alpha^2 = 4p_{\text{work}} \ln 2$). For each fixed column $u_2 \in \{0, \dots, r-1\}$, define $y := u_2/r \in [0, 1)$, $Y^{\circ}_{u_1} := (F_{t_1,r} U^{\circ})_{u_1,u_2}$, $Z_{u_1} := (F_R x)_{u_2 + r u_1}$, and*

$$c_0(u_2) := \hat{g}(y), \qquad c_1(u_2) := \hat{g}(1-y) = \hat{g}(y-1), \qquad M(u_2) := \max\big(c_0(u_2), c_1(u_2)\big). \tag{19}$$

Then:

1. ***Partition of unity, passband, and telescoping stopband alias:*** *For every $u_2 \in \{0, \dots, r-1\}$, $\sum_{m \in \mathbb{Z}} \hat{g}(y - m) = 1$, the dominant coefficient $M(u_2)$ satisfies*

$$\frac{1}{2} - 0.08 \cdot 2^{-4p_{\text{work}}} \leq \frac{1}{2}\operatorname{erf}(\sqrt{\pi}\alpha) = \hat{g}\left(\frac{1}{2}\right) \leq M(u_2) < 1 \qquad (M(u_2) > 0.49999999 \text{ for } p_{\text{work}} \geq 8), \tag{20}$$

and the excluded Poisson stopband aliases ($m \notin \{0, 1\}$) telescope in closed form to

$$\begin{aligned} \Psi(y) := \sum_{m \notin \{0,1\}} \hat{g}(y - m) &= \frac{1}{2}\operatorname{erfc}\left(\sqrt{\pi}\alpha\left(y + \frac{1}{2}\right)\right) + \frac{1}{2}\operatorname{erfc}\left(\sqrt{\pi}\alpha\left(\frac{3}{2} - y\right)\right) \\ &\leq \Psi(0) < 0.120 \cdot 2^{-p_{\text{work}}} < 0.15 \cdot 2^{-p_{\text{work}}}. \end{aligned} \tag{21}$$

2. ***Cyclic bidiagonal system for $u_2 \neq r/2$:*** *For all $u_2 \neq r/2$ and $t_1 \in \{r, r/2\}$, let $a := \min(c_0, c_1)/M(u_2) \in (0, 1)$ and $\rho := -a \in (-1, 0)$. Because $t_1 \geq 2$ is even, $\rho^{t_1} = a^{t_1}$, and*

$$a^{t_1} \leq e^{-1.9999\alpha t_1 |1/2 - y|} \leq e^{-0.99995\alpha} < 0.1400, \qquad \Gamma(u_2) := M(u_2)(1 - a^{t_1}) > 0.424. \tag{22}$$

The two-term cyclic bidiagonal operator $(BZ)_{u_1} := c_0(u_2) Z_{u_1} + c_1(u_2) Z_{(u_1 - 1) \bmod t_1} = Y_{u_1}$ ($u_1 \in \mathbb{Z}_{t_1}$) is invertible with exact closed form

$$Z_{u_1} = \begin{cases} \dfrac{1}{c_0(1 - \rho^{t_1})} \displaystyle\sum_{s=0}^{t_1 - 1} \rho^s Y_{(u_1 - s) \bmod t_1}, & \text{if } c_0 > c_1 \ (u_2 < r/2), \\[2ex] \dfrac{1}{c_1(1 - \rho^{t_1})} \displaystyle\sum_{s=0}^{t_1 - 1} \rho^s Y_{(u_1 + 1 + s) \bmod t_1}, & \text{if } c_1 > c_0 \ (u_2 > r/2), \end{cases} \tag{23}$$

and its $\ell_\infty \to \ell_\infty$ inverse operator norm satisfies

$$\|B^{-1}\|_{\infty \to \infty} = \frac{1}{|c_0(u_2) - c_1(u_2)|} = \frac{1 + a + \cdots + a^{t_1 - 1}}{M(u_2)(1 - a^{t_1})} \leq \frac{t_1}{\Gamma(u_2)} < 2.36\, t_1. \tag{24}$$

3. ***Nyquist alternating-sum identity at*** $u_2 = r/2$*: At the Nyquist column* $u_2 = r/2$ *(*$y = 1/2$*), where* $c := c_0(r/2) = c_1(r/2) = \frac{1}{2}\operatorname{erf}(\sqrt{\pi}\alpha) \in (0.49999999, 1/2)$*, the alternating sum of* $Z_{u_1} = (F_R x)_{r/2+ru_1}$ *is given in terms of the input vector* x *with zero scalar multiplications by*

$$A_{\mathrm{N}} := \sum_{u_1=0}^{t_1-1} (-1)^{u_1} Z_{u_1} = \frac{i}{r} \sum_{j_2=0}^{r-1} (-1)^{j_2} x\Big(\frac{t_1}{2} + t_1 j_2\Big). \tag{25}$$

Replacing the redundant equation at $u_1 = 0$ *by* (25) *yields a uniquely solvable linear system whose inverse has* ℓ_∞ *operator norm* $\frac{t_1-1}{2c} < 1.0000001\, t_1 < 2.36\, t_1$ *on* $(Y_1, \ldots, Y_{t_1-1})$ *and* $\frac{1}{t_1}$ *on* A_{N}.

Proof idea. Part 1 tiles $\mathbb{R}$ by unit intervals to sum the Gaussian integrals to 1 and checks monotonicity of $\hat{g}$ on $[0, \infty)$ for the passband lower bound. It then telescopes the complement of $m \in \{0, 1\}$ into two complementary error-function tails maximized at $y \in \{0, 1\}$. Part 2 bounds the log-ratio $\ln \hat{g}(1/2 - \varepsilon) - \ln \hat{g}(1/2 + \varepsilon)$ across the half-bin gap $\varepsilon = |1/2 - y| \geq 1/r$ so that unrolling $t_1 \geq r/2$ steps around the cycle contracts by $a^{t_1} < 0.1400$. Part 3 applies character orthogonality modulo t_1 to collapse the alternating frequency sum at $u_2 = r/2$ onto the single input row $j_1 = t_1/2$ and writes out the explicit prefix-sum inverse.

Proof. **Part 1.** Because the unit intervals $[y - m - 1/2, y - m + 1/2]$ for $m \in \mathbb{Z}$ partition $\mathbb{R}$ up to endpoints of measure zero,

$$\sum_{m\in\mathbb{Z}} \hat{g}(y - m) = \sum_{m\in\mathbb{Z}} \int_{y-m-1/2}^{y-m+1/2} \alpha e^{-\pi\alpha^2\xi'^2}\, d\xi' = \int_{-\infty}^{\infty} \alpha e^{-\pi\alpha^2\xi'^2}\, d\xi' = 1.$$

For $y \in [0, 1)$, let $\eta := \min(y, 1 - y) \in [0, 1/2]$, so $M(u_2) = \hat{g}(\eta)$ since $\hat{g}$ is even. For $\eta > 0$,

$$\hat{g}'(\eta) = \alpha\Big(e^{-\pi\alpha^2(\eta+1/2)^2} - e^{-\pi\alpha^2(\eta-1/2)^2}\Big) < 0,$$

so $\hat{g}$ is strictly decreasing on $[0, \infty)$ and $M(u_2) \geq \hat{g}(1/2) = \frac{1}{2}\operatorname{erf}(\sqrt{\pi}\alpha) = \frac{1}{2}(1 - \operatorname{erfc}(\sqrt{\pi}\alpha))$. Since $\pi\alpha^2 = 4p_{\mathrm{work}} \ln 2$ and $\operatorname{erfc}(z) \leq \frac{e^{-z^2}}{\sqrt{\pi}z}$ for $z > 0$, at $p_{\mathrm{work}} \geq 8$ where $\sqrt{\pi}\alpha = \sqrt{4p_{\mathrm{work}} \ln 2} \geq \sqrt{32 \ln 2} > 4.7096$, we have $\frac{1}{2}\operatorname{erfc}(\sqrt{\pi}\alpha) \leq \frac{2^{-4p_{\mathrm{work}}}}{2\sqrt{4\pi p_{\mathrm{work}} \ln 2}} < 0.060 \cdot 2^{-4p_{\mathrm{work}}} < 0.08 \cdot 2^{-4p_{\mathrm{work}}}$ and $\hat{g}(1/2) > 0.499999999 > 0.49999999$.

Removing $m \in \{0, 1\}$ from $\mathbb{R}$ leaves $(-\infty, y - 3/2] \cup [y + 1/2, \infty)$, which directly yields the formula for $\Psi(y)$ in (21). Since $\frac{d^2}{dz^2}\operatorname{erfc}(z) = \frac{4z}{\sqrt{\pi}} e^{-z^2} > 0$ for all $z > 0$ and both $y + 1/2 \geq 1/2 > 0$ and $3/2 - y \geq 1/2 > 0$ on $y \in [0, 1]$, $\Psi(y)$ is strictly convex on $[0, 1]$ and symmetric under $y \mapsto 1 - y$. Hence $\max_{y\in[0,1]} \Psi(y) = \Psi(0) = \Psi(1)$. At $y = 0$, set $z_1 := \sqrt{\pi}\alpha(1/2) = \sqrt{p_{\mathrm{work}} \ln 2}$ and $z_2 := \sqrt{\pi}\alpha(3/2) = 3\sqrt{p_{\mathrm{work}} \ln 2}$. Using $\operatorname{erfc}(z) \leq \frac{e^{-z^2}}{\sqrt{\pi}z}$, we have for all $p_{\mathrm{work}} \geq 8$:

$$\begin{aligned}\Psi(0) &= \frac{1}{2}\operatorname{erfc}\big(\sqrt{p_{\mathrm{work}} \ln 2}\big) + \frac{1}{2}\operatorname{erfc}\big(3\sqrt{p_{\mathrm{work}} \ln 2}\big) \\ &\leq \frac{2^{-p_{\mathrm{work}}}}{2\sqrt{\pi p_{\mathrm{work}} \ln 2}} + \frac{2^{-9p_{\mathrm{work}}}}{6\sqrt{\pi p_{\mathrm{work}} \ln 2}} < 0.1199 \cdot 2^{-p_{\mathrm{work}}} < 0.120 \cdot 2^{-p_{\mathrm{work}}}.\end{aligned}$$

Applying the sharper three-term integration-by-parts bound $\operatorname{erfc}(z) \leq \frac{e^{-z^2}}{\sqrt{\pi}z}\big(1 - \frac{1}{2z^2} + \frac{3}{4z^4}\big)$ gives $\Psi(0) < 0.112 \cdot 2^{-p_{\mathrm{work}}}$ for $p_{\mathrm{work}} \geq 8$, matching $\Psi(0) \approx 0.1111 \cdot 2^{-p_{\mathrm{work}}}$ at $p_{\mathrm{work}} = 8$.

Part 2. Fix $u_2 \neq r/2$ and let $\varepsilon := |1/2 - y| \geq 1/r > 0$. Then $M(u_2) = \hat{g}(1/2 - \varepsilon)$ and $\min(c_0, c_1) = \hat{g}(1/2 + \varepsilon)$. For $s \in [0, \varepsilon]$, let $\phi(s) := \ln \hat{g}(1/2 - s) - \ln \hat{g}(1/2 + s)$. Since $\hat{g}$ is

decreasing on $[0,\infty)$, we have $-\hat{g}'(1/2-s) \geq 0$ for all $s \in [0,\varepsilon] \subseteq [0,1/2]$, while $-\hat{g}'(1/2+s) = \alpha e^{-\pi\alpha^2 s^2}(1-e^{-\pi\alpha^2(1+2s)}) \geq \alpha e^{-\pi\alpha^2 s^2}(1-2^{-4p_{\mathrm{work}}})$, and for $s \geq 0$,

$$\hat{g}(1/2+s) = \int_0^1 \alpha e^{-\pi\alpha^2(\xi'+s)^2}\,d\xi' \leq e^{-\pi\alpha^2 s^2}\int_0^1 \alpha e^{-\pi\alpha^2\xi'^2}\,d\xi' = \hat{g}(1/2)e^{-\pi\alpha^2 s^2}.$$

Hence $\phi'(s) \geq -\hat{g}'(1/2+s)/\hat{g}(1/2+s) \geq \frac{\alpha(1-2^{-4p_{\mathrm{work}}})}{\hat{g}(1/2)} = \frac{2\alpha(1-2^{-4p_{\mathrm{work}}})}{\mathrm{erf}(\sqrt{\pi}\alpha)} > 1.9999\alpha$ for $p_{\mathrm{work}} \geq 8$. Integrating $\phi'(s) > 1.9999\alpha$ over $s \in [0,\varepsilon]$ yields $a = e^{-\phi(\varepsilon)} \leq e^{-1.9999\alpha\varepsilon}$. Since $t_1 \in \{r, r/2\}$ and $\varepsilon \geq 1/r$, $t_1\varepsilon \geq \frac{r}{2}\cdot\frac{1}{r} = \frac{1}{2}$, so $a^{t_1} \leq e^{-1.9999\alpha t_1\varepsilon} \leq e^{-0.99995\alpha}$. For $p_{\mathrm{work}} \geq 8$, $\alpha = 2\sqrt{p_{\mathrm{work}}\ln 2/\pi} \geq 2\sqrt{8\ln 2/\pi} > 2.6571$, so $e^{-0.99995\alpha} < 0.0702 < 0.1400$ and $\Gamma(u_2) = M(u_2)(1-a^{t_1}) > 0.49999999 \times (1-0.1400) > 0.424$.

When $c_0 > c_1$, we have $Z_{u_1} - \rho Z_{u_1-1} = Y_{u_1}/c_0$ with $\rho = -c_1/c_0$, and unrolling t_1 steps around $\mathbb{Z}_{t_1}$ gives $(1-\rho^{t_1})Z_{u_1} = c_0^{-1}\sum_{s=0}^{t_1-1}\rho^s Y_{(u_1-s) \bmod t_1}$. When $c_1 > c_0$, dividing $c_1 Z_{u_1} + c_0 Z_{u_1+1} = Y_{u_1+1}$ by c_1 gives $Z_{u_1} - \rho Z_{u_1+1} = Y_{u_1+1}/c_1$ with $\rho = -c_0/c_1$. Unrolling t_1 steps backward with the $+1$ index shift gives $(1-\rho^{t_1})Z_{u_1} = c_1^{-1}\sum_{s=0}^{t_1-1}\rho^s Y_{(u_1+1+s) \bmod t_1}$. Because $t_1 \geq 2$ is even, $1-\rho^{t_1} = 1-a^{t_1} > 0.86$. In both cases, the absolute row sum of B^{-1} is $\frac{1+a+\cdots+a^{t_1-1}}{M(u_2)(1-a^{t_1})} = \frac{1}{|c_0-c_1|} \leq \frac{t_1}{\Gamma(u_2)} < 2.36\,t_1$.

Part 3. Since $t_1 \geq 2$ is a power of two, t_1 is even and $t_1/2 \in \{1,\ldots,t_1-1\}$ is an integer row index. At $u_2 = r/2$, writing $j = j_1 + t_1 j_2$ and $u = r/2 + r u_1$ gives $\frac{ju}{t_1 r} = \frac{j_1}{2t_1} + \frac{j_2}{2} + \frac{j_1 u_1}{t_1} + j_2 u_1 \equiv \frac{j_1}{2t_1} + \frac{j_2}{2} + \frac{j_1 u_1}{t_1} \pmod 1$. Summing $(-1)^{u_1} = e^{\pi i u_1}$ over $u_1 \in \{0,\ldots,t_1-1\}$ applies character orthogonality on $\mathbb{Z}_{t_1}$:

$$\sum_{u_1=0}^{t_1-1}(-1)^{u_1}e^{2\pi i j_1 u_1/t_1} = \sum_{u_1=0}^{t_1-1} e^{2\pi i(j_1-t_1/2)u_1/t_1} = t_1\,\delta_{j_1,\,t_1/2}.$$

Substituting $j_1 = t_1/2$ gives $e^{2\pi i(t_1/2)/(2t_1)}e^{\pi i j_2} = e^{\pi i/2}(-1)^{j_2} = i(-1)^{j_2}$ and $\frac{t_1}{t_1 r} = \frac{1}{r}$, proving (25). Setting $z'_{u_1} := (-1)^{u_1} Z_{u_1}$, the equations $z'_{u_1} - z'_{u_1-1} = (-1)^{u_1} Y_{u_1}/c$ for $1 \leq u_1 < t_1$ and $\sum_{u_1=0}^{t_1-1} z'_{u_1} = A_{\mathrm{N}}$ have unique solution $z'_{u_1} = p_{u_1} + \frac{1}{t_1}(A_{\mathrm{N}} - \sum_{v=0}^{t_1-1} p_v)$, where $p_{u_1} = \sum_{k=1}^{u_1}(-1)^k Y_k/c$. The explicit inverse expresses z'_{u_1} as $\frac{1}{t_1}A_{\mathrm{N}} + \frac{1}{c}\sum_{k=1}^{t_1-1} w_{u_1,k}(-1)^k Y_k$ with $w_{u_1,k} = \frac{k}{t_1}$ for $k \leq u_1$ and $w_{u_1,k} = \frac{k}{t_1} - 1$ for $k > u_1$, whose absolute row sum over Y is $\frac{1}{c}\sum_{k=1}^{t_1-1}|w_{u_1,k}| \leq \frac{t_1-1}{2c} < 1.0000001\,t_1$. □

5.2 Finite-precision column solvers

We now define the fixed-point column solvers `CyclicSolve` and `NyquistSolve`. Both procedures run on the child grid $\widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ with $q_{\mathrm{child}} = p_{\ell+1} + 2$ and unit accuracy scale $u := 2^{-p_{\ell+1}}$. For $u_2 \neq r/2$, `CyclicSolve` unrolls the cyclic recurrence (23) using the closure scalar $\theta(u_2) := (1-\rho(u_2)^{t_1})^{-1}$, represented on the table tape by its 1-ulp faithful entry $\hat{\theta}(u_2) \in \widetilde{\mathbb{R}}_{q_{\mathrm{child}}}$. For $u_2 = r/2$, `NyquistSolve` evaluates the alternating prefix-sum inverse from Part 3 of Theorem 5.1.

Procedure `CyclicSolve` for $u_2 \neq r/2$. The inputs are the routed column $\widehat{Y} = (\widehat{Y}_0,\ldots,\widehat{Y}_{t_1-1}) \in \widetilde{\mathbb{C}}^{t_1}_{q_{\mathrm{child}}}$ and the 1-ulp faithful precomputed table scalars $\mu = \mu(u_2)$, $\hat{\rho} = \hat{\rho}(u_2)$, and $\hat{\theta} = \hat{\theta}(u_2)$ in $\widetilde{\mathbb{R}}_{q_{\mathrm{child}}}$ from Section 3.3. These entries satisfy $|\mu - M(u_2)^{-1}|, |\hat{\rho}-\rho|, |\hat{\theta}-\theta(u_2)| \leq \frac{1}{4}u = 0.25u$. `CyclicSolve` branches directly on $u_2 < r/2$ versus $u_2 > r/2$:

- *Case 1: $u_2 < r/2$, where $c_0(u_2) > c_1(u_2)$ and sweeps run forward:*

  1. *Prescale, t_1 real-by-complex products:* Set $\tilde{B}_i := \mathrm{RM}_{q_{\mathrm{child}}}(\mu, \widehat{Y}_i)$ for $i = 0,\ldots,t_1-1$.
  2. *Forward prefix sweep, t_1-1 real-by-complex products:* Set $\tilde{P}_0 := \tilde{B}_0$, and for $i = 1,\ldots,t_1-1$, set $\tilde{P}_i := \tilde{B}_i + \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\rho}, \tilde{P}_{i-1})$. In exact arithmetic over $\mathbb{C}$, the unrounded prefix sum satisfies $P_{t_1-1} = \sum_{s=0}^{t_1-1}\rho^s B_{t_1-1-s} = (1-\rho^{t_1})Z^{\dagger}_{t_1-1}$.

3. *Wrap-around closure and forward recurrence pass, t_1 real-by-complex products:* Set

$$Z_{t_1-1}^{\mathrm{comp}} := \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\theta}, \tilde{P}_{t_1-1}), \qquad Z_0^{\mathrm{comp}} := \tilde{B}_0 + \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\rho}, Z_{t_1-1}^{\mathrm{comp}}),$$

and for $i = 1, \ldots, t_1 - 2$, set $Z_i^{\mathrm{comp}} := \tilde{B}_i + \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\rho}, Z_{i-1}^{\mathrm{comp}})$.

- *Case 2: $u_2 > r/2$, where $c_1(u_2) > c_0(u_2)$ and sweeps run backward:*

1. *Prescale, t_1 real-by-complex products:* Set $\tilde{B}_i := \mathrm{RM}_{q_{\mathrm{child}}}(\mu, \widehat{Y}_{(i+1) \bmod t_1})$ for $i = 0, \ldots, t_1 - 1$.
2. *Backward prefix sweep, $t_1 - 1$ real-by-complex products:* Set $\tilde{P}_{t_1-1} := \tilde{B}_{t_1-1}$, and for $i = t_1 - 2, \ldots, 0$, set $\tilde{P}_i := \tilde{B}_i + \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\rho}, \tilde{P}_{i+1})$. In exact arithmetic over $\mathbb{C}$, the unrounded prefix sum satisfies $P_0 = \sum_{s=0}^{t_1-1} \rho^s B_s = (1 - \rho^{t_1}) Z_0^{\dagger}$.
3. *Wrap-around closure and backward recurrence pass, t_1 real-by-complex products:* Set

$$Z_0^{\mathrm{comp}} := \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\theta}, \tilde{P}_0), \qquad Z_{t_1-1}^{\mathrm{comp}} := \tilde{B}_{t_1-1} + \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\rho}, Z_0^{\mathrm{comp}}),$$

and for $i = t_1 - 2, \ldots, 1$, set $Z_i^{\mathrm{comp}} := \tilde{B}_i + \mathrm{RM}_{q_{\mathrm{child}}}(\hat{\rho}, Z_{i+1}^{\mathrm{comp}})$.

In both cases, `CyclicSolve` performs $t_1 + (t_1 - 1) + t_1 = 3t_1 - 1$ real-by-complex rounded multiplications and $2t_1 - 2$ complex additions.

Procedure `NyquistSolve` for $u_2 = r/2$ and final output transpose. The inputs to `NyquistSolve` are $\widehat{Y} \in \widetilde{\mathbb{C}}_{q_{\mathrm{child}}}^{t_1}$, the input alternating sum $A_{\mathrm{N}} \in \widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ from (25), and the 1-ulp faithful Nyquist reciprocal $\mu_{\mathrm{N}} = \mu(r/2) \in \widetilde{\mathbb{R}}_{q_{\mathrm{child}}}$ satisfying $|\mu_{\mathrm{N}} - c^{-1}| \le \frac{1}{4}u$. Note that A_{N} is exact on $\widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ because $x \in \widetilde{\mathbb{C}}_{p_\ell+2}^{R_\ell}$ and $q_{\mathrm{child}} \ge p_\ell + 2 + \log_2 r$. `NyquistSolve` executes two passes:

1. Set $\tilde{P}_0 := 0$, and for $i = 1, \ldots, t_1 - 1$, compute $\tilde{D}_i := \mathrm{RM}_{q_{\mathrm{child}}}(\mu_{\mathrm{N}}, \widehat{Y}_i)$ and $\tilde{P}_i := \tilde{D}_i - \tilde{P}_{i-1}$, using $t_1 - 1$ real-by-complex products.
2. Compute the alternating sum $\tilde{S} := \sum_{i=0}^{t_1-1} (-1)^i \tilde{P}_i$ in a scratch accumulator of width $w_{\mathrm{coord},\ell} + k_{1,\ell} + 1$ bits, form $\tilde{\delta} := \mathcal{Q}_{q_{\mathrm{child}}}((A_{\mathrm{N}} - \tilde{S})/t_1)$, and set $Z_i^{\mathrm{comp}} := \tilde{P}_i + (-1)^i \tilde{\delta}$ for $i = 0, \ldots, t_1 - 1$.

Once all r columns $u_2 \in \{0, \ldots, r-1\}$ have been solved in column-major order (u_2, u_1), each entry is quantized to $\widehat{Z}_{u_1,u_2} = \mathcal{Q}_{p_\ell+2}(Z_{u_1,u_2}^{\mathrm{comp}})$. A single transpose from column-major order (u_2, u_1) to row-major order (u_1, u_2) via part (i) of Lemma 3.3 then places the output in natural one-dimensional frequency order $u_2 + r u_1$. On the uniform records of (6), this transpose costs $O\big(R_\ell w_\ell (1 + \log_2 t_{1,\ell})\big) = O\big(R_\ell p_{\ell+1} (1 + \log_2 t_{1,\ell})\big)$ bit operations.

The next lemma bounds the forward rounding error introduced by `CyclicSolve` and `NyquistSolve` relative to the exact solution $Z^{\dagger}$ of the retained two-term linear system on the same right-hand side $\widehat{Y}$. This bound isolates the fixed-point solver error from the stopband-alias and child-transform errors combined later in Section 6.3.

Lemma 5.2 (Forward error of the column solvers)**.** *Let $\mathcal{M}_Y := \max(1, \|\widehat{Y}\|_\infty)$ and $u = 2^{-p_{\ell+1}}$ ($q_{\mathrm{child}} = p_{\ell+1} + 2$). For every column $u_2 \in \{0, \ldots, r-1\}$ with $t_1 \ge 2$ and $t_1 u \le 2^{-20}$, under the hypotheses of Theorem 5.1 ($p_{\mathrm{work}} \ge 8$), the computed vector Z^{comp} and the exact solution $Z^{\dagger}$ of the retained two-term system with right-hand side $\widehat{Y}$ satisfy*

$$\|Z^{\mathrm{comp}} - Z^{\dagger}\|_\infty \le 80\, t_1^2\, u\, \mathcal{M}_Y. \tag{26}$$

The proof of Lemma 5.2 establishes the tighter bounds $\|Z^{\mathrm{comp}} - Z^{\dagger}\|_\infty < 1.85\, t_1^2\, u\, \mathcal{M}_Y$ for $u_2 \ne r/2$ and $\le 5\, t_1\, u\, \mathcal{M}_Y$ for $u_2 = r/2$; Appendix C.1 unifies both cases below $2.5\, t_1^2\, u\, \mathcal{M}_Y$, and we state the generous constant 80 used in Lemma 6.5.

6 Spreading, forward error, and the additive schedule

This section replaces the ideal infinite-support row spreading of Theorem 4.2 by a finite, fixed-point algorithm and closes the recursive error analysis across all levels of `FastDFT`. Section 6.1 truncates, folds, and quantizes each row's sinc-Gaussian filter to $\tilde{W}_\ell \le r_\ell$ taps within error $O(\tilde{W}_\ell 2^{-p_{\ell+1}})$. Section 6.2 evaluates all $t_{1,\ell}$ cyclic row convolutions via overlap-add and signed Kronecker substitution so that the window width $\tilde{W}_\ell$ cancels out of the total multiplication cost. Section 6.3 combines the spreading, polynomial-transform, alias, and solver errors under the additive schedule $p_{\ell+1} = p_\ell + 2K_\ell + 12$ to prove accuracy $2^{-p_\ell}\max(1, \|x\|_\infty)$ at every level. Readers who trust Theorem 6.3 and Lemma 6.5 may skip directly to Section 7.

6.1 Filter estimates and modulo-r folding

We begin by relating the ideal periodic row filter of Theorem 4.2 to the truncated, quantized tap vector stored on the table tape. At recursion depth ℓ, let $R_\ell = 2^{K_\ell} = t_{1,\ell} r_\ell$, with working cutoff $p_{\mathrm{work},\ell}$ and child precision $p_{\ell+1}$. We distinguish four filter objects for each row $j_1 \in \{0, \dots, t_{1,\ell} - 1\}$:

(a) The *unfolded sample sequence* is $\kappa_d^{(j_1)} := g(d + j_1/t_{1,\ell})$ for $d \in \mathbb{Z}$.

(b) The *exact r_ℓ-periodic filter* is $h_d^{(j_1),\mathrm{per}} := \sum_{a\in\mathbb{Z}} \kappa_{d+ar_\ell}^{(j_1)}$ for $d \in \{0, \dots, r_\ell - 1\}$.

(c) The *truncated modulo-r_ℓ folded filter* is $h_d^{(j_1),\mathrm{tr}} := \sum_{|e|\le m_{1,\ell},\, e\equiv d \pmod{r_\ell}} \kappa_e^{(j_1)}$, supported on at most $\tilde{W}_\ell := \min(W_\ell, r_\ell)$ indices where $W_\ell := 2m_{1,\ell} + 1$.

(d) The *guarded q_{child}-grid tap vector* $\bar{H}^{(j_1)} \in \widetilde{\mathbb{R}}_{q_{\mathrm{child}}}^{\tilde{W}_\ell}$ and its *re-indexed r_ℓ-periodic filter* $\bar{h}^{(j_1)} \in \widetilde{\mathbb{R}}_{q_{\mathrm{child}}}^{r_\ell}$ are built from $\kappa_e^{(j_1)}$ for $|e| \le m_{1,\ell}$. First, each raw sample $\kappa_e^{(j_1)}$ is evaluated via Lemma 3.2 at guard precision $v_{\mathrm{tap}} := q_{\mathrm{child}} + \lceil \log_2(W_\ell + 1) \rceil + 10$. To turn the $j_2 - k_2$ correlation of Theorem 4.2 into a forward polynomial product for Section 6.2, we store the taps in reversed index order. When $W_\ell > r_\ell$, we sum the samples in each residue class $a = (-e) \bmod r_\ell$ with cyclic offset $o := 0$; when $W_\ell \le r_\ell$, we instead reverse the W_ℓ contiguous taps as $\kappa_{m_{1,\ell}-a}^{(j_1)}$ with cyclic offset $o := m_{1,\ell}$. Rounding each resulting tap once via $\mathcal{Q}_{q_{\mathrm{child}}}$ produces $\bar{H}_a^{(j_1)}$, and we set $\bar{h}_d^{(j_1)} := \bar{H}_a^{(j_1)}$ when $d \equiv o - a \pmod{r_\ell}$ for some $a \in \{0, \dots, \tilde{W}_\ell - 1\}$ and $\bar{h}_d^{(j_1)} := 0$ otherwise.

Lemma 6.1 bounds the ℓ_1 norm of the unfolded kernel, the Gaussian spatial tail outside radius $m_{1,\ell}$, and the discrepancy between the exact periodic filter $h^{(j_1),\mathrm{per}}$ and the quantized filter $\bar{h}^{(j_1)}$. We need the logarithmic ℓ_1 bound to keep the Kronecker bit slots compact in Section 6.2, and the tail and rounding bounds to control row-spreading error; the proof appears in Appendix C.2.

Lemma 6.1 (Filter estimates)**.** *Let $p_{\mathrm{work},\ell} \ge 8$, $\alpha_\ell = 2\sqrt{p_{\mathrm{work},\ell} \ln 2/\pi}$, $p_{\ell+1} \ge \max(p_{\mathrm{work},\ell}, 20)$, and $m_{1,\ell} \ge \left\lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \right\rceil$. For every row $j_1 \in \{0, \dots, t_{1,\ell} - 1\}$:*

1. Unfolded ℓ_1 bound:

$$\|\kappa^{(j_1)}\|_1 = \sum_{d\in\mathbb{Z}} \left| g\left(d + \frac{j_1}{t_{1,\ell}}\right)\right| \le \frac{2}{\pi}\ln\alpha_\ell + \frac{5.25 - \ln\pi + E_1(1)}{\pi}$$
$$\le \frac{2}{\pi}\ln\alpha_\ell + \frac{5.25 - \ln\pi + \frac{7}{6e}}{\pi} < \frac{2}{\pi}\ln\alpha_\ell + 1.444 < \frac{2}{\pi}\ln\alpha_\ell + 1.62, \tag{27}$$

where $E_1(1) = \int_1^\infty e^{-t} t^{-1}\, dt < 0.2194$.

2. Spatial tail bound:

$$\tau := \sum_{|d|>m_{1,\ell}} \left| g\left(d + \frac{j_1}{t_{1,\ell}}\right)\right| \le \left(\frac{1}{\pi m_{1,\ell}} + \frac{1}{\pi p_{\ell+1} \ln 2}\right) 2^{-p_{\ell+1}} < 0.077 \cdot 2^{-p_{\ell+1}} < 0.10 \cdot 2^{-p_{\ell+1}}. \quad (28)$$

3. Periodic and folded filter bounds:

$$\begin{aligned} \|h^{(j_1),\mathrm{per}}\|_2 &\le 1, & \|h^{(j_1),\mathrm{per}}\|_1 &\le \sqrt{r_\ell}, \\ \|h^{(j_1),\mathrm{tr}} - h^{(j_1),\mathrm{per}}\|_1 &\le \tau, & \|\bar{h}^{(j_1)} - h^{(j_1),\mathrm{tr}}\|_1 &\le \tilde{W}_\ell 2^{-p_{\ell+1}-1}. \end{aligned} \quad (29)$$

6.2 Spreading by overlap-add and Kronecker substitution

Direct schoolbook evaluation of all $t_{1,\ell}$ cyclic row convolutions of length r_ℓ against filters of width $\tilde{W}_\ell$ would cost $\tilde{W}_\ell = \Theta(\log n)$ scalar multiplications per output entry, one per filter tap. Instead, we combine classical overlap-add block convolution [Nus82] with Kronecker substitution for polynomial multiplication [Sch82, Har09] in a signed two's-complement radix representation.

Lemma 6.2 shows that s signed integers bounded in magnitude by $2^{B_{\mathrm{slot}}-1}$ can be packed into a single signed radix-$2^{B_{\mathrm{slot}}}$ integer and unpacked again in linear time $O(sB_{\mathrm{slot}})$. This signed packing lets us multiply two integer polynomials via a single integer multiplication without splitting positive and negative coefficients.

Lemma 6.2 (Signed radix Kronecker packing and extraction)**.** *Let $s \ge 1$ and $B_{\mathrm{slot}} \ge 4$ be integers, and let $a_0, \dots, a_{s-1} \in \mathbb{Z}$ satisfy $|a_j| < 2^{B_{\mathrm{slot}}-1}$. Define the signed radix-$2^{B_{\mathrm{slot}}}$ evaluation*

$$\mathrm{Pack}_{B_{\mathrm{slot}}}(a_0, \dots, a_{s-1}) := \sum_{j=0}^{s-1} a_j \, 2^{jB_{\mathrm{slot}}} \in \mathbb{Z}. \quad (30)$$

(i) Given the signed binary representations of $a_0, \dots, a_{s-1}$, the integer $\mathrm{Pack}_{B_{\mathrm{slot}}}(a)$ has bit-length at most $sB_{\mathrm{slot}} + 1$ and can be assembled in $O(sB_{\mathrm{slot}})$ bit operations on a multitape Turing machine via a single low-to-high streaming carry pass.

(ii) Conversely, if $c_0, \dots, c_{m-1} \in \mathbb{Z}$ satisfy $|c_k| < 2^{B_{\mathrm{slot}}-1}$ for all k and $T_0 := \sum_{k=0}^{m-1} c_k 2^{kB_{\mathrm{slot}}}$, then the recurrence

$$r_k := T_k \bmod 2^{B_{\mathrm{slot}}} \in [0, 2^{B_{\mathrm{slot}}}), \quad c_k := \begin{cases} r_k, & r_k < 2^{B_{\mathrm{slot}}-1}, \\ r_k - 2^{B_{\mathrm{slot}}}, & r_k \ge 2^{B_{\mathrm{slot}}-1}, \end{cases} \quad T_{k+1} := \frac{T_k - c_k}{2^{B_{\mathrm{slot}}}},$$

uniquely recovers $c_0, \dots, c_{m-1}$ in $O(mB_{\mathrm{slot}})$ bit operations.

Proof. For the packing claim, write each $a_j \in (-2^{B_{\mathrm{slot}}-1}, 2^{B_{\mathrm{slot}}-1})$ with a borrow $\sigma_j \in \{-1, 0\}$ into the next slot. Initialize $\sigma_0 := 0$, and for $j = 0, \dots, s-1$, let $a'_j := a_j + \sigma_j$. If $a'_j \ge 0$, emit the B_{slot}-bit unsigned word $u_j := a'_j \in [0, 2^{B_{\mathrm{slot}}-1})$ and set $\sigma_{j+1} := 0$; if $a'_j < 0$, emit the B_{slot}-bit unsigned word $u_j := a'_j + 2^{B_{\mathrm{slot}}} \in [2^{B_{\mathrm{slot}}-1}, 2^{B_{\mathrm{slot}}})$ and set $\sigma_{j+1} := -1$. Then $\sum_{j=0}^{s-1} a_j 2^{jB_{\mathrm{slot}}} = \sigma_s 2^{sB_{\mathrm{slot}}} + \sum_{j=0}^{s-1} u_j 2^{jB_{\mathrm{slot}}}$, and concatenating the B_{slot}-bit words u_j takes $O(sB_{\mathrm{slot}})$ bit operations.

For the extraction claim, $T_k = c_k + 2^{B_{\mathrm{slot}}} T_{k+1}$ implies $c_k \equiv T_k \pmod{2^{B_{\mathrm{slot}}}}$. Because $[-2^{B_{\mathrm{slot}}-1}, 2^{B_{\mathrm{slot}}-1})$ contains a unique representative of each residue class modulo $2^{B_{\mathrm{slot}}}$, both c_k and T_{k+1} are uniquely determined and extracted in $O(B_{\mathrm{slot}})$ bit operations per slot. □

Theorem 6.3 applies overlap-add and signed Kronecker packing across all $t_{1,\ell}$ rows at a non-leaf node. It shows that the computed spread array $\widehat{U}$ matches the ideal spread array U° within $2r_\ell\, 2^{-p_{\ell+1}} \max(1, \|x\|_\infty)$, while reducing all row convolutions to $O(R_\ell/\tilde{W}_\ell)$ integer multiplications of bit-length $\Theta(\tilde{W}_\ell p_{\ell+1})$. In (33), $\mathsf{M}(N)$ denotes the cost of one N-bit integer multiplication. When Section 7.2 instantiates M with an inner multiplier of cost $O(N \log 2N)$, the factor $\tilde{W}_\ell$ cancels and the convolution term in (33) totals $o(n \log n)$ steps across all levels, apart from the row-de-interleaving transpose.

Theorem 6.3 (Spreading by overlap-add and Kronecker substitution)**.** *At any non-leaf node ($R_\ell > 4p_\ell^2$, with $p_\ell \geq 8$) under the hypotheses of Lemma 6.1 with $p_{\mathrm{work},\ell} = p_\ell + K_\ell + 6$ and $p_{\ell+1} = p_\ell + 2K_\ell + 12$, let $x \in \widetilde{\mathbb{C}}_q^{R_\ell}$ satisfy $\|x\|_\infty \leq 2^{p_\ell}$, let $q := q_{\mathrm{child}} = p_{\ell+1} + 2$, $\tilde{W}_\ell := \min(2m_{1,\ell}+1, r_\ell) \leq r_\ell$, and*

$$B_{\mathrm{slot}} := 2q + p_\ell + \lceil \log_2(\tilde{W}_\ell + 1) \rceil + 4 = \Theta(p_{\ell+1}). \tag{31}$$

Computing the cyclic row convolutions for all $j_1 \in \{0, \ldots, t_{1,\ell} - 1\}$ with the guarded q-grid filters $\bar{H}^{(j_1)}$ via overlap-add of block length $\tilde{W}_\ell$, signed Kronecker substitution at slot width B_{slot}, exact integer overlap addition on $\widetilde{\mathbb{C}}_{2q}$, and a single final rounding per output entry to $\widetilde{\mathbb{C}}_q$ produces an array $\widehat{U} \in \widetilde{\mathbb{C}}_q^{t_{1,\ell} \times r_\ell}$ satisfying

$$\|\widehat{U} - U^\circ\|_\infty \leq \left(\tfrac{1}{2}\tilde{W}_\ell + 0.10\right) 2^{-p_{\ell+1}} \|x\|_\infty + \frac{\sqrt{2}}{8} 2^{-p_{\ell+1}} \leq 2r_\ell\, 2^{-p_{\ell+1}} \max(1, \|x\|_\infty) \tag{32}$$

in time

$$\mathcal{C}_{\mathrm{spread}}(R_\ell, p_{\ell+1}) = O\left(\frac{R_\ell}{\tilde{W}_\ell}\, \mathsf{M}(\tilde{W}_\ell B_{\mathrm{slot}} + 1) + R_\ell p_{\ell+1} \log_2 t_{1,\ell} \right) \tag{33}$$

bit operations, where $\mathsf{M}(N)$ denotes the bit complexity of multiplying two N-bit integers and $\tilde{W}_\ell B_{\mathrm{slot}} + 1 = \Theta(\tilde{W}_\ell p_{\ell+1})$ is the operand length of the block products in the proof.

Proof idea. We first de-interleave the input into $t_{1,\ell}$ contiguous rows and verify from Lemma 6.1 that the ℓ_1 norm of each scaled filter is at most $(\tilde{W}_\ell + 1)2^q$, so every block-convolution coefficient fits strictly inside a B_{slot}-bit signed slot. Packing each block of length at most $\tilde{W}_\ell$ via Lemma 6.2 reduces its linear convolution with the pre-packed filter to two integer products of bit-length at most $\tilde{W}_\ell B_{\mathrm{slot}} + 1$. Sequential carry-over addition on two scratch tapes then accumulates the overlapping tails modulo r_ℓ before a single final rounding and cyclic rotation pass produces $\widehat{U}$.

Proof. First, by part (i) of Lemma 3.3, de-interleaving $x \in \widetilde{\mathbb{C}}_q^{R_\ell}$ into the $t_{1,\ell}$ contiguous rows $v^{(j_1)} = \left(x(j_1 + t_{1,\ell} j_2)\right)_{j_2=0}^{r_\ell - 1}$ takes $O(R_\ell p_{\ell+1} \log_2 t_{1,\ell})$ bit operations. At any non-leaf node we have $p_\ell \geq 8$ and $R_\ell > 4p_\ell^2 \geq 256$, so $K_\ell \geq 9$, $r_\ell = 2^{\lceil K_\ell/2 \rceil} \geq 32$, and $p_\ell < \frac{1}{2}\sqrt{R_\ell} \leq \frac{1}{2} r_\ell$. Under $p_{\mathrm{work},\ell} = p_\ell + K_\ell + 6$, since $K_\ell \leq 2 \log_2 r_\ell < r_\ell$ and $6 < \frac{1}{2} r_\ell$ for all $r_\ell \geq 32$, we have $p_{\mathrm{work},\ell} < 2r_\ell$, and therefore

$$\begin{aligned} \alpha_\ell = 2\sqrt{p_{\mathrm{work},\ell} \ln 2/\pi} &< 2\sqrt{r_\ell} < r_\ell, \\ r - \frac{2}{\pi} \ln r \geq 32 - \frac{2}{\pi} \ln 32 &> 29 > 1.62 \quad (r \geq 32), \\ \frac{2}{\pi} \ln \alpha_\ell + 2.62 < \frac{2}{\pi} \ln r_\ell + 2.62 &\leq r_\ell + 1. \end{aligned}$$

Meanwhile, since $p_{\ell+1} \geq p_{\mathrm{work},\ell}$ and $\sqrt{p_{\mathrm{work},\ell} \ln 2/\pi} = \frac{1}{2}\alpha_\ell$, Lemma 6.1 gives $m_{1,\ell} \geq \max(6, \lceil \frac{1}{2}\alpha_\ell^2 \rceil)$, so $2m_{1,\ell} + 2 \geq \max(14, \alpha_\ell^2 + 2) > \frac{2}{\pi} \ln \alpha_\ell + 2.62$ for all $\alpha_\ell > 0$. Combining both branches yields $\frac{2}{\pi} \ln \alpha_\ell + 2.62 \leq \min(2m_{1,\ell} + 1, r_\ell) + 1 = \tilde{W}_\ell + 1$ across all non-leaves.

For each row j_1, partition $v^{(j_1)} \in \widetilde{\mathbb{C}}_q^{r_\ell}$ into $\lceil r_\ell/\tilde{W}_\ell \rceil \le \frac{2r_\ell}{\tilde{W}_\ell}$ contiguous blocks of length $L_b \le \tilde{W}_\ell$. On $\widetilde{\mathbb{C}}_q$, each scaled input coordinate $2^q \operatorname{Re}(v_{j_2}^{(j_1)}), 2^q \operatorname{Im}(v_{j_2}^{(j_1)}) \in \mathbb{Z}$ has absolute value at most 2^{q+p_ℓ}. By parts 1 and 3 of Lemma 6.1, the guarded filter satisfies

$$\|\bar{H}^{(j_1)}\|_1 = \|\bar{h}^{(j_1)}\|_1 \le \|h^{(j_1),\mathrm{tr}}\|_1 + \frac{1}{2}\tilde{W}_\ell 2^{-p_{\ell+1}} \le \|\kappa^{(j_1)}\|_1 + 1 < \frac{2}{\pi}\ln \alpha_\ell + 2.62 \le \tilde{W}_\ell + 1,$$

so $\sum_{a=0}^{\tilde{W}_\ell - 1} |2^q \bar{H}_a^{(j_1)}| \le (\tilde{W}_\ell + 1)2^q$. By the ℓ_∞–ℓ_1 inequality, whether $W_\ell \le r_\ell$ or $W_\ell > r_\ell$, every integer linear-convolution coefficient between a block and $2^q \bar{H}^{(j_1)}$ has absolute value at most

$$2^{q+p_\ell} \cdot (\tilde{W}_\ell + 1)2^q = (\tilde{W}_\ell + 1)2^{2q+p_\ell} \le 2^{B_{\mathrm{slot}}-4} < 2^{B_{\mathrm{slot}}-1}.$$

By Lemma 6.2, we pack the real and imaginary parts of each block via $\mathrm{Pack}_{B_{\mathrm{slot}}}$ and multiply each by the pre-packed filter integer $\mathrm{Pack}_{B_{\mathrm{slot}}}(2^q \bar{H}^{(j_1)})$ replayed from T_{tab}. Each block thus performs two integer multiplications of bit-length at most $\tilde{W}_\ell B_{\mathrm{slot}} + 1 = \Theta(\tilde{W}_\ell p_{\ell+1})$, and extracting $L_b + \tilde{W}_\ell - 1$ slots recovers the exact integer block convolutions at fractional precision $2q$.

To accumulate the overlapping block outputs modulo r_ℓ on a multitape Turing machine, scratch tape T_1 holds the $(\tilde{W}_\ell - 1)$-entry carry-over tail from block $b-1$, initialized to 0 at $b = 0$. As block b of row j_1 is unpacked into $L_b + \tilde{W}_\ell - 1 \ge \tilde{W}_\ell$ coefficients, this tail on T_1 is added in lockstep to the first $\tilde{W}_\ell - 1$ coefficients. The first L_b resulting coefficients are appended sequentially to scratch tape T_2, while the remaining $\tilde{W}_\ell - 1$ coefficients overwrite T_1. After the final block of row j_1, the residual $(\tilde{W}_\ell - 1)$-entry tail on T_1 wraps cyclically modulo r_ℓ to row positions $0, \ldots, \tilde{W}_\ell - 2$. Rewinding T_2 to the start of row j_1 and T_1 to index 0, one pass over the first $\tilde{W}_\ell - 1$ entries adds this wrapped tail in $O(\tilde{W}_\ell B_{\mathrm{slot}}) = O(r_\ell p_{\ell+1})$ steps.

Finally, when $W_\ell \le r_\ell$, the reversed contiguous filter $\kappa_{m_{1,\ell}-a}^{(j_1)}$ for $0 \le a < W_\ell$ offsets the cyclic convolution indices by $o = m_{1,\ell} \in \{0, \ldots, r_\ell - 1\}$. When $W_\ell > r_\ell$, the modulo-r_ℓ folded filter is already indexed by $a = (-e) \bmod r_\ell$ with offset $o = 0$. During the single final rounding pass from scale $2q$ to $\widetilde{\mathbb{C}}_q$, we copy the first o entries of T_2 to T_1 and stream the remaining $r_\ell - o$ entries of T_2 followed by the o entries of T_1. This rotates row j_1 into natural index order $k_2 = 0, \ldots, r_\ell - 1$ in $O(r_\ell w_\ell) = O(r_\ell p_{\ell+1})$ steps while incurring a single rounding error of at most $\frac{\sqrt{2}}{2}2^{-q} = \frac{\sqrt{2}}{8}2^{-p_{\ell+1}}$ per entry. Combining this bound with $\|\bar{h}^{(j_1)} - h^{(j_1),\mathrm{per}}\|_1 \le (\frac{1}{2}\tilde{W}_\ell + 0.10)2^{-p_{\ell+1}}$ from part 3 of Lemma 6.1 yields (32). Across all $t_{1,\ell}$ rows there are at most $2R_\ell/\tilde{W}_\ell$ blocks, proving (33). □

The following remark explains when the raw window length $W_\ell = 2m_{1,\ell} + 1$ can exceed the row length r_ℓ and why folding modulo r_ℓ handles both cases uniformly.

Remark 6.4 (Modulo-r_ℓ folding at small non-leaves). At any non-leaf $R_\ell > 4p_\ell^2$, we have $p_\ell < \frac{1}{2}\sqrt{R_\ell} \le \frac{1}{2}r_\ell$, so $W_\ell = 2m_{1,\ell} + 1 = \Theta(p_{\ell+1}) = O(r_\ell)$. For the default radius $m_{1,\ell} = \lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \rceil$ under schedule (34), $W_\ell < r_\ell$ at every non-leaf reached from the root parameters (for $n > 64$ one has $p_\ell \ge 2K_\ell + 14$ at every depth, whence $W_\ell \le 1.53\, p_\ell + 3 < 2p_\ell < r_\ell$), but $W_\ell > r_\ell$ does occur inside the hypotheses of Theorem 6.3 (for the larger admissible radii, up to $m_{1,\ell} = p_{\ell+1}$, and for parameter pairs not reached from any root) and outside them (for the lower leaf cutoff of the C program, Section 9): for example, the parameters $(p_\ell, K_\ell) = (15, 10)$, which satisfy $R_\ell = 1024 > 4p_\ell^2 = 900$ but are not reached from any root, give with $m_{1,\ell} = \lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \rceil = 17$ the values $r_\ell = 32$ and $W_\ell = 35 > r_\ell$. Folding modulo r_ℓ (and never modulo W_ℓ) preserves the exact cyclic convolution modulo r_ℓ while keeping the effective filter length $\tilde{W}_\ell = \min(W_\ell, r_\ell) \le r_\ell$.

6.3 Forward error and the additive schedule

We now combine the spreading, polynomial-transform, stopband-alias, and column-solver error bounds to close the precision induction across all levels of `FastDFT`.

Lemma 6.5 defines the additive precision schedule $p_{\mathrm{work},\ell} = p_\ell + K_\ell + 6$ and $p_{\ell+1} = p_\ell + 2K_\ell + 12$, where $K_\ell = \log_2 R_\ell$. Adding $2K_\ell + 12$ bits when passing from length R_ℓ to child length r_ℓ absorbs both the $O(t_{1,\ell})$ deconvolution conditioning and the $O(R_\ell)$ fixed-point rounding growth. Consequently, `FastDFT` delivers output accuracy $2^{-p_\ell}\max(1, \|x\|_\infty)$ at every non-leaf and leaf node, while the total precision increment across all levels sums to $O(K_0) = O(\log n)$ bits.

Lemma 6.5 (Forward error and the additive schedule)**.** *At recursion depth ℓ, let $R_\ell = 2^{K_\ell}$ ($K_\ell \geq 2$) and let the target accuracy be $p_\ell \geq \max(8, K_\ell)$ on an input $x \in \widetilde{\mathbb{C}}^{R_\ell}_{p_\ell+2}$ with $\|x\|_\infty \leq 2^{p_\ell}$. Define the additive precision schedule*

$$p_{\mathrm{work},\ell} := p_\ell + K_\ell + 6, \qquad p_{\ell+1} := p_{\mathrm{work},\ell} + K_\ell + 6 = p_\ell + 2K_\ell + 12, \qquad q_{\mathrm{child}} := p_{\ell+1} + 2, \quad (34)$$

and choose any integer window radius $m_{1,\ell} \in \big[\lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \rceil,\, p_{\ell+1}\big]$ (for instance $m_{1,\ell} := \lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \rceil$ or $m_{1,\ell} := p_{\ell+1}$). Assume inductively that every recursive child call at length r_ℓ and precision $p_{\ell+1}$ satisfies $\|\widehat{C}_{v_1} - F_{r_\ell}(\widehat{V}_{v_1,\cdot})\|_\infty \leq 2^{-p_{\ell+1}} \max(1, \|\widehat{V}_{v_1,\cdot}\|_\infty)$. Then:

1. ***Forward error progression at a non-leaf:*** *Assume $R_\ell > 4p_\ell^2$ (non-leaf), and let $\mathcal{A} := \max(1, \|x\|_\infty)$ and $u := 2^{-p_{\ell+1}}$ (following the three-step decomposition of `FastDFT` in Algorithm 1 and Listing 1: Step 1 row spreading, Step 2 polynomial-transform 2D DFT via Steps 2a–2d, and Step 3 column deconvolution). Let $Z := F_{R_\ell} x$ be the exact 1D DFT, Z^* be the exact solution of the retained two-term system with ideal right-hand side $Y^\circ = F_{t_{1,\ell}, r_\ell} U^\circ$, $Z^\dagger$ be the exact solution of the retained two-term system with the routed child output $\widehat{Y}$, and Z^{comp} be the output of `CyclicSolve/NyquistSolve`. Then:*

$$\begin{aligned} \|Z^{\mathrm{comp}} - F_{R_\ell} x\|_\infty &\leq \underbrace{\frac{1}{2} t_{1,\ell}\, 2^{-p_{\mathrm{work},\ell}} \mathcal{A}}_{\|Z^* - Z\|_\infty \text{ (aliases)}} + \underbrace{27 R_\ell\, u \mathcal{A}}_{\|Z^\dagger - Z^*\|_\infty \text{ (Steps 1–2)}} + \underbrace{160 R_\ell\, u \mathcal{A}}_{\|Z^{\mathrm{comp}} - Z^\dagger\|_\infty \text{ (Step 3)}} \\ &\leq \frac{1}{2} t_{1,\ell}\, 2^{-p_{\mathrm{work},\ell}} \mathcal{A} + 200 R_\ell\, u \mathcal{A}. \end{aligned} \quad (35)$$

2. ***Inductive closure at non-leaves and leaves:*** *After quantizing Z^{comp} to $\widetilde{\mathbb{C}}_{p_\ell+2}$ and transposing to natural index order when $R_\ell > 4p_\ell^2$ (or after executing `BaseRadix2FFT` at internal precision $q_b := p_\ell + K_\ell + 4$, which `FastDFT` does when $R_\ell \leq 4p_\ell^2$; the `BaseRadix2FFT` bound below holds for every length $R_\ell = 2^{K_\ell}$, $K_\ell \geq 2$, and is also used at the root length by the bottom stage of Section 7.2), the returned vector $\widehat{Z} = \texttt{FastDFT}(x, R_\ell, p_\ell)$ satisfies*

$$\|\widehat{Z} - F_{R_\ell} x\|_\infty \leq 2^{-p_\ell} \max(1, \|x\|_\infty). \quad (36)$$

Proof idea. At a non-leaf node, we track the forward error step by step from the spread array $\widehat{U}$ through the twisted polynomial transform and child calls to the routed array $\widehat{Y}$. This chain establishes $\|\widehat{Y} - Y^\circ\|_\infty \leq 9 r_\ell u \mathcal{A}$ and $\|\widehat{V}_{v_1,\cdot}\|_\infty \leq 2 r_\ell \mathcal{A} < 2^{p_{\ell+1}}$. Multiplying the stopband tail of Theorem 5.1 and the right-hand-side perturbation by the column inverse norm bound $2.36\, t_{1,\ell} < 3 t_{1,\ell}$ and adding the solver rounding error of Lemma 5.2 yields (35). Substituting $p_{\mathrm{work},\ell} - p_\ell = K_\ell + 6$ and $p_{\ell+1} - p_\ell = 2K_\ell + 12$ bounds the non-leaf error below $0.197 \cdot 2^{-p_\ell} \mathcal{A}$, while a stagewise contraction bound gives $< 0.267 \cdot 2^{-p_\ell} \mathcal{A}$ for `BaseRadix2FFT` at leaves.

Proof. First verify that the interval $\big[\lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \rceil,\, p_{\ell+1}\big]$ is non-empty and satisfies the hypotheses $p_{\mathrm{work},\ell} \geq 16 \geq 8$ and $p_{\ell+1} \geq 24 \geq \max(p_{\mathrm{work},\ell}, 20)$ of Lemma 6.1. Since $\alpha_\ell = 2\sqrt{p_{\mathrm{work},\ell} \ln 2/\pi}$ and $\frac{2 \ln 2}{\pi} < 0.442 < 1$, for all $p_\ell \geq 8$ and $K_\ell \geq 2$ we have

$$\alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} = \frac{2 \ln 2}{\pi} \sqrt{p_{\mathrm{work},\ell}\, p_{\ell+1}} < 0.442\, p_{\ell+1} < p_{\ell+1}.$$

Steps 1 and 2 of Algorithm 1, with Step 2 realized by Proposition 4.1: bounding $\widehat{Y}-Y^\circ$ at a non-leaf $R_\ell > 4p_\ell^2$. By part 3 of Lemma 6.1, $\|U^\circ\|_\infty \le \max_{j_1} \|h^{(j_1),\mathrm{per}}\|_1 \|x\|_\infty \le \sqrt{r_\ell}\mathcal{A}$. Since $R_\ell > 4p_\ell^2$, Theorem 6.3 applies and gives $\|\widehat{U} - U^\circ\|_\infty \le 2r_\ell u\mathcal{A}$.

In the pre-twist Step 2a, by (7) the 1-ulp faithful table roots $\bar{\zeta}_{k_2} \in \widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ satisfy $|\bar{\zeta}_{k_2} - \zeta^{k_2}| \le \frac{\sqrt{2}}{4}u < 0.354u$ and $|\bar{\zeta}_{k_2}| \le 1 + 0.354u$. Since $r_\ell \ge 2$ implies $\frac{0.354}{\sqrt{r_\ell}} + \frac{0.177}{r_\ell} < 0.34$ and $\frac{\log_2 t_{1,\ell}}{r_\ell} \le \frac{1}{2}$, each entry $\widehat{P}_{j_1,k_2} = \mathrm{RM}_{q_{\mathrm{child}}}(\bar{\zeta}_{k_2}, \widehat{U}_{j_1,k_2})$ differs from $P^\circ_{j_1,k_2} = \zeta^{k_2} U^\circ_{j_1,k_2}$ by at most

$$|\bar{\zeta}_{k_2}|\,|\widehat{U}_{j_1,k_2} - U^\circ_{j_1,k_2}| + |\bar{\zeta}_{k_2} - \zeta^{k_2}|\,|U^\circ_{j_1,k_2}| + \frac{\sqrt{2}}{8}u \le (1+u)(2r_\ell u\mathcal{A}) + 0.354\sqrt{r_\ell}u\mathcal{A} + 0.177u\mathcal{A} \le 2.6r_\ell u\mathcal{A},$$

and $\|\widehat{P}\|_\infty \le \sqrt{r_\ell}\mathcal{A} + 2.6r_\ell u\mathcal{A} \le 2r_\ell\mathcal{A}$.

In the polynomial-transform Step 2b, $G_{t_{1,\ell}}$ is an average over $t_{1,\ell}$ monomial shifts and has coefficient-wise ℓ_∞ norm at most 1. Lemma 3.5 therefore gives $\|\widehat{Q} - Q^\circ\|_\infty \le \|\widehat{P} - P^\circ\|_\infty + 0.177(\log_2 t_{1,\ell})u \le 3r_\ell u\mathcal{A}$ and $\|Q^\circ\|_\infty \le \sqrt{r_\ell}\mathcal{A}$. In the post-twist Step 2c by $\overline{\bar{\zeta}_{k_2}}$, the same product bound and the norm chain give

$$\begin{aligned}
\|\widehat{V} - V^\circ\|_\infty &\le (1+u)(3r_\ell u\mathcal{A}) + 0.354\sqrt{r_\ell}u\mathcal{A} + 0.177u\mathcal{A} \le 3.6r_\ell u\mathcal{A} < 7r_\ell u\mathcal{A},\\
\|V^\circ\|_\infty &= \|Q^\circ\|_\infty \le \|P^\circ\|_\infty = \|U^\circ\|_\infty \le \sqrt{r_\ell}\mathcal{A} \le r_\ell\mathcal{A},\\
\|\widehat{V}_{v_1,\cdot}\|_\infty &\le r_\ell\mathcal{A} + 7r_\ell u\mathcal{A} \le 2r_\ell\mathcal{A} \le 2^{p_\ell+K_\ell+1} < 2^{p_\ell+2K_\ell+12} = 2^{p_{\ell+1}}.
\end{aligned}$$

Thus each child call $\widehat{C}_{v_1} = \texttt{FastDFT}(\widehat{V}_{v_1,\cdot}, r_\ell, p_{\ell+1})$ in Step 2d satisfies the child input precondition $\|\widehat{V}_{v_1,\cdot}\|_\infty \le 2^{p_{\ell+1}}$. By the induction hypothesis and $\|F_{r_\ell}\|_{\infty\to\infty} \le 1$, we obtain

$$\|\widehat{C}_{v_1} - F_{r_\ell} V^\circ_{v_1,\cdot}\|_\infty \le \|\widehat{C}_{v_1} - F_{r_\ell}\widehat{V}_{v_1,\cdot}\|_\infty + \|\widehat{V}_{v_1,\cdot} - V^\circ_{v_1,\cdot}\|_\infty \le u(2r_\ell\mathcal{A}) + 7r_\ell u\mathcal{A} = 9r_\ell u\mathcal{A}.$$

Because the odd-stride permutation (14) is an index bijection, the routed array $\widehat{Y}$ satisfies $\|\widehat{Y} - Y^\circ\|_\infty \le 9r_\ell u\mathcal{A}$, and

$$\begin{aligned}
\|Y^\circ\|_\infty &\le \sum_{m\in\mathbb{Z}} \hat{g}(u_2/r_\ell - m)\|F_{R_\ell}x\|_\infty \le \|x\|_\infty \le \mathcal{A},\\
\mathcal{M}_Y &= \max(1, \|\widehat{Y}\|_\infty) \le \mathcal{A} + 9r_\ell u\mathcal{A} \le 2\mathcal{A}.
\end{aligned}$$

Step 3 error progression $Z \to Z^* \to Z^\dagger \to Z^{\mathrm{comp}}$. By Theorem 4.2 and Part 1 of Theorem 5.1, for every column u_2 the exact DFT Z satisfies the retained two-term equations with right-hand side $Y^\circ - \Delta^{\mathrm{alias}}$, where $\|\Delta^{\mathrm{alias}}\|_\infty \le \Psi(0)\|F_{R_\ell}x\|_\infty < 0.15 \cdot 2^{-p_{\mathrm{work},\ell}}\mathcal{A}$. At $u_2 = r_\ell/2$, the alternating sum A_{N} in (25) is exact on $\widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ because $q_{\mathrm{child}} = p_\ell + 2K_\ell + 14 > p_\ell + 2 + \log_2 r_\ell$. Applying the inverse norm bound $< 2.36\,t_{1,\ell} < 3t_{1,\ell}$ from Parts 2 and 3 of Theorem 5.1 gives

$$\|Z^* - Z\|_\infty \le 2.36\,t_{1,\ell} \cdot 0.15 \cdot 2^{-p_{\mathrm{work},\ell}}\mathcal{A} < \frac{1}{2}t_{1,\ell}\,2^{-p_{\mathrm{work},\ell}}\mathcal{A},$$

and perturbing the right-hand side from Y° to $\widehat{Y}$ gives

$$\|Z^\dagger - Z^*\|_\infty \le 3t_{1,\ell}\|\widehat{Y} - Y^\circ\|_\infty \le 3t_{1,\ell}(9r_\ell u\mathcal{A}) = 27R_\ell u\mathcal{A}.$$

Furthermore,

$$p_{\ell+1} - \lfloor K_\ell/2 \rfloor \ge 8 + \frac{3}{2}K_\ell + 12 \ge 23 > 20, \qquad t_{1,\ell}u \le 2^{-23} < 2^{-20},$$

so Lemma 5.2 and $\mathcal{M}_Y \le 2\mathcal{A}$ yield $\|Z^{\mathrm{comp}} - Z^{\dagger}\|_\infty \le 80t_{1,\ell}^2 u(2\mathcal{A}) \le 160R_\ell u\mathcal{A}$. Summing the three bounds proves (35).

Inductive closure. Under (34), we have $p_{\mathrm{work},\ell} - p_\ell = K_\ell + 6$ and $p_{\ell+1} - p_\ell = 2K_\ell + 12$, where $t_{1,\ell} = 2^{\lfloor K_\ell/2 \rfloor} \le 2^{K_\ell}$ and $R_\ell = 2^{K_\ell}$ with $K_\ell \ge 2$. Thus

$$\frac{1}{2} t_{1,\ell}\, 2^{-(p_{\mathrm{work},\ell} - p_\ell)} \le \frac{1}{2} 2^{K_\ell} \cdot 2^{-K_\ell - 6} = 2^{-7},$$
$$200 R_\ell\, 2^{-(p_{\ell+1} - p_\ell)} = 200 \cdot 2^{-K_\ell - 12} \le 200 \cdot 2^{-14} = \frac{25}{2048} < 2^{-6}.$$

Final nearest-grid rounding $\widehat{Z} = \mathcal{Q}_{p_\ell+2}(Z^{\mathrm{comp}})$ adds at most $\frac{\sqrt{2}}{8} 2^{-p_\ell} < 0.177 \cdot 2^{-p_\ell}$, so

$$\|\widehat{Z} - F_{R_\ell} x\|_\infty \le \left(\frac{\sqrt{2}}{8} + 2^{-7} + \frac{25}{2048} \right) 2^{-p_\ell} \mathcal{A} < 0.197 \cdot 2^{-p_\ell} \mathcal{A} < \frac{1}{4} 2^{-p_\ell} \mathcal{A} < 2^{-p_\ell} \mathcal{A}.$$

Leaf case: `BaseRadix2FFT`. For `BaseRadix2FFT` at any length $R_\ell = 2^{K_\ell}$ with $K_\ell \ge 2$, and in particular at a leaf $R_\ell \le 4p_\ell^2$, the procedure bit-reverses its $R_\ell = 2^{K_\ell}$ input records and executes $K_\ell = \log_2 R_\ell$ decimation-in-time radix-2 Cooley–Tukey butterfly stages on $\widetilde{\mathbb{C}}_{q_b}$ with $q_b \ge p_\ell + K_\ell + 4$. In stage $s \in \{1, \dots, K_\ell\}$, each block of 2^s records is split into its upper and lower halves $(A_j)_{j=0}^{2^{s-1}-1}$ and $(B_j)_{j=0}^{2^{s-1}-1}$ on T_1 and T_2. These halves are paired in lockstep with the stage-s 1-ulp faithful twiddle list $(\bar{\omega}_{s,j})_{j=0}^{2^{s-1}-1}$ replayed from T_{tab}, which satisfies $|\bar{\omega}_{s,j} - \omega_{s,j}| \le \sqrt{2} \cdot 2^{-q_b}$. Each pair is then updated via $W_j := \mathrm{RM}_{q_b}(\bar{\omega}_{s,j}, B_j)$ and $A_j^{\mathrm{new}}, B_j^{\mathrm{new}} := \mathcal{Q}_{q_b}(\frac{1}{2}(A_j \pm W_j))$. Inductively $\|A\|_\infty, \|B\|_\infty \le 1.01\mathcal{A}$, and each stage incurs twiddle approximation error at most $\sqrt{2} \cdot 2^{-q_b}(1.01\mathcal{A})$ plus product and halving-rounding errors at most $\sqrt{2} \cdot 2^{-q_b}\mathcal{A}$, so $e_s \le (1 + 2^{-q_b}) e_{s-1} + 2.85 \cdot 2^{-q_b}\mathcal{A}$. Since $K_\ell 2^{-K_\ell} \le \frac{1}{2}$ for all $K_\ell \ge 1$, across K_ℓ stages we obtain

$$e_{K_\ell} \le 2.86 K_\ell 2^{-q_b}\mathcal{A} \le 2.86 K_\ell 2^{-K_\ell - 4} 2^{-p_\ell}\mathcal{A} \le 1.43 \cdot 2^{-4} 2^{-p_\ell}\mathcal{A} < 0.090 \cdot 2^{-p_\ell}\mathcal{A}$$

before final rounding to $\widetilde{\mathbb{C}}_{p_\ell+2}$. That final rounding adds at most $0.177 \cdot 2^{-p_\ell}\mathcal{A}$, giving total error strictly below $0.267 \cdot 2^{-p_\ell}\mathcal{A} < 2^{-p_\ell}\mathcal{A}$. □

We close the section by noting that the same precision schedule governs the written proof, the Lean 4 formalization, and the C/GMP implementation.

Remark 6.6 (Shared precision schedule). Because the additive increments $p_{\mathrm{work},\ell} - p_\ell = K_\ell + 6$ and $p_{\ell+1} - p_\ell = 2K_\ell + 12$ in (34) bound the worst-case ℓ_∞ alias amplification ($\le 2^{-7} \cdot 2^{-p_\ell}\mathcal{A}$) and the worst-case ℓ_∞ rounding and child-DFT amplification ($\le 200 \cdot 2^{-K_\ell - 12} \cdot 2^{-p_\ell}\mathcal{A} < 2^{-6} \cdot 2^{-p_\ell}\mathcal{A}$) for all $K_\ell \ge 2$ with more than a factor-of-five total margin ($< 0.197 \cdot 2^{-p_\ell}\mathcal{A}$), the exact same precision schedule (34) and root precision $p_0 = 2b + 2K_0 + 6$ (with window radius $m_{1,\ell} \in [\lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \rceil, p_{\ell+1}]$) are used across our written proof, our Lean 4 formalization [LeanMS26] (Section 8), and our C/GMP implementation [CGMP26] (Section 9). As noted in Section 9, the leaf threshold $R_\ell \le 4p_\ell^2$ of the written proof and of [LeanMS26] (`isBaseLeaf`, Table 3) is lowered by default to `bt` $= p_0^2$ in [CGMP26], and, as noted in Section 8, the tie-breaking rule on half-integer midpoints differs slightly across implementations.

7 The algorithm and the $O(n \log n)$ bound

This section assembles the subroutines of Sections 3–6 into the complete integer multiplier `Multiply` and proves Theorem 1. Section 7.1 presents the top-level pseudocode for `Multiply` and the recursive

transform `FastDFT` in Listing 1, linking each call to its Turing-machine realization. Section 7.2 then proves bit-exact recovery of $A \cdot B$ at root precision $p_0 = 2b + 2K_0 + 6$ and bounds the total running time on a deterministic multitape Turing machine. To avoid circularity when multiplying polylogarithmic-bit integers inside the transform without growing the tape count, the cost proof uses a fixed three-stage bootstrap on $O(1)$ dedicated tape banks. Readers who have followed the component bounds in Sections 3–6 can read this section straight through as the end-to-end synthesis.

7.1 Algorithm specification

We first record the complete top-level procedure and trace where each subroutine is defined. Listing 1 specifies `Multiply` and its recursive transform `FastDFT` in executable Python syntax. Inputs of bit-length $2 \le n \le 64$ are multiplied directly by schoolbook multiplication in $O(1)$ Turing machine steps. For $n > 64$, `BuildSharedLevelTables` precomputes all per-level tables once on the table tape using Section 3.3. The recursion tree then executes via the breadth-first tape sweeps of Appendix A.2. Inside each non-leaf call to `FastDFT`, Step 1 runs `OverlapAddSignedKroneckerRowSpread` from Section 6.2 under the additive precision schedule (34) of Section 6.3. Step 2 evaluates the two-dimensional transform via the polynomial-transform reduction of Section 4.1 and the odd-stride routing of Section 3.4. Step 3 finishes the level by running `CyclicSolve` and `NyquistSolve` from Section 5.2 and transposing the output back to natural frequency order. At the leaves $R \le 4p_{\text{bits}}^2$, `FastDFT` calls `BaseRadix2FFT` as analyzed in part 2 of Lemma 6.5.

Listing 1: The top-level multiplier `Multiply` and the recursive transform `FastDFT`, in Python syntax. `Multiply` packs the normalized digits of A and B into one complex vector and calls `FastDFT` twice, following Lemma 3.4.

```
def Multiply(A: int, B: int, n: int) -> int:
    """
    Multiply two n-bit non-negative integers A, B in O(n log n) steps.
    Uses two-call complex forward packing and root precision p0 = 2b + 2K0 + 6.
    """
    if n <= 64:
        return A * B
    b = max(4, n.bit_length() - 1)                # Chunk size b = floor(log_2 n)
    N = (n + b - 1) // b                          # Number of chunks
    K0 = (2 * N - 1).bit_length()
    R0 = 1 << K0                                  # Smallest power of 2 >= 2N
    p0 = 2 * b + 2 * K0 + 6                       # Root precision p0 = Theta(log n)

    # 1. Normalize chunks by 2^{-b} into [0, 1) and pack A + i*B into one complex vector
    x_AB = PackNormalizedComplex(A, B, b, R0, p0)

    # 2. Build shared per-level tables once, then make two top-level calls to FastDFT
    tables = BuildSharedLevelTables(R0, p0)
    y_AB = FastDFT(x_AB, R0, p0, tables)
    y_sq_conj = [FixedPointSquare(y_AB[u], p0 + 2).conjugate() for u in range(R0)]
    z_out = FastDFT(y_sq_conj, R0, p0, tables)

    # 3. Scale imaginary part by -2^{2b - 1} * R0^2, round to nearest integers, propagate carries
    c_chunks = [RoundToNearestInt(-(1 << (2 * b - 1)) * (R0 * R0) * z.imag, p0 + 2) for z in z_out]
    return PropagateCarries(c_chunks, b)


def FastDFT(x: list, R: int, p_bits: int, tables: dict) -> list:
    """
    Recursive 1D power-of-two DFT F_R(x) returning accuracy 2^{-p_bits} * max(1, ||x||_inf).
    Uses windowed sinc-Gaussian spreading and the additive precision schedule.
    """
    if R <= 4 * p_bits * p_bits:
        return BaseRadix2FFT(x, R, p_bits, tables)
```

```
    K = R.bit_length() - 1
    t1 = 1 << (K // 2)                          # t1 = 2^{floor(K/2)}
    r  = 1 << ((K + 1) // 2)                    # r  = 2^{ceil(K/2)}, t1 | r, R = t1 * r

    # Additive precision schedule: p_child = p_bits + 2*K + 12
    p_work  = p_bits + K + 6
    p_child = p_work + K + 6
    q_child = p_child + 2

    # Save exact Nyquist spatial alternating sum A_N before modifying input layout
    A_N = ExactNyquistAlternatingSum(x, t1, r, q_child)

    # Step 1 (row spreading): sinc-Gaussian fractional shift via overlap-add and signed Kronecker packing
    x_tilde = OverlapAddSignedKroneckerRowSpread(x, t1, r, p_work, p_child, tables)

    # Step 2 (polynomial-transform 2D DFT over C[y]/(y^r + 1), Steps 2a--2d):
    # Step 2a (pre-twist)
    U_pre  = PreTwist(x_tilde, t1, r, q_child, tables)
    # Step 2b (polynomial transform)
    Q      = PolyTransform_Radix2_Mod_YrPlus1(U_pre, t1, r, q_child)
    # Step 2c (post-twist)
    Q_post = PostTwist(Q, t1, r, q_child, tables)
    # Step 2d (recursive DFTs and odd-stride permutation)
    C_child = [FastDFT(Q_post[v1], r, p_child, tables) for v1 in range(t1)]
    Y = OddStrideColumnRoute(C_child, t1, r)

    # Step 3 (column deconvolution): two-term cyclic and Nyquist solvers + output transpose
    Z_cols = SolveTwoTermBidiagonalOutput(Y, A_N, t1, r, p_work, p_child, p_bits, tables)
    return TransposeColumnMajorToRowMajor(Z_cols, t1, r)
```

7.2 Cost of the algorithm

We now combine the forward error bound of Lemma 6.5 with the per-level step counts to bound the total running time of `Multiply`. Specifically, we establish the explicit two-term complexity bound

$$\mathcal{C}_{\mathrm{mul}}(n) = O(n \log n) + O\big(n(\log\log n)^2\big) = O(n \log n) \tag{37}$$

on a deterministic multitape Turing machine with a fixed number of tapes.

Theorem 1, restated next from Section 1.2, asserts that a single deterministic multitape Turing machine multiplies any two n-bit integers in at most $Cn \log_2 n$ steps. Proving this bound requires checking three points. First, root precision $p_0 = 2b + 2K_0 + 6$ suffices to round every convolution chunk to its exact integer value. Second, the multiplication-free polynomial transforms and tape-routing passes sum geometrically across levels to $O(n \log n)$ steps. Third, a three-stage bootstrap handles all polylogarithmic-bit inner multiplications in $O\big(n(\log\log n)^2\big)$ steps on $O(1)$ tapes.

Theorem 1 (restated). *There is a deterministic multitape Turing machine and a constant $C > 0$ such that, for every $n \geq 2$ and any two n-bit integers A and B, given the binary representations of A and B on the input tape, the machine writes the $2n$-bit binary product on the output tape in at most $Cn \log_2 n$ steps.*

The Remarks following Theorem 1 in Section 1.2 summarize what the algorithm uses and what it avoids.

Proof idea. The proof has three parts. First, we sum the additive precision increments across the $L = O(\log\log n)$ recursion levels to show $p_\ell = \Theta(\log n)$ everywhere. Lemma 3.4 and Lemma 6.5 then imply that two root calls to `FastDFT` recover every integer chunk convolution entry with pre-rounding error below $3.19/128 < 1/2$. Second, because the grid never expands, the R_0 active entries of width $\Theta(\log n)$ bits incur $O(n \log t_{1,\ell})$ polynomial-transform and routing steps at depth ℓ, which telescope

geometrically to $O(n \log n)$. Third, a three-stage hierarchy $\texttt{BasicMul} \to \texttt{Multiply}_1 \to \texttt{Multiply}_2$ on three disjoint banks of $O(1)$ tapes bounds all inner polylogarithmic-bit multiplications and leaf transforms by $O\big(n(\log\log n)^2\big)$ without circularity.

Proof. For $2 \le n \le 64$, schoolbook multiplication computes $A \cdot B$ in $O(1)$ Turing machine steps. Assume henceforth that $n > 64$, which implies $b \ge 6$, $K_0 \ge 5$, and $p_0 \ge 28$.

1. Additive precision schedule and exact root integer recovery. At the root level $\ell = 0$, we have $b = \lfloor \log_2 n \rfloor = \Theta(\log n)$ and $N = \lceil n/b \rceil$. The root transform length is $R_0 = 2^{K_0} = \Theta(n/\log n)$ with $2N \le R_0 < 4N$, and the root target precision is $p_0 = 2b + 2K_0 + 6 = \Theta(\log n)$. At recursion depth ℓ, the exponent satisfies $K_{\ell+1} = \lceil K_\ell/2 \rceil$, so $K_\ell = \lceil K_0/2^\ell \rceil \le K_0/2^\ell + 1$. The recursion therefore reaches a leaf $R_L \le 4p_L^2$ at depth $L = O(\log K_0) = O(\log\log n)$. Summing the additive increments (34) gives

$$p_\ell = p_0 + \sum_{j=0}^{\ell-1}(2K_j + 12) \le p_0 + 4K_0 + 14\ell = \Theta(\log n), \qquad R_0\, p_\ell = \Theta(n),$$

uniformly across all levels $\ell \in \{0, \dots, L\}$.

At the root, the normalized chunks $\tilde{a}_j = a_j 2^{-b}$ and $\tilde{b}_j = b_j 2^{-b}$ lie in $[0, 1)$ for $0 \le j < N$, and are padded with zeros for $N \le j < R_0$. Since $b < p_0 + 2$, every entry is represented exactly on $\widetilde{\mathbb{C}}_{p_0+2}$. Their packed vector $\tilde{x}_{AB} = \tilde{a} + i\tilde{b}$ satisfies $\|\tilde{x}_{AB}\|_\infty < \sqrt{2}$. Because at least half of the $R_0 \ge 2N$ entries are zero, we have

$$\|F_{R_0}\tilde{x}_{AB}\|_\infty \le \frac{1}{R_0}\sum_{j=0}^{N-1}|\tilde{x}_{AB,j}| < \frac{N}{R_0}\sqrt{2} \le \frac{1}{\sqrt{2}}.$$

Let $\epsilon := 2^{-p_0}$. By Lemma 6.5, the first call $\hat{y}_{AB} = \texttt{FastDFT}(\tilde{x}_{AB}, R_0, p_0)$ satisfies $\|\hat{y}_{AB} - F_{R_0}\tilde{x}_{AB}\|_\infty \le \sqrt{2}\epsilon$. Pointwise squaring and conjugating on $\widetilde{\mathbb{C}}_{p_0+2}$ via $\widehat{w}_u := \overline{\mathrm{RM}_{p_0+2}(\hat{y}_{AB,u}, \hat{y}_{AB,u})}$ incurs error

$$|\widehat{w}_u - \overline{(F_{R_0}\tilde{x}_{AB})_u^2}| \le 2\|F_{R_0}\tilde{x}_{AB}\|_\infty(\sqrt{2}\epsilon) + (\sqrt{2}\epsilon)^2 + \frac{\sqrt{2}}{8}\epsilon < \left(2 + 2\epsilon + \frac{\sqrt{2}}{8}\right)\epsilon < 2.19\epsilon.$$

Consequently $\|\widehat{w}\|_\infty < (1/\sqrt{2})^2 + 2.19\epsilon < 1$, so $\max(1, \|\widehat{w}\|_\infty) = 1$. The second call $\hat{z}_{\text{out}} = \texttt{FastDFT}(\widehat{w}, R_0, p_0)$ therefore adds error at most $1 \cdot \epsilon$, giving

$$\big\|\hat{z}_{\text{out}} - F_{R_0}\big(\overline{(F_{R_0}\tilde{x}_{AB})^2}\big)\big\|_\infty < 3.19\epsilon.$$

By Lemma 3.4, the exact integer chunk convolution $c = a \circledast b$ equals $-2^{2b-1}R_0^2\,\mathrm{Im}\big(F_{R_0}(\overline{(F_{R_0}\tilde{x}_{AB})^2})\big)$. Multiplying $-\mathrm{Im}(\hat{z}_{\text{out}})$ by $2^{2b-1}R_0^2 = 2^{2b+2K_0-1}$ produces each integer convolution coefficient c_j with pre-rounding error strictly less than

$$3.19 \cdot 2^{2b+2K_0-1} \cdot 2^{-p_0} = 3.19 \cdot 2^{-7} = \frac{3.19}{128} = 0.024921875 < \frac{1}{2}.$$

Rounding to the nearest integer therefore recovers every integer $c_j \in \mathbb{Z}$ exactly. Linear carry propagation in radix 2^b then outputs $A \cdot B$ in $O(n)$ bit operations.

2. Geometric $(1/2)^\ell$ contraction of polynomial-transform and tape-routing work to $O(n \log n)$. At depth $\ell \in \{0, \dots, L-1\}$, there are $\frac{R_0}{R_\ell}$ subproblems of length $R_\ell = t_{1,\ell} r_\ell$. Because there is no grid expansion, the total number of active complex entries at depth ℓ is $\frac{R_0}{R_\ell} \cdot R_\ell = R_0$. By (5), every scalar record at depth ℓ has $O(p_{\ell+1} + H_\ell) = O(p_{\ell+1}) = O(\log n)$ bits.

Four movement and transform passes run at each node: multitape input row de-interleaving, the $\log_2 t_{1,\ell}$-stage radix-2 polynomial transform $G_{t_{1,\ell}}$, odd-stride merge-sort column routing, and the final column-to-row output transpose. By Lemma 3.3 and Lemma 3.5, each of those four passes takes $O\big(R_\ell p_{\ell+1}(1+\log_2 t_{1,\ell})\big)$ bit operations per node. Furthermore, $\log_2 t_{1,\ell} = \lfloor K_\ell/2 \rfloor \le K_0/2^{\ell+1} + 1$. Since $R_0 p_{\ell+1} = \Theta(n)$ holds uniformly across all ℓ, summing over all $\frac{R_0}{R_\ell}$ nodes at depth ℓ yields the geometric work envelope

$$\mathcal{C}_{\text{syn+route}}(\ell) = \frac{R_0}{R_\ell} \cdot O\big(R_\ell p_{\ell+1}(1+\log_2 t_{1,\ell})\big) = O\left(n\left(\frac{K_0}{2^{\ell+1}} + 1\right)\right).$$

We now sum this bound across all non-leaf depths. Using the telescoping sum

$$\sum_{\ell=0}^{L-1} \log_2 t_{1,\ell} = K_0 - K_L \le K_0 = \log_2 R_0 = O(\log n),$$

summing over $\ell = 0, \ldots, L-1$ gives

$$\mathcal{C}_{\text{syn+route}} = \sum_{\ell=0}^{L-1} \mathcal{C}_{\text{syn+route}}(\ell) = O(nK_0 + nL) = O(n \log n). \tag{38}$$

3. Secondary work of $O(n(\log\log n)^2)$ and constant-tape discipline via three-stage bootstrap. By (8) and Appendix A.2, shared per-level table generation for the outer call takes $o(n)$ Turing machine steps. Sequential level-table replay, including per-stage leaf twiddle list replay, takes $O(n \log\log n)$ steps. Every non-table integer multiplication inside the root pointwise squaring, Steps 1, 2a, 2c, 3, and the leaf `BaseRadix2FFT` at depth ℓ has operand bit-length at most

$$N_\ell = O(\tilde{W}_\ell p_{\ell+1}) = O(p_L^2) = O((\log n)^2) \ll n.$$

To establish (37) without circularity in the asymptotic constant and with a fixed constant number of tapes, we define a three-stage hierarchy of multiplier procedures $\texttt{Multiply}_2 \to \texttt{Multiply}_1 \to$ `BasicMul`. Each stage receives its own constant bank of work tapes. Three disjoint banks require $3 \times O(1) = O(1)$ tapes in total. Whenever an outer sweep invokes an inner N-bit multiplier, it first copies the two N-bit operands onto the callee stage's input tape. It next runs the callee on its dedicated tape bank and copies the $2N$-bit product back. Finally, it erases and rewinds the callee's tapes in $O(\text{space}) \le O(\mathcal{C}_{\text{callee}}(N))$ steps. The three stages operate as follows:

- *Bottom stage, `BasicMul`:* This stage multiplies two N-bit integers using the two-call root reduction directly to `BaseRadix2FFT` of length $R_0 = \Theta(N/\log(2N))$ at precision $q_b = \Theta(\log(2N))$. For $N \le 64$, `BasicMul` multiplies directly by schoolbook as in Listing 1. For $N > 64$, its accuracy follows from the `BaseRadix2FFT` clause in part 2 of Lemma 6.5. That clause holds for every length 2^K with $K \ge 2$. All butterfly twiddle products, root pointwise squarings, and root table evaluations are performed by schoolbook multiplication and division. By Lemma 3.2, evaluating each stage twiddle at $D = \Theta(\log(2N))$ takes $O(D^3)$ steps. Across all $R_0 - 1 < R_0$ stage twiddles, root table evaluation therefore costs $O\big(R_0(\log(2N))^3\big) = O\big(N(\log(2N))^2\big)$ steps. By Appendix A.2, executing $\log_2 R_0$ breadth-first `BaseRadix2FFT` stages of $R_0/2$ schoolbook $O(q_b)$-bit butterfly products and $O(R_0 q_b)$-bit scans also costs $O\big(R_0(\log R_0)q_b^2\big) = O\big(N(\log(2N))^2\big)$ steps. Hence the total step count $M_0(N)$ of `BasicMul` satisfies

$$M_0(N) = O\big(N(\log(2N))^2\big).$$

- *Middle stage, $\texttt{Multiply}_1$:* This stage runs the recursive algorithm of Listing 1 on its dedicated tape bank. It invokes `BasicMul` with cost M_0 for all inner integer products at the root, in Steps 1–3, and at the leaves. By (8), building $\texttt{Multiply}_1$'s internal shared tables for an N-bit input takes $O(\sqrt{N}(\log(2N))^{7/2}\log\log(4N)) = o(N)$ steps. Root chunk packing and final scaling, rounding, and carry propagation take $O(N)$ steps. Moreover, the $R_0(N) = \Theta(N/\log(2N))$ root pointwise squarings and conjugations via M_0 take $R_0(N)\cdot O(M_0(p_0(N))) = O(N(\log\log(4N))^2)$ steps. At each level $\ell < L(N)$ of an N-bit call to $\texttt{Multiply}_1$, the inner products have bit-length $O((\log(2N))^2)$. Evaluating each inner product via M_0 thus takes $M_0(O(\tilde{W}_\ell p_{\ell+1})) = O(\tilde{W}_\ell p_{\ell+1}(\log\log(4N))^2)$ steps. By Theorem 6.3 and (38), summing over $L(N) = O(\log\log(4N))$ levels plus the root and leaf levels gives

$$\mathcal{C}_1(N) = O(N\log(2N)) + O\big(N(\log\log(4N))^3\big) = O\big(N\log(2N)\big).$$

- *Top stage, $\texttt{Multiply} := \texttt{Multiply}_2$:* This stage runs the recursive algorithm on the n-bit input A and B. For all inner integer products it invokes $\texttt{Multiply}_1$, whose bound $\mathcal{C}_1(N) = O(N\log(2N))$ already includes $\texttt{Multiply}_1$'s internal table-building and tape-reset work:

  (a) *Root packing, pointwise squaring, and carry propagation:* `PackNormalizedComplex` and `PropagateCarries` take $O(R_0p_0 + n) = O(n)$ steps. The $R_0 = \Theta(n/\log n)$ root pointwise squarings and conjugations on $\widetilde{\mathbb{C}}_{p_0+2}$ via $\texttt{Multiply}_1$ cost $R_0\cdot O(\mathcal{C}_1(p_0)) = O(R_0p_0\log p_0) = O(n\log\log n)$ steps.

  (b) *Per-level multiplications across $\ell = 0,\dots,L-1$:* At level ℓ, the $O(R_\ell p_{\ell+1}\log_2 t_{1,\ell})$ row de-interleaving scan is already counted in $\mathcal{C}_{\text{syn+route}}$ in Part 2. By Theorem 6.3 with $\mathsf{M} = \mathcal{C}_1$, the packing, Kronecker multiplication, and overlap-add work in Step 1 across all $\frac{R_0}{R_\ell}$ nodes costs

  $$\frac{R_0}{R_\ell}\cdot O\Big(\frac{R_\ell}{\tilde{W}_\ell}\cdot\tilde{W}_\ell p_{\ell+1}\log(\tilde{W}_\ell p_{\ell+1})\Big) = O(R_0p_{\ell+1}\log p_{\ell+1}) = O(n\log\log n)$$

  steps. Likewise, the $2R_\ell$ pre-twists and post-twists and the at most $3R_\ell$ real-by-complex solver products per node each multiply $O(p_{\ell+1})$-bit integers via $\texttt{Multiply}_1$ in $O(p_{\ell+1}\log p_{\ell+1})$ steps. Across all nodes at level ℓ, those scalar products also cost $O(R_0p_{\ell+1}\log p_{\ell+1}) = O(n\log\log n)$ steps. Summing over all $L = O(\log\log n)$ levels gives $\mathcal{C}_{\text{ovh}} = O(n(\log\log n)^2)$.

  (c) *Leaf base-case FFTs at $\ell = L$, where $R_L \le 4p_L^2$:* At depth L, we have

  $$1 + K_L = 1 + \log_2 R_L = O(\log p_L) = O(\log\log n), \qquad q_b = \Theta(\log n).$$

  Executing `BaseRadix2FFT` across all $\frac{R_0}{R_L}$ leaves costs

  $$\mathcal{C}_{\text{base}} = \frac{R_0}{R_L}\cdot O\big(R_L(1+K_L)\,\mathcal{C}_1(q_b)\big) = O\big(R_0(\log\log n)\,p_L\log p_L\big) = O(n(\log\log n)^2).$$

Combining $\mathcal{C}_{\text{syn+route}} + \mathcal{C}_{\text{ovh}} + \mathcal{C}_{\text{base}} + \mathcal{C}_{\text{tables}}$ proves (37) and Theorem 1. □

8 Formal verification in Lean 4

This section describes the formal proof of Theorem 2 in the Lean 4 theorem prover [dMU21] on top of the Mathlib mathematical library [Mat20]. The development [LeanMS26] forms the library `IntMul`, consisting of `IntMul.lean` and 26 modules under `IntMul/`, or 27 files and 82,377 lines in total. No familiarity with Lean is assumed below, and we explain every quoted formal declaration in plain computational terms. We present in turn what was proved, what must be trusted, how the proof and its toolchain were checked, how the library is organized, and where the formal construction departs from the written text.

What was proved. The headline formal theorem is `IntMul.tmIntMul_evalsToInTime_n_log_n` in `IntMul/TMEndToEnd.lean`. It concerns one concrete, parameter-free eight-stack binary machine `tmIntMul`, defined in the same file in Mathlib's `Turing.FinTM2` model. For every bit length $n \geq 2$ and all n-bit integers $A, B < 2^n$, it proves that `tmIntMul` halts with the exact $2n$-bit product $A \cdot B$ on its output stack in at most $C \cdot n\lfloor \log_2 n \rfloor$ steps. Below we quote verbatim the bit-encoding definitions from `IntMul/Defs.lean` and the machine and complexity declarations from `IntMul/TMEndToEnd.lean`. Definitions appear with their bodies and theorems as statements only, while docstrings and proofs are omitted:

```
def encodeNatBits (w : ℕ) (x : ℕ) : List Bool :=
  List.ofFn (fun i : Fin w => x.testBit i.val)

def encodeInputBits (n A B : ℕ) : List Bool :=
  encodeNatBits n A ++ encodeNatBits n B

def tmIntMul : Turing.FinTM2 :=
  (bmRunRaw bmIntMulCore).toFinTM2

theorem isUniformFinTM2_tmIntMul :
    IsUniformFinTM2 tmIntMul (bmRunRaw bmIntMulCore).numLabels 32 2

theorem tmIntMul_evalsToInTime_n_log_n :
    ∃ C > 0, ∀ n : ℕ, 2 ≤ n → ∀ A B : ℕ, A < 2 ^ n → B < 2 ^ n →
      ∃ t ≤ C * n * Nat.log 2 n,
        Turing.EvalsToInTime tmIntMul.step
          (Turing.initList tmIntMul (encodeInputBits n A B))
          (some (Turing.haltList tmIntMul (encodeNatBits (2 * n) (A * B)))) t
```

Each piece of the statement has a direct meaning on binary lists. The function `encodeNatBits w x`, which equals `(List.range w).map (x.testBit ·)`, writes x as a fixed-width w-bit little-endian list of booleans. The input encoding `encodeInputBits n A B` concatenates the two n-bit inputs on stack 0, while `Turing.initList` initializes all remaining stacks to the empty list. At termination, `Turing.haltList tmIntMul s` requires the control label to be `none`, the internal register to be reset to `initialState`, the output list `s` to sit on the output stack, and every other stack to be empty. In the definition of `tmIntMul`, `bmIntMulCore` is our structured eight-stack multiplication program, and `bmRunRaw` compiles it into a flat `Turing.FinTM2` machine.

The constant C in the formal proof is witnessed by the closed arithmetic expression `C_tm_total` = `C_tm_d2` + 500. The term `C_tm_d2` is the product of three factors: the constant `C_tm_d1` $\approx 3.1 \cdot 10^{22}$ of the first bootstrap tier (see departure (2) below), the factor $1.5 \cdot 10^{13}$, and twice the counted-operation constant `C_nested_total` $\approx 5.4 \cdot 10^{56}$. The witness evaluates to about $5.0 \cdot 10^{92}$; no attempt was made to optimize it.

In Mathlib, `Turing.EvalsToInTime f a b m` is a data record of kind `Type` rather than a logical proposition of kind `Prop`. It stores an execution of the step function `f` from state `a` to state `b` taking at most `m` steps. Because an existential quantifier expects a logical proposition, our `CoeDep` type-conversion rule maps that record type to `Nonempty`, asserting that at least one valid execution exists. Below we quote verbatim this conversion rule and the uniformity predicate from `IntMul/TMPrimitives.lean`, followed by the elementary-operation counter and step-bound predicate from `IntMul.lean`:

```
instance instCoeDepEvalsToInTimeProp {σ : Type u} (f : σ → Option σ) (a : σ) (b : Option σ)
    (m : ℕ) : CoeDep Type (Turing.EvalsToInTime f a b m) Prop where
  coe := Nonempty (Turing.EvalsToInTime f a b m)
```

```
def IsUniformFinTM2 (tm : Turing.FinTM2) (maxLabels maxStates maxAlpha : ℕ) : Prop :=
  Fintype.card tm.Λ ≤ maxLabels ∧
  Fintype.card tm.σ ≤ maxStates ∧
  Fintype.card (tm.Γ tm.k₀) ≤ maxAlpha ∧
  Fintype.card tm.K ≤ 8 ∧
  ∃ inst : ∀ k : tm.K, Fintype (tm.Γ k),
    ∀ k : tm.K, @Fintype.card (tm.Γ k) (inst k) ≤ maxAlpha

def stmtMaxElemOps {K : Type*} {Γ : K → Type*} {Λ σ : Type*} :
    Turing.TM2.Stmt Γ Λ σ → ℕ
  | .push _ _ q => 1 + stmtMaxElemOps q
  | .peek _ _ q => 1 + stmtMaxElemOps q
  | .pop _ _ q => 1 + stmtMaxElemOps q
  | .load _ q => 1 + stmtMaxElemOps q
  | .branch _ q₁ q₂ => 1 + max (stmtMaxElemOps q₁) (stmtMaxElemOps q₂)
  | .goto _ => 1
  | .halt => 1

def FinTM2BoundedElemOps (M : Turing.FinTM2) (C_step : ℕ) : Prop :=
  ∀ l : M.Λ, stmtMaxElemOps (M.m l) ≤ C_step
```

Here `Fintype.card` denotes finite set cardinality, and `tm.Λ`, `tm.σ`, `tm.Γ`, and `tm.K` are the machine's label, register-state, stack-alphabet, and stack-index types. Theorem `isUniformFinTM2_tmIntMul` certifies at most eight stacks, stack alphabets of size at most 2, and at most 32 register states. It also certifies a fixed finite control of `(bmRunRaw bmIntMulCore).numLabels` labels, independent of n, A, B. Concretely, the stack index type `TapeIdx` has eight `Bool` stacks directly by definition, proved by Lean's reflexivity check `rfl`, and the register type `DelimReg` has cardinality `Fintype.card DelimReg = 32` by theorem `card_delimReg`.

In Mathlib's `Turing.TM2` model, one `TM2.step` runs the entire branching statement tree attached to one control label, so bounding steps alone would not rule out large trees. Using `stmtMaxElemOps` and `FinTM2BoundedElemOps` to bound primitive stack actions per label, `IntMul.lean` lifts the step bound to total elementary stack operations. Below we quote that theorem together with the type signature of `eightStacksEquivFourBiInfiniteTapes`, omitting its short proof and definition body:

```
theorem tmIntMul_elementary_stack_ops_n_log_n :
    ∃ C_step C_time C_ops : ℕ, 0 < C_step ∧ 0 < C_time ∧ 0 < C_ops ∧
      FinTM2BoundedElemOps tmIntMul C_step ∧
      ∀ n : ℕ, 2 ≤ n → ∀ A B : ℕ, A < 2 ^ n → B < 2 ^ n →
        ∃ t ≤ C_time * n * Nat.log 2 n,
          C_step * t ≤ C_ops * n * Nat.log 2 n ∧
          Turing.EvalsToInTime tmIntMul.step
            (Turing.initList tmIntMul (encodeInputBits n A B))
            (some (Turing.haltList tmIntMul (encodeNatBits (2 * n) (A * B)))) t

def eightStacksEquivFourBiInfiniteTapes :
    (Fin 8 → List Bool) ≃ (Fin 4 → List Bool × List Bool)
```

The declaration `eightStacksEquivFourBiInfiniteTapes` only pairs eight boolean lists into four left-right list pairs; it is not a machine simulation theorem. A k-stack machine over finite alphabets is simulated by a k-tape Turing machine with constant-factor slowdown, using one tape per stack, and the formal statement is about the stack machine `tmIntMul`. That standard simulation is outside our formalization.

What must be trusted. Accepting Theorem 2 does not require reading the 82,377 lines of Lean proof scripts. Instead, the reader trusts four small components. First is Lean's type-checking kernel, which verifies every proof term from first principles. Second are the Lean standard-library and Mathlib definitions referenced in the quoted statements, including `Turing.FinTM2`, `Turing.EvalsToInTime`, `Turing.initList`, `Turing.haltList`, `Turing.TM2.Stmt`, `Fintype.card`, `Nat.log`, and `List`. Third is the faithfulness of our short bit-encoding, uniformity, and elementary-operation definitions quoted verbatim above. Fourth are Lean's three standard logical axioms: propositional extensionality `propext`, classical choice `Classical.choice`, and quotient soundness `Quot.sound`.

How it was checked and toolchain provenance. None of the 27 library files uses any proof-skipping placeholder, extra axiom, or unverified code-execution hook: there are no `sorry`, `admit`, `axiom`, `native_decide`, `implemented_by`, `extern`, `opaque`, `unsafe`, or `partial` declarations. The umbrella file `IntMul.lean` defines a compile-time command `#verify_standard_axioms` on top of `Lean.collectAxioms`. At the end of `IntMul.lean`, that command checks `tmIntMul`, `isUniformFinTM2_tmIntMul`, `tmIntMul_evalsToInTime_n_log_n`, and `tmIntMul_elementary_stack_ops_n_log_n`. It likewise checks `intMul_complete_endToEnd_summary` and `intMul_endToEnd_unconditional_summary`. Compilation fails if the transitive axiom closure of any of those six declarations contains anything beyond `propext`, `Classical.choice`, and `Quot.sound`. The file then runs `#print axioms` on the same six declarations.

The library itself leaves the finite-control size `(bmRunRaw bmIntMulCore).numLabels` symbolic, because running Lean's built-in evaluator `#eval` directly would enumerate about 10^9 control labels. Instead, the snapshot includes a separate check file `checks/EvalProven.lean`, which is not one of the 27 library files and is not part of the package build command `lake build`. Run after building via `lake env lean checks/EvalProven.lean`, this metaprogram folds label counts and statement-depth maxima over the machine's syntax tree.

In Lean, elaboration is the front-end phase that builds a candidate proof term before handing it to the trusted kernel. The check file's proof-generating elaborator uses compiled native helpers via `unsafe`, `implemented_by`, and `opaque` during elaboration to traverse the syntax tree quickly. However, the two proof terms it produces are checked from scratch by the Lean kernel, and its own `#print axioms` commands report only the three standard axioms for them. Those two theorems prove `(bmRunRaw bmIntMulCore).numLabels = 1041232178`, so the finite control has about 10^9 labels, and `finTM2MaxStmtElemOps tmIntMul = 16`. To guarantee a positive bound for any machine without a separate nonemptiness proof, the general lifting lemma `finTM2BoundedElemOps_self` adds 1 and uses $C_{\text{step}} = \texttt{finTM2MaxStmtElemOps tmIntMul} + 1 = 17$, while 16 also satisfies `FinTM2BoundedElemOps`.

The development was carried out under Lean 4.27.0 and Mathlib revision `a3a10db0`, and ported to Lean 4.33.1 and Mathlib `0df444a3`. No theorem statement was changed in the port: we verified that the declaration headers recorded in `SIGNATURES.txt` are byte-identical to those of the original development, which is not part of the snapshot.

To build under the newer toolchain, the port adjusted imports, automatic typeclass lookup for instance resolution, and proof scripts. In 18 files it sets `backward.isDefEq.respectTransparency false` so front-end proof search matches pre-4.3x rules when filling in omitted arguments. Those files also raise the front-end search-step budget `maxHeartbeats` and recursion-depth budget `maxRecDepth`. These options affect elaboration only, and every finished proof term is still re-checked from scratch by the Lean kernel. The distributed snapshot `d7d142b0db9b` [LeanMS26] compiles with `lake build` with exit code 0, and `#print axioms` reports only `propext`, `Classical.choice`, and `Quot.sound`

Table 3: Map from the main definitions, lemmas, and theorems of this paper to their formal counterparts in the `IntMul` Lean 4 library. Where the formal proof uses a looser numerical constant than the written text, the right-hand column records both values.

Paper statement	Lean 4 declarations and constant notes
Definition 3.1, schedule (34)	`cSpec`, `pWorkEll`, `pChildEll`, `qChildEll`, `isBaseLeaf`, `roundShiftInt` (`Defs.lean`; identical schedule and leaf predicate $2^{K_\ell} \le 4p_\ell^2$ (`isBaseLeaf`), floor-shifted tie rule $\lfloor 2^q v + \frac{1}{2} \rfloor$)
Lemma 3.2 (elementary series)	`abs_approxPiRat_sub_pi_le`, `abs_approxLogTwoRat_sub_log_two_le`, `abs_approxSqrtRat_sub_sqrt_le`, `abs_expTaylorRat_sub_exp_le`, `norm_expITaylorRat_sub_exp_le`, `abs_sincTaylorRat_sub_sinc_le`, `abs_erfIntegralTaylorRat_sub_integral_le` (`ComputableSchedule.lean`; rational series and integer-square-root bounds, with $\frac{\pi}{4} = \arctan\frac{1}{2} + \arctan\frac{1}{3}$)
Lemma 3.3 (transpose & odd-stride)	`transposeMatrix_spec`, `oddColGather_oddColScatter`, `oddColScatter_oddColGather` (`TMPrimitives.lean`, value-level); `bmTransposeMatrix_computesInTime` (`TMSpreadFFT.lean`), `bmOddColScatterCols_computesInTime` (`TMKronecker.lean`, stack-machine time bounds)
Lemma 3.4, Lemma 3.5	`twoCallDFTConv_re_eq_linearConvCoeff` (`FastDFT.lean`; `twoCallDFTConv` evaluates $R\sum_u \frac{(F_R z)_u^2}{2i} e^{-2\pi i u k/R} = R^2\,\overline{F_R\big((F_R z)^2/(2i)\big)}_k$ and reads the real part, which equals $-\frac{R^2}{2}\operatorname{Im} F_R\big((F_R z)^2\big)_k$, the identity (10), and states it for linear chunk convolutions, which coincide with the cyclic ones because $R_0 \ge 2N$); `radix2SyntheticFFTExact_eq_syntheticFFT`, `syntheticFFT_rounding_error_le` (`SyntheticFFT.lean`; bounds stage rounding by $\le k_{1,\ell}\frac{\sqrt{2}}{4}2^{-p_{\ell+1}} \le 0.18K_\ell 2^{-p_{\ell+1}}$ vs. $0.177k_{1,\ell}2^{-p_{\ell+1}}$)
Proposition 4.1, Theorem 4.2	`twisted_fourier_duality_reduction` (`SyntheticFFT.lean`); `gHat_eq_sub`, `untruncatedFilterResponse_eq_untruncatedPoissonModeSum`, `poisson_fractional_shift_identity` (`PoissonKernel.lean`)
Theorem 5.1 (passband, two-term, Nyquist)	`tsum_gHat_sub_int_eq_one`, `gHat_passband_max_ge` (≥ 0.49999999), `gHat_stopband_alias_lt` ($< 0.15 \cdot 2^{-p_{\mathrm{work}}}$ vs. $< 0.120 \cdot 2^{-p_{\mathrm{work}}}$), `gHat_ratio_pow_t1_le_015` ($a^{t_1} \le 0.15$ vs. < 0.14; conditioning $\le 2.36\,t_1$ in `gHat_deconvolution_conditioning_le_236_t1`); `nyquist_alternating_sum_eq`, `nyquistColumnSolve_spec`, `vecInfNorm_nyquistColumnSolve_le` ($t_1/c_0 \le 2.36\,t_1$ vs. $(t_1-1)/(2c) < 1.0000001\,t_1$)
Lemma 6.1, Lemma 6.2, Theorem 6.3	`filter_l1_norm_le` ($\le \frac{2}{\pi}\ln\alpha + 1.62$ vs. $< \frac{2}{\pi}\ln\alpha + 1.444$), `spatial_truncation_tail_le` ($< 0.4 \cdot 2^{-p_{\ell+1}}$ vs. $< 0.077 \cdot 2^{-p_{\ell+1}}$); `overlapAdd_eq_cyclicConv`, `kronecker_spread_exact_of_bounds`, `kronecker_spread_complexity_bound` (`KroneckerSpread.lean`)
Lemma 5.2, Lemma 6.5	`level_error_budget_le`, `base_leaf_error_budget_le` (formal statements prove the total $\le 2^{-p_\ell}$; the paper's margins are 0.197 and 0.267), `inductive_accuracy_closure` (`Deconvolution.lean`); `vecInfNorm_fastDFTRecursive_sub_normDFT1D_le`, `vecInfNorm_fastDFTRecursive_sub_normDFT1D_le_two_zpow` (`FastDFT.lean`)
Theorems 1 and 2	`reconstructProductViaFastDFT_eq_mul` (root margin $\le \frac{1}{8} < \frac{1}{2}$ vs. $< 3.19 \cdot 2^{-7}$); `computableNatMulTracked_spec`, `computableNatMulBitsTracked_spec` (`EndToEnd.lean`); `tmIntMul_evalsToInTime_n_log_n`, `tmIntMul_elementary_stack_ops_n_log_n`, `intMul_complete_endToEnd_summary`

for the headline theorems.

Structure of the library. Alongside the stack machine `tmIntMul`, the library verifies an executable functional multiplier in `IntMul/EndToEnd.lean`. Theorem `computableNatMulTracked_spec` proves its value-level correctness, and `computableNatMulBitsTracked_spec` proves its bottom-

up counted $O(n \log n)$ bit-operation bound. Next, `intMul_complete_endToEnd_summary` packages those functional bounds with `isUniformFinTM2_tmIntMul` and `tmIntMul_evalsToInTime_n_log_n`. A companion summary theorem, `intMul_endToEnd_unconditional_summary`, adds the uniform `FinTM2` primitive and composition suites.

All analytic and fixed-point estimates on which the machine proof rests are proved in the same library. These include the Poisson identity, passband, stopband, and conditioning bounds, spreading, multi-level accuracy, and root recovery. Table 3 maps the main statements of the paper to their Lean declarations and records every place where the formal constants are looser than the printed ones. In its bottom row, "Theorems 1 and 2" lists the formal counterparts of Theorem 1's ingredients, namely root recovery and counted operations, alongside Theorem 2; Theorem 1 itself is a multitape statement with no formal counterpart.

Departures from the written text. The formal construction departs from the written text in several low-level choices, none of which affects the $O(n \log n)$ statement of Theorem 2. As recorded in Table 3, elementary functions are evaluated by direct rational Taylor polynomials without range reduction for exp, $e^{i\theta}$, sinc, and $\int_0^z e^{-t^2}\,dt$, and π is computed from $\frac{\pi}{4} = \arctan\frac{1}{2} + \arctan\frac{1}{3}$. Five further differences appear in the machine construction:

(1) *Rounding.* Lean uses the floor-shifted tie rule $\lfloor 2^q v + \frac{1}{2} \rfloor$, formalized as `roundShiftInt`. By contrast, the C program uses MPFR nearest rounding `MPFR_RNDN` when quantizing precomputed tables and $\lfloor m/2^s + \frac{1}{2} \rfloor$ in fixed-point shifts. Definition 3.1 allows any fixed nearest tie rule.

(2) *Bootstrap.* The machine `tmIntMul` uses a two-tier self-recursive bootstrap with binary shift-add schoolbook multiplication at the bottom, giving secondary work $O(n(\log\log n)^4)$. Section 7 instead uses a three-stage bootstrap with secondary work $O(n(\log\log n)^2)$. In `IntMul/EndToEnd.lean`, `nestedSubMulBitOps_le_log_cube` bounds the middle-tier multiplier by $O(q(\log q)^3)$ per q-bit product. Theorem `log_rootK0_add_one_fourth_le` in the same file absorbs the resulting $(\log\log n)^4$ factor into $O(n \log n)$.

(3) *Input encoding.* Inputs use the fixed-width encoding `encodeInputBits n A B`, defined as the list `encodeNatBits n A ++ encodeNatBits n B` on the input stack. Bit i of each operand sits at position i, least significant bit first, and the output is `encodeNatBits (2 * n) (A * B)`, fixing little-endian boolean lists on stacks whereas Section 3.2 leaves the tape bit order unspecified.

(4) *Small inputs.* The machine `tmIntMul` handles every $n \geq 2$ uniformly without the $n \leq 64$ shortcut of the pseudocode.

(5) *Window radius.* The formal development uses the integer-square-root window radius `computeM1Ell K p` $= \lfloor \sqrt{p_{\mathrm{work},\ell}\, p_{\ell+1}} \rfloor + 1$. Theorems `m1Ell_le_computeM1Ell` and `computeM1Ell_le_pChildEll` place this value inside the admissible interval of Lemma 6.5. Remark 6.4 and the C program instead use the default radius $\lceil \alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi} \rceil$. Meanwhile, the Turing machine of Appendix A.2 evaluates a fixed-point approximation $\tilde{x}$ to $\alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi}$ and computes $\lceil \tilde{x} + 2^{-s-20} \rceil$, which equals that default ceiling or exceeds it by 1. Every statement in the paper holds for every admissible radius.

Separately, the companion paper's development [CLW26] contains its own kernel-checked multiplier for this algorithm.

9 Implementation and measurements

To test the algorithm on large inputs, we measure the C/GMP implementation `intmul_fastdft` [CGMP26], about 1,700 lines of C, in mode `algo=1`, which selects the windowed sinc-Gaussian kernel. It implements this paper's two-dimensional algorithm with the windowed sinc-Gaussian kernel and the additive precision schedule, delegating inner integer products to GMP. It does not implement the companion paper's d-dimensional algorithm [CLW26]. Across all tested sizes from 10^7 to 10^9 bits, every run agrees bit for bit with GMP while running 153 to 323 times slower on a single core.

Implementation choices and the billion-bit regime. Compared with the Turing machine specification in Sections 3–7, `intmul_fastdft.c` uses the identical additive schedule (34), namely $p_{\mathrm{work},\ell} = p_\ell + K_\ell + 6$, $p_{\ell+1} = p_\ell + 2K_\ell + 12$, and $p_0 = 2b + 2K_0 + 6$. It differs in four low-level engineering choices. Precomputed tables are evaluated with nearest rounding in MPFR, a multiple-precision floating-point library, and signed Kronecker slots are packed into non-negative GMP `mpz_mul` operands via a precomputed bias offset. In addition, the two-term column solver performs direct fixed-point division, and the default leaf cutoff is lowered from $4p_\ell^2$ to $\mathtt{bt} = p_0^2$ evaluated at the root p_0.

At $n = 10^9$ bits, the root parameters are $b = 29$, $K_0 = 27$, $R_0 = 2^{27}$, and $p_0 = 118$. Because $\mathtt{bt} = 13{,}924 < R_1 = 2^{14} = 16{,}384$, the 10^9-bit run exercises two recursion levels, $2^{27} \to 2^{14} \to 2^7 = 128$. At level 1, however, $R_1 = 2^{14} < 4p_1^2 = 135{,}424$, and the window width $W_1 = 191 > r_1 = 128$ exercises the folding case of Remark 6.4. Those level-1 nodes therefore lie outside the non-leaf hypothesis of Theorem 6.3 and Lemma 6.5(1), so the bit-exact agreement observed at 10^9 bits is an empirical observation rather than a consequence of the proofs. By contrast, the runs at 10^7, 10^8, and $5 \cdot 10^8$ bits treat level 1 as a leaf and stay within the parameter regime of the proofs, subject to the four implementation differences above.

Table 4: Single-threaded running times and memory usage of our C/GMP implementation `intmul_fastdft` compared against GMP's `mpz_mul`. Runs use mode `algo=1` with `OMP_NUM_THREADS=1` and `seed` 7 on an AMD EPYC 7B13, and every run printed `EXACT MATCH`. Slowdown ratios are those printed by the program from the unrounded timings, so recomputing them from the rounded timings shown here can differ by one in the last digit.

n (bits)	Chain (R_ℓ, p_ℓ)	Core (s)	End-to-end (s)	Tables (s)	`mpz_mul` (s)	Slowdown	Peak RSS
10^7	2^{20} (92) → 1024 (144)	9.63	9.92	1.82	0.0478	207×	0.56 GiB
10^8	2^{23} (104) → 4096 (162)	90.00	92.51	5.46	0.6059	153×	4.4 GiB
$5 \cdot 10^8$	2^{26} (114) → 8192 (178)	786.57	805.37	19.44	3.4417	234×	33.8 GiB
10^9	2^{27} (118) → 2^{14} (184) → 128 (224)	2574.67	2612.65	25.95	8.0969	323×	67.3 GiB

Timings and comparison with GMP. Table 4 gives the single-threaded timings with `OMP_NUM_THREADS=1`. Here "core" is the time in the two top-level `FastDFT` calls, and "tables" is the one-time filter and twiddle setup for that size. The "end-to-end" column includes chunk unpacking, both transforms, pointwise squaring, and carry assembly, and "Peak RSS" records peak resident memory. Single-threaded on one Zen 3 core, the C/GMP program multiplies two 10^8-bit integers in 92.5 s end-to-end excluding 5.46 s of table setup, or 98.0 s including setup. By comparison, GMP's `mpz_mul` takes 0.606 s on the same operands, making our program about 153 times slower.

Across the four sizes, the end-to-end slowdowns are 153× to 323×. Including table setup, the single-threaded slowdowns relative to `mpz_mul` are 245×, 162×, 240×, and 326× in `bench/`

BENCHMARK.md; recomputing the first ratio from the rounded timings in Table 4 gives 246×. The 10^9-bit run needs about 72 GB of RAM, with peak RSS 67.3 GiB.

In one 64-thread run at 10^8 bits with OMP_NUM_THREADS=64 on the same binary and operands, intmul_fastdft in mode algo=1 took 4.90 s in the FastDFT core and 6.31 s end-to-end with 4.4 GiB peak RSS. That time is 10.4× slower than single-threaded GMP mpz_mul at 0.6056 s. Relative to single-threaded algo=1 on the same machine and operands, the single-threaded plain-Gaussian variant with the multiplicative schedule, algo=0, took 1.87× and 1.77× as long at 10^7 and 10^8 bits. It also used 3.6× and 2.4× the memory at those two sizes.

At these bit lengths GMP's mpz_mul uses a Schönhage–Strassen FFT; the constant factors of spreading, twisting, and deconvolution dominate our runtime at every tested size, and no crossover is expected at any physically storable size. Accordingly, no practical speedup is claimed.

Measurement environment and provenance. All measurements were run on an AMD EPYC 7B13 Zen 3 virtual machine under KVM with 64 vCPUs and 117 GiB of RAM under Linux x86_64. The binary was compiled with gcc 15.2.0 using -O2 -march=native -fopenmp -DNDEBUG against GMP 6.3.0 and MPFR 4.2.2 [CGMP26]. During the single-threaded runs the host was lightly shared at load average 2–5 on 64 vCPUs, and "one core" means one vCPU of the KVM guest.

The measured binary was compiled from a copy of intmul_fastdft.c that differs from the shipped file only in a header comment and two preprocessor-guard blocks. Specifically, an unused include branch under a build-internal macro was removed from the shipped file, and a guard was added so that the shipped file also compiles without OpenMP. Under the stated flags neither difference changes the compiled code, and both SHA-256 digests are recorded in bench/MACHINE.txt.

Each benchmark run tested one random operand pair per size by invoking intmul_fastdft mul <n> 0 7 <algo> with seed 7. That command generates random n-bit operands with the top bit set and compares the product bit for bit against single-threaded GMP mpz_mul on the same operands in the same process. The program's built-in forced-deep self-test, which lowers the leaf cutoff to force multi-level recursion at 4,096 and 65,536 bits, reported SELFTEST: ALL PASSED, with every product EXACT for both kernels. Every benchmark run from 10^7 to 10^9 bits printed EXACT MATCH, confirming bit-exact agreement on every tested input.

10 Discussion

Within the two-dimensional algorithm of Theorem 1, every scalar and polynomial multiplication takes $O(n(\log\log n)^2)$ steps in total on a multitape Turing machine. This bound covers root pointwise squaring, overlap-add Kronecker spreading, pre- and post-twists, two-term and Nyquist column deconvolution, and leaf radix-2 FFT butterflies, while shared per-level table generation takes $o(n)$ steps. The only $\Theta(n\log n)$ term in the upper bound is $\mathcal{C}_{\mathrm{syn+route}}$ in (38). That term consists of the multiplication-free polynomial transform $G_{t_{1,\ell}}$ over $\mathbb{C}[y]/(y^{r_\ell}+1)$ together with the per-level matrix transposes and odd-stride tape routing.

Known $\Omega(n\log n)$ lower bounds for integer multiplication are conditional. For Boolean circuits they rest on the network-coding conjecture [AFKL19], and for multitape Turing machines they rest on the conjectured $\Omega(n^2\log n)$ complexity of $n\times n$ binary matrix transposition [HvdH25]. Whether the $\Theta(n\log n)$ polynomial-transform and tape-routing term in a fixed-dimension algorithm can be reduced unconditionally remains open. Letting the grid dimension grow with n yields the $o(n\log n)$ upper bound given in [OAI26], and the companion manuscript [CLW26] claims an $o(n\log n)$ bound along that growing-dimension route on a single power-of-two grid. No lower bound is claimed in this paper.

References

[AFKL19] Peyman Afshani, Casper Benjamin Freksen, Lior Kamma, and Kasper Green Larsen. Lower bounds for multiplication via network coding. In *Proceedings of the 46th International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 132 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 10:1–10:12, 2019.

[BMK19] Alexander H. Barnett, Jeremy Magland, and Ludvig af Klinteberg. A parallel nonuniform fast Fourier transform library based on an "exponential of semicircle" kernel. *SIAM Journal on Scientific Computing*, 41(5):C479–C504, 2019.

[CF94] Richard Crandall and Barry Fagin. Discrete weighted transforms and large-integer arithmetic. *Mathematics of Computation*, 62(205):305–324, 1994.

[CGMP26] Piaoyang Cui, Yiming Liu, and David P. Woodruff. intmul-cgmp: C/GMP implementation and benchmark of the two-dimensional fractional-shift synthetic-FFT $O(n \log n)$ integer multiplier. Software package (`intmul_fastdft.c`, benchmark runner, and raw measurement logs in `bench/BENCHMARK.md` and `bench/logs/`) implementing the multiplier of this paper; directory `companion/impl/` of the repository, content snapshot `8d300053117b` (first 12 hex digits of the SHA-256 of `MANIFEST.sha256`), 2026. Available at `https://github.com/lockqueue/intmul`.

[CLW26] Piaoyang Cui, Yiming Liu, and David P. Woodruff. A simpler algorithm for integer multiplication below $n \log n$, with a formally verified proof. Companion manuscript. Available at `https://github.com/lockqueue/intmul`, 2026.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[DR93] Alok Dutt and Vladimir Rokhlin. Fast Fourier transforms for nonequispaced data. *SIAM Journal on Scientific Computing*, 14(6):1368–1393, 1993.

[Für07] Martin Fürer. Faster integer multiplication. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing (STOC)*, pages 57–66. ACM, 2007.

[Für09] Martin Fürer. Faster integer multiplication. *SIAM Journal on Computing*, 39(3):979–1005, 2009.

[GL04] Leslie Greengard and June-Yub Lee. Accelerating the nonuniform fast Fourier transform. *SIAM Review*, 46(3):443–454, 2004.

[Har09] David Harvey. Faster polynomial multiplication via multipoint Kronecker substitution. *Journal of Symbolic Computation*, 44(10):1502–1510, 2009.

[HvdH21] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021.

[HvdH25] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. In *Proceedings of the 66th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1366–1381. IEEE Computer Society, 2025.

[HvdHL16] David Harvey, Joris van der Hoeven, and Grégoire Lecerf. Even faster integer multiplication. *Journal of Complexity*, 36:1–30, 2016.

[LeanMS26] Piaoyang Cui, Yiming Liu, and David P. Woodruff. Sinc-Gaussian fractional-shift integer multiplication in $O(n \log n)$: a Lean 4 formalization. Lean 4 formalization accompanying this paper (library `IntMul`, 27 files, 82,377 lines, Lean 4.33.1, Mathlib `0df444a3`; headline theorem `IntMul.tmIntMul_evalsToInTime_n_log_n`); directory `companion/lean/` of the repository, content snapshot `d7d142b0db9b`, 2026. Available at `https://github.com/lockqueue/intmul`.

[Mat20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[NQ79] Henri J. Nussbaumer and Philippe Quandalle. Fast computation of discrete Fourier transforms using polynomial transforms. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 27(2):169–181, 1979.

[Nus80] Henri J. Nussbaumer. Fast polynomial transform algorithms for digital convolution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(2):205–215, 1980.

[Nus82] Henri J. Nussbaumer. *Fast Fourier Transform and Convolution Algorithms*, volume 2 of *Springer Series in Information Sciences*. Springer-Verlag, Berlin, 2nd edition, 1982.

[OAI26] OpenAI. Integer multiplication below $n \log n$. OpenAI Math Release preprint, `https://github.com/openai/math`, September 2026. Preprint dated September 23, 2026 (`preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf`).

[PST01] Daniel Potts, Gabriele Steidl, and Manfred Tasche. Fast Fourier transforms for nonequispaced data: a tutorial. In John J. Benedetto and Paulo J. S. G. Ferreira, editors, *Modern Sampling Theory: Mathematics and Applications*, pages 247–270. Birkhäuser, Boston, 2001.

[Sch82] Arnold Schönhage. Asymptotically fast algorithms for the numerical multiplication and division of polynomials with complex coefficients. In Jacques Calmet, editor, *Computer Algebra (EUROCAM '82)*, volume 144 of *Lecture Notes in Computer Science*, pages 3–15, Berlin, 1982. Springer-Verlag.

[SS71] Arnold Schönhage and Volker Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, 7(3–4):281–292, 1971.

A Elementary functions, shared tables, and tape schedule

This appendix supplies the low-level numerical and tape-scheduling details of the multitape Turing machine specification. First, we prove Lemma 3.2 by giving explicit rational series and fixed-point error bounds for every elementary function used when precomputing the filter, twiddle, and column-solver tables. Second, we record the bit widths of all intermediate scratch registers, the contents of the shared table tape, and the breadth-first tape traversal across all recursion levels. Readers who accept Lemma 3.2 and standard breadth-first tree traversal on tapes may skip this appendix on a first reading.

A.1 Proof of Lemma 3.2

This subsection bounds the truncation and fixed-point rounding errors of the elementary-function series in Lemma 3.2 and combines them to establish the 1-ulp table-evaluation bound (7).

Proof idea. Every primitive is evaluated on a high-precision guard grid $\widetilde{\mathbb{R}}_S$ with $S = s + 15D + 64$ using $N_{\text{terms}} = 2S + 36D$ series terms, making both the truncation tail and the accumulated $\widetilde{\mathbb{R}}_S$ rounding error far smaller than 2^{-s-20}. Evaluating each composite table formula at $s = q + 10$ then leaves a pre-rounded error below 2^{-q-10}, so a single final rounding $\mathcal{Q}_q$ lands within 1 ulp of the exact value.

Proof of Lemma 3.2. We establish the bit-width, accuracy, and time bounds for primitives (i)–(v) in turn, and then verify the composite table-evaluation contract (7).

(i) Series for π and $\ln 2$. This block bounds the truncation tails, loop-termination rule, integer divisors, and $\widetilde{\mathbb{R}}_S$-summation error for the approximations $\tilde{\pi}$, $\tilde{\ell}_2$, and $\tilde{h}$. Recall $S = s + 15D + 64$ and $N_{\text{terms}} = 2S + 36D$ with $D = \max(s, K_0) \geq 8$. The denominator base power satisfies

$$5^{2N_{\text{terms}}+1} > 3^{2N_{\text{terms}}} = 9^{N_{\text{terms}}} > 8^{2S+36D} = 2^{6S+108D} \gg 20 \cdot 2^{S+16}.$$

Consequently, the combined alternating series tails for π from both $\arctan(1/5)$ and $\arctan(1/239)$ and the geometric series tail for $\ln 2$ satisfy, respectively,

$$\left|\pi - 4\sum_{k=0}^{N_{\text{terms}}-1}(-1)^k \frac{4\cdot 5^{-2k-1} - 239^{-2k-1}}{2k+1}\right| \leq 20(2N_{\text{terms}}+1)^{-1}5^{-(2N_{\text{terms}}+1)} < 2^{-S-16},$$

$$2\sum_{k=N_{\text{terms}}}^{\infty} \frac{1}{(2k+1)3^{2k+1}} \leq \frac{3}{4(2N_{\text{terms}}+1)}3^{-2N_{\text{terms}}} < 2^{-S-16}.$$

In each series loop for $b \in \{3, 5, 239\}$, the machine stops incrementing b^{2k+1} as soon as $b^{2k+1} > 32 \cdot 2^S$. Every subsequent term with $k < N_{\text{terms}}$ has exact magnitude below $\frac{16}{32\cdot 2^S} = \frac{1}{2}2^{-S}$ and therefore rounds via $\mathcal{Q}_S$ to 0 on $\widetilde{\mathbb{R}}_S$ identically. For all $D \geq 8$, every integer divisor formed before termination satisfies

$$(2k+1)b^{2k+1} \leq (2N_{\text{terms}}+1)\cdot 239^2 \cdot 32 \cdot 2^S < 2^{S+23+\lceil \log_2(169D)\rceil} \ll 2^{2S+4D+16}.$$

Summing the non-zero rounded terms on $\widetilde{\mathbb{R}}_S$ incurs rounding error at most $4N_{\text{terms}}2^{-S} < 2^{-s-D-32}$, defining the approximations and error bounds

$$\tilde{\pi},\, \tilde{\ell}_2 \in \widetilde{\mathbb{R}}_S, \qquad \tilde{h} := \mathcal{Q}_S(\tilde{\pi}/2),$$
$$|\tilde{\pi} - \pi|,\, |\tilde{\ell}_2 - \ln 2| \leq (4N_{\text{terms}}+1)2^{-S}, \qquad |\tilde{h} - \pi/2| \leq (2N_{\text{terms}}+1)2^{-S}.$$

(ii) Square root. This block defines the scaled integer radicand $N_{\tilde{x}}$ and its integer square root m and bounds the fixed-point and input-perturbation errors on $[2^{-D}, 2^{2D}]$. Since both x and $\tilde{x}$ lie in $[2^{-D}, 2^{2D}]$, define

$$N_{\tilde{x}} := 2^{2S}\tilde{x} \in \mathbb{Z}_{\geq 1}, \qquad m := \lfloor \sqrt{N_{\tilde{x}}} \rfloor.$$

The integer $N_{\tilde{x}}$ has bit-length at most $2S + 2D + 1 = O(D)$. Computing m via digit-by-digit binary integer square root takes $O(D^2)$ steps and satisfies $|2^{-S}m - \sqrt{\tilde{x}}| < 2^{-S} < 2^{-s-D-32}$. On $[2^{-D}, 2^{2D}]$, the input perturbation satisfies

$$|\sqrt{\tilde{x}} - \sqrt{x}| \leq \frac{1}{2\sqrt{2^{-D}}}\delta_{\text{in}} = 2^{D/2-1}\delta_{\text{in}}.$$

(iii) Exponential. This block verifies the underflow cutoff, defines the quotient-aligned range-reduction variables $m, \tilde{r}, r_m$ and Taylor terms $\tilde{T}_k$, and bounds exp on $[-\pi(D+2)^2, 0]$. Whenever the exact $\widetilde{\mathbb{R}}_S$ cutoff $\tilde{x} < -(S+16)\tilde{\ell}_2$ holds, the bound $|\tilde{\ell}_2 - \ln 2| \le (4N_{\text{terms}}+1)2^{-S}$ implies $\tilde{x} < -(S+16)\ln 2 + 2^{-100}$ and $e^{\tilde{x}} < 2^{-S-15} < 2^{-s-D-32}$, so returning 0 is valid.

Otherwise $-(S+16)\tilde{\ell}_2 \le \tilde{x} \le 0$, and integer division of $2^S(-\tilde{x})$ by $2^S\tilde{\ell}_2$ defines

$$\begin{aligned} m &:= \lfloor (-\tilde{x})/\tilde{\ell}_2 \rfloor \in \{0, \dots, S+16\}, \\ \tilde{r} &:= -\tilde{x} - m\tilde{\ell}_2 \in \widetilde{\mathbb{R}}_S \cap [0, \tilde{\ell}_2), \qquad r_m := -\tilde{x} - m\ln 2. \end{aligned}$$

Because r_m is defined with that same integer m, we have $e^{\tilde{x}} = 2^{-m}e^{-r_m}$ identically, and comparing $\tilde{r}$ to r_m incurs no quotient-boundary jump. Their difference satisfies

$$|\tilde{r} - r_m| = m|\tilde{\ell}_2 - \ln 2| \le (S+16)(4N_{\text{terms}}+1)2^{-S} < 2^{-100},$$

with $r_m \in [-2^{-100}, \ln 2 + 2^{-100}]$ and $|(e^{-t})'| \le e^{2^{-100}} < 1 + 2^{-99}$ between $\tilde{r}$ and r_m.

The N_{terms} Taylor terms for $\exp(-\tilde{r})$ are generated on $\widetilde{\mathbb{R}}_S$ by the recurrence

$$\tilde{T}_0 := 1, \qquad \tilde{T}_k := \mathrm{RD}_S\big(\mathrm{RM}_S(\tilde{T}_{k-1}, -\tilde{r}),\, k\big) \quad (1 \le k < N_{\text{terms}}).$$

Since $\tilde{r}/k < 0.70$, each term satisfies $|\tilde{T}_k - T_k| < 3.34 \cdot 2^{-S}$, and every intermediate integer stays within $2S + 4D + 16$ bits. Moreover, the truncation tail satisfies

$$\frac{|\tilde{r}|^{N_{\text{terms}}}}{N_{\text{terms}}!} \le \left(\frac{e}{N_{\text{terms}}}\right)^{N_{\text{terms}}} < 2^{-4S} < 2^{-S-16}.$$

After summing the N_{terms} terms on $\widetilde{\mathbb{R}}_S$, the machine applies a rounded m-bit right shift $\mathcal{Q}_S(2^{-m}\cdot)$ onto $\widetilde{\mathbb{R}}_S$ for 2^{-m}, adding at most $\frac{1}{2}2^{-S}$. For all $D \ge 8$, the total error relative to $e^{\tilde{x}}$ is at most

$$((S+18)(4N_{\text{terms}}+1)+1)2^{-S} < 2^{14}D^2 2^{-S} < 2^{-s-D-32}.$$

(iv) Sine and cosine. This block defines the $\pi/2$ range-reduction quantities $m, \tilde{r}, r_m$ and recurrence terms $\tilde{T}_k$ on $|x| \le \pi(D+2)$, followed by the direct sinc series on $[-1/2, 1/2]$. Using $\tilde{h} = \mathcal{Q}_S(\tilde{\pi}/2)$ with $|\tilde{h} - \pi/2| \le (2N_{\text{terms}}+1)2^{-S}$, define

$$m := \lfloor \tilde{x}/\tilde{h} + 1/2 \rfloor \in \mathbb{Z}, \qquad \tilde{r} := \tilde{x} - m\tilde{h} \in \widetilde{\mathbb{R}}_S, \qquad r_m := \tilde{x} - m(\pi/2).$$

Here $|m| \le 2(D+2)+1 < S+16$, and the reduced remainder satisfies

$$\begin{aligned} |\tilde{r}| &\le \frac{\tilde{h}}{2} < 0.79 < 1, \\ |\tilde{r} - r_m| &= |m|\,|\tilde{h} - \pi/2| \le (S+16)(4N_{\text{terms}}+1)2^{-S} < 2^{-100}. \end{aligned}$$

Using that same integer m in r_m avoids any quotient-boundary jump and gives $\sin\tilde{x} = \sin(m\frac{\pi}{2} + r_m)$ and $\cos\tilde{x} = \cos(m\frac{\pi}{2} + r_m)$ identically via $m \bmod 4$, while $|(\sin t)'|, |(\cos t)'| \le 1$.

For $\cos\tilde{r}$ and $\sin\tilde{r}$, the N_{terms} Taylor terms are generated on $\widetilde{\mathbb{R}}_S$ by

$$\begin{aligned} &\tilde{T}_0 := 1 \quad (\text{for } \cos\tilde{r},\ \text{divisor } 2k(2k-1)), \qquad \tilde{T}_0 := \tilde{r} \quad (\text{for } \sin\tilde{r},\ \text{divisor } 2k(2k+1)), \\ &\tilde{T}_k := \mathrm{RD}_S\Big(\mathrm{RM}_S(\tilde{T}_{k-1}, -\mathrm{RM}_S(\tilde{r}, \tilde{r})),\, 2k(2k \mp 1)\Big) \quad (1 \le k < N_{\text{terms}}). \end{aligned}$$

Because $\tilde{r}^2/2 < 0.32$, each term satisfies $|\tilde{T}_k - T_k| < 1.48 \cdot 2^{-S}$, and all intermediate integers stay within $2S + 4D + 16$ bits. For all $D \geq 8$, the alternating tail bound of Lemma 3.2(iv) and the total error $|\tilde{y} - f(\tilde{x})|$ for $f \in \{\sin, \cos\}$ satisfy

$$\frac{|\tilde{r}|^{2N_{\text{terms}}}}{(2N_{\text{terms}})!} \leq \frac{|\tilde{r}|^{N_{\text{terms}}}}{N_{\text{terms}}!} < 2^{-S-16},$$
$$|\tilde{y} - f(\tilde{x})| \leq ((S+18)(4N_{\text{terms}}+1)+1)2^{-S} < 2^{14}D^2 2^{-S} < 2^{-s-D-32}.$$

Claim: Direct sinc **series on** $[-1/2, 1/2]$**.** For $|t| \leq 1/2$, define the scaled argument $\tilde{u}$ and Taylor terms $\tilde{T}_k$ on $\widetilde{\mathbb{R}}_S$ by

$$\tilde{u} := \mathrm{RM}_S(\tilde{\pi}, \tilde{t}), \qquad \tilde{T}_0 := 1,$$
$$\tilde{T}_k := \mathrm{RD}_S\Big(\mathrm{RM}_S(\tilde{T}_{k-1}, -\mathrm{RM}_S(\tilde{u}, \tilde{u})),\, 2k(2k+1)\Big) \quad (1 \leq k < N_{\text{terms}}).$$

Then $|\tilde{u} - \pi\tilde{t}| \leq (2N_{\text{terms}}+1)2^{-S}$ and $|\tilde{u}| < 1.58$. Every step ratio satisfies $\tilde{u}^2/(2k(2k+1)) < 0.42 < 1/2$, and every term satisfies $|\tilde{T}_k| \leq 1$. The alternating tail bound of Lemma 3.2(iv) and the derivative on $|u| \leq \pi/2$ satisfy

$$\frac{(\pi/2)^{2N_{\text{terms}}}}{(2N_{\text{terms}}+1)!} \leq \left(\frac{\pi e}{4N_{\text{terms}}}\right)^{2N_{\text{terms}}} < 2^{-4S} < 2^{-S-16},$$
$$\left|\frac{d}{du}\left(\frac{\sin u}{u}\right)\right| \leq \frac{4}{\pi^2} < 0.41 \leq 1 \quad (|u| \leq \pi/2),$$

with $|\operatorname{sinc}'(t)| \leq \frac{4}{\pi} < 1.28$ on $[-1/2, 1/2]$. Summing $\tilde{T}_0, \ldots, \tilde{T}_{N_{\text{terms}}-1}$ on $\widetilde{\mathbb{R}}_S$ therefore gives

$$|\tilde{y} - \operatorname{sinc}(\tilde{t})| \leq (6N_{\text{terms}}+2)2^{-S} + 2^{-S-16} < 2^{-s-D-32}.$$

(v) Error function erf**.** This block analyzes the early-exit cutoff, the power-series recurrence $\tilde{T}_n$ with auxiliary quantities $\Sigma(\tilde{z}), w_{\text{sq}}, V, T_n, \delta_k$, and the Gaussian prefactor multiplication on $[0, 3\sqrt{D}]$.

Claim: Early-exit branch of erf**.** Comparing $\mathrm{RM}_S(\tilde{z}, \tilde{z})$ against $(s+D+24)\tilde{\ell}_2$ on $\widetilde{\mathbb{R}}_S$ tests the cutoff within $O(D^2 2^{-S})$. Whenever $\mathrm{RM}_S(\tilde{z}, \tilde{z}) > (s+D+24)\tilde{\ell}_2$, we have $\tilde{z}^2 > (s+D+24)\ln 2 - 2^{-100}$, and $s + D + 24 \geq 40$ gives $\tilde{z} > 5.26$. Hence returning 1 incurs error

$$\operatorname{erfc}(\tilde{z}) \leq \frac{e^{-\tilde{z}^2}}{\sqrt{\pi}\tilde{z}} < 2^{-s-D-26} < 2^{-s-D-23}.$$

Claim: Series branch and recurrence rounding for erf**.** On the series branch, $\tilde{z} \in \widetilde{\mathbb{R}}_S \cap [0, 3\sqrt{D}]$ satisfies $\tilde{z}^2 \leq (s+D+24)\ln 2 + 2^{-100}$, so $\tilde{z}^2 \leq 9D$ and $e^{\tilde{z}^2} < 1.01 \cdot 2^{s+D+24} \leq 1.01 \cdot 2^{2D+24}$. Define the positive series

$$\Sigma(\tilde{z}) := \tilde{z} \sum_{n=0}^{\infty} \frac{(2\tilde{z}^2)^n}{(2n+1)!!} = \frac{\sqrt{\pi}}{2} e^{\tilde{z}^2} \operatorname{erf}(\tilde{z}).$$

Since $e^9 < 8104 < 2^{13}$, $\Sigma(\tilde{z})$ satisfies $\Sigma(\tilde{z}) < 2^{s+D+24} \leq 2^{2D+24}$ and $\Sigma(\tilde{z}) \leq 3\sqrt{D}\, e^{9D} < 2^{14D}$. Using $(2n+1)!! \geq 2^n n! \geq 2^n (n/e)^n$, for every index n in the tail we have

$$n \geq N_{\text{terms}} = 2S + 36D > 66D, \qquad \frac{\tilde{z}^2 e}{n} < \frac{9eD}{66D} < 0.371 < 2^{-1.43},$$

so the series tail for $n \geq N_{\text{terms}}$ is bounded by $6\sqrt{D} \cdot 2^{-1.43 N_{\text{terms}}} < 2^{-S-16}$.

To bound the fixed-point rounding error on $\widetilde{\mathbb{R}}_S$, define

$$w_{\mathrm{sq}} := \mathrm{RM}_S(2\tilde{z}, \tilde{z}), \qquad V := \max(2\tilde{z}^2, w_{\mathrm{sq}}) \le 2\tilde{z}^2 + 1,$$

so $|w_{\mathrm{sq}} - 2\tilde{z}^2| \le \frac{1}{2}2^{-S}$, and compute the recurrence

$$\tilde{T}_0 := \tilde{z}, \qquad \tilde{T}_n := \mathrm{RD}_S(\mathrm{RM}_S(\tilde{T}_{n-1}, w_{\mathrm{sq}}),\, 2n+1) \quad (1 \le n < N_{\mathrm{terms}}).$$

Define the exact n-th term T_n and the step residual δ_k by

$$\begin{aligned} T_n &:= \frac{(2\tilde{z}^2)^n \tilde{z}}{(2n+1)!!}, \qquad |\delta_0| = |\tilde{T}_0 - T_0| = 0, \\ \delta_k &:= (\tilde{T}_k - T_k) - \frac{w_{\mathrm{sq}}}{2k+1}(\tilde{T}_{k-1} - T_{k-1}) \\ &= \left(\tilde{T}_k - \frac{w_{\mathrm{sq}}}{2k+1}\tilde{T}_{k-1}\right) + \frac{w_{\mathrm{sq}} - 2\tilde{z}^2}{2k+1} T_{k-1} \quad (k \ge 1). \end{aligned}$$

Each residual satisfies

$$|\delta_k| \le \frac{T_{k-1}}{2k+1}|w_{\mathrm{sq}} - 2\tilde{z}^2| + \left(\frac{1}{2(2k+1)} + \frac{1}{2}\right)2^{-S} < \frac{T_{k-1}}{2(2k+1)}2^{-S} + 2^{-S}.$$

Unrolling the linear recurrence gives $\tilde{T}_n - T_n = \sum_{k=1}^{n} \delta_k \prod_{j=k+1}^{n} \frac{w_{\mathrm{sq}}}{2j+1}$. Using $(2k-1)!!(2k+1)\prod_{j=k+1}^{n}(2j+1) = (2n+1)!! \ge 3\cdot 2^{n-1}(n-1)!$ and $\prod_{j=k+1}^{n}(2j+1) \ge 2^{n-k}(n-k)!$, the propagation products satisfy

$$\begin{aligned} \frac{T_{k-1}}{2k+1}\prod_{j=k+1}^{n}\frac{w_{\mathrm{sq}}}{2j+1} &\le \frac{\tilde{z}V^{n-1}}{(2n+1)!!} \le \sqrt{D}\, e^{V/2}, \\ \prod_{j=k+1}^{n}\frac{w_{\mathrm{sq}}}{2j+1} &\le \frac{(V/2)^{n-k}}{(n-k)!} \le e^{V/2} \le e^{9D+1/2} = \sqrt{e}\,(e^9)^D < 2\cdot(2^{13})^D = 2^{13D+1}. \end{aligned}$$

Thus each step $k \le n$ contributes at most $(\sqrt{D}+1)e^{V/2}2^{-S} \le 8D\cdot 2^{13D+1-S}$ to $|\tilde{T}_n - T_n|$. For all $D = \max(s, K_0) \ge 8$, we have $S = s + 15D + 64 \le 24D$, $D^3 2^{-D} \le 8^3 2^{-8} = 2$, and

$$N_{\mathrm{terms}} = 2S + 36D \le 84D < 2^7 D, \qquad \frac{N_{\mathrm{terms}}(N_{\mathrm{terms}}+1)}{2} < 2^{12}D^2.$$

Summing over all pairs (k, n) with $1 \le k \le n$ and $n < N_{\mathrm{terms}}$ and adding the truncation tail $< 2^{-S-16}$ yields

$$\begin{aligned} |\widetilde{\Sigma}(\tilde{z}) - \Sigma(\tilde{z})| &\le \frac{N_{\mathrm{terms}}(N_{\mathrm{terms}}+1)}{2}\cdot 8D\cdot 2^{13D+1-S} + 2^{-S-16} \\ &< 2^{16}D^3 2^{-2D}\cdot 2^{-s-64} + 2^{-S-16} \le 2^{-s-D-47} + 2^{-S-16} < 2^{-s-D-24}. \end{aligned}$$

Claim: Prefactor multiplication and final erf **error.** For $\tilde{z} \in [0, 3\sqrt{D}]$, forming $-\mathrm{RM}_S(\tilde{z}, \tilde{z})$ perturbs $-\tilde{z}^2$ by at most $\frac{1}{2}2^{-S}$, so by (i)–(iii) the prefactor $\frac{2}{\sqrt{\pi}}e^{-\tilde{z}^2} \in (0, \frac{2}{\sqrt{\pi}}] \subset (0, 1.13]$ is computed on $\widetilde{\mathbb{R}}_S$ with error $< 2^{15}D^2 2^{-S}$. Because $|\widetilde{\Sigma}(\tilde{z})| < 2^{2D+25}$, the scaled integer product between this prefactor and $\widetilde{\Sigma}(\tilde{z})$ has bit-length at most $2S + 2D + 27 < 2S + 4D + 16$. For all $D \ge 8$, multiplying by $\widetilde{\Sigma}(\tilde{z})$ amplifies the prefactor error by at most

$$2^{2D+25}\cdot 2^{15}D^2 2^{-S} = D^2 2^{-12D-24}\cdot 2^{-s-D} < 2^{-s-D-110}.$$

Rounding the product onto $\widetilde{\mathbb{R}}_S$ adds at most $\frac{1}{2}2^{-S}$, which gives

$$|\tilde{y} - \mathrm{erf}(\tilde{z})| \le 1.13\cdot 2^{-s-D-24} + 2^{-s-D-110} + \frac{1}{2}2^{-S} < 2^{-s-D-23} < 2^{-s-30}$$

together with the Lipschitz bound $|\mathrm{erf}(\tilde{z}) - \mathrm{erf}(z)| \le \frac{2}{\sqrt{\pi}}|\tilde{z} - z|$.

Composite table expressions and faithful rounding. This block verifies domain membership, derivative amplification, repeated-squaring stability for $\theta(u_2)$, the 1-ulp rounding bound (7), and the $O(D^3)$ step count across all table expressions. The real constants and composite table expressions evaluated in our algorithm are as follows.

- First are the width parameter $\alpha_\ell = 2\sqrt{p_{\mathrm{work},\ell} \ln 2/\pi}$, the window-radius expression $\alpha_\ell \sqrt{p_{\ell+1} \ln 2/\pi}$ in Appendix A.2(a), and the filter samples $g(t)$ at exact dyadic rational arguments $t = d + j_1/t_{1,\ell} \in \widetilde{\mathbb{R}}_{K_0} \subseteq \widetilde{\mathbb{R}}_S$ with $|d| \le m_{1,\ell} \le D$. Here $g(t)$ uses the direct $\mathrm{sinc}(t)$ series on $|t| \le 1/2$ and divides by πt on $|t| > 1/2$, where $|\pi t| > \pi/2 > 1.57$.
- Second are the twist roots $e^{\pi i k_2/r_\ell}$ and leaf butterfly roots $e^{2\pi i j \cdot 2^{K_L - s}/R_L}$.
- Third are the passband coefficients $c_0(u_2), c_1(u_2) \in [0,1]$, the Nyquist reciprocal $c(r_\ell/2)^{-1}$, and for $u_2 \ne r_\ell/2$ the non-Nyquist column solver scalars $M(u_2)^{-1}$, $\rho(u_2)$, and $\theta(u_2) = (1 - \rho(u_2)^{t_{1,\ell}})^{-1}$, where $M(u_2) > 0.499$ and $1 - \rho(u_2)^{t_{1,\ell}} > 0.86$ for all $u_2 \ne r_\ell/2$.

Each expression composes $O(1)$ evaluations of (i)–(v) on $\widetilde{\mathbb{R}}_S$ with all divisions bounded away from zero by at least 0.42, plus at most $\log_2 t_{1,\ell} \le D/2$ squarings on $\widetilde{\mathbb{R}}_S$.

Claim: Call-site domain membership. At every call site in the algorithm, both the exact argument and the computed $\widetilde{\mathbb{R}}_S$-input lie inside the stated domain of Lemma 3.2. Every $\sqrt{x}$ evaluation, namely $\sqrt{\pi}$, $\alpha_\ell = 2\sqrt{p_{\mathrm{work},\ell} \ln 2/\pi}$, and $\sqrt{p_{\ell+1} \ln 2/\pi}$, satisfies

$$8 \le p_{\mathrm{work},\ell} \le p_{\ell+1} < s \le D, \qquad x, \tilde{x} \in [1, 2^{2D}].$$

On $[1, 2^{2D}]$, $\sqrt{\cdot}$ has local Lipschitz constant at most $\frac{1}{2}$ rather than $2^{D/2-1}$, and its inputs from (i) have $\widetilde{\mathbb{R}}_S$-error $O(D2^{-S})$. Likewise, every negated-square exp evaluation has $x, \tilde{x} \in [-\pi(D+2)^2, 0]$, every $\sin, \cos$ evaluation has $|x|, |\tilde{x}| \le \pi(D+2)$, and every erf evaluation has $z, \tilde{z} \in [0, 3\sqrt{D}]$ by using $\mathrm{erf}(-z) = -\mathrm{erf}(z)$ on $y - 1/2 \le 0$ for $0 \le u_2 \le r_\ell/2$.

Claim: Non-θ composite error bound. From (i) and (ii) we have $|\tilde{\pi} - \pi|, |\tilde{\ell}_2 - \ln 2|, |\tilde{\alpha}_\ell - \alpha_\ell| \le O(D^2 2^{-S}) < 2^{-s-D-100}$. Across all non-$\theta$ composite expressions, the partial derivatives along the compositions satisfy the following five bounds.

- For $(\alpha_\ell, \pi) \mapsto -\pi t^2/\alpha_\ell^2 \le 0$, the bound is $\frac{2\pi(D+1)^2}{\alpha_\ell^3} + \frac{(D+1)^2}{\alpha_\ell^2} < \frac{1}{2}(D+1)^2$, since $|t| < D+1$ and $\alpha_\ell > 2.65$.
- For the product $\alpha_\ell \cdot \sqrt{p_{\ell+1} \ln 2/\pi}$ of two factors in $[1, 2\sqrt{D}]$, the bound is $2\sqrt{D} < \frac{1}{2}(D+1)^2$.
- For $\alpha_\ell \mapsto \mathrm{erf}(\sqrt{\pi}\alpha_\ell(y \pm \frac{1}{2}))$, the bound is $2|y \pm \frac{1}{2}| \le 3$.
- For each reciprocal of a quantity ≥ 0.42, the bound is $1/0.42^2 < 5.7$, and for $(c_{\min}, M) \mapsto -c_{\min}/M$ with $0 \le c_{\min} \le M$ and $M(u_2) > 0.499$, it is $2/M(u_2) < 4.1 < 5.7$.
- For exp on $(-\infty, 0]$, sin, cos, and each factor of a product on $[-1,1]^2$, the bound is at most 1, while exact $t \in \widetilde{\mathbb{R}}_S$ gives $|\widetilde{\mathrm{sinc}}(t) - \mathrm{sinc}(t)| < 2^{-s-D-23}$ directly.

With primitive errors $< 2^{-s-D-23}$ from (i)–(v) and $\widetilde{\mathbb{R}}_S$ arithmetic rounding $O(D^2 2^{-S}) \ll 2^{-s-D-100}$, every non-$\theta$ table expression has error at most

$$5.7\big(\tfrac{1}{2}(D+1)^2 + 4\big) \cdot 2^{-s-D-23} < 2^{D+3} \cdot 2^{-s-D-23} = 2^{-s-20} \quad (D \ge 8).$$

At $D = 8$, this inequality holds because

$$5.7 \times 44.5 = 253.65 < 2048 = 2^{11}.$$

In particular, the passband coefficients and recurrence ratio satisfy

$$|\tilde{c}_0(u_2) - c_0(u_2)|,\ |\tilde{c}_1(u_2) - c_1(u_2)| < 2 \cdot 2^{-s-D-23} = 2^{-s-D-22},$$
$$|\tilde{\rho}(u_2) - \rho(u_2)| < 12 \cdot 2^{-s-D-23} < 2^{-s-D-19}.$$

Claim: Repeated squaring for $\theta(u_2)$ and 1-ulp rounding. For $\theta(u_2) = (1-\rho(u_2)^{t_{1,\ell}})^{-1}$ at $u_2 \neq r_\ell/2$, we have $M(u_2) > 0.499$, $t_{1,\ell} \le 2^{D/2}$, and

$$|c_0(u_2) - c_1(u_2)| > \frac{0.424}{t_{1,\ell}} \ge 0.424 \cdot 2^{-D/2} \gg 2^{-s-D-20}.$$

Together with $|\tilde{c}_0(u_2) - c_0(u_2)|, |\tilde{c}_1(u_2) - c_1(u_2)| < 2^{-s-D-22}$, this ensures $|\tilde{\rho}(u_2) - \rho(u_2)| < 2^{-s-D-19}$ and $|\tilde{\rho}(u_2)| < 1$. Forming $\widetilde{(\rho^{t_{1,\ell}})}$ by $\log_2 t_{1,\ell} \le D/2$ repeated squarings on $\widetilde{\mathbb{R}}_S$ therefore satisfies

$$\begin{aligned}\left|\widetilde{(\rho^{t_{1,\ell}})} - \rho(u_2)^{t_{1,\ell}}\right| &\le t_{1,\ell} 2^{-S} + t_{1,\ell} \max\left(|\tilde{\rho}(u_2)|, |\rho(u_2)|\right)^{t_{1,\ell}-1} |\tilde{\rho}(u_2) - \rho(u_2)| \\ &\le 2^{D/2-S} + 2^{D/2} \cdot 2^{-s-D-19} < 2^{-s-22},\end{aligned}$$

and dividing $(1 - \widetilde{(\rho^{t_{1,\ell}})})^{-1}$ yields $|\tilde{\theta}(u_2) - \theta(u_2)| < 2^{-s-20}$ unconditionally for all $s \ge 8$ and $D = \max(s, K_0)$.

At $s = q + 10$, every pre-rounded table value $\tilde{v} \in \widetilde{\mathbb{R}}_S$ satisfies $|\tilde{v} - v| < 2^{-s-20} < 2^{-q-10}$, so rounding $\bar{v} := \mathcal{Q}_q(\tilde{v})$ gives

$$|\bar{v} - v| \le |\mathcal{Q}_q(\tilde{v}) - \tilde{v}| + |\tilde{v} - v| \le \left(\frac{1}{2} + 2^{-10}\right) 2^{-q} < 2^{-q},$$

proving (7). The reused error-budget thresholds across (i)–(v) and the composite expressions are summarized below:

Error component	Bound	Where used
Series truncation tail (N_{terms} terms)	$< 2^{-S-16}$	π, $\ln 2$, exp, sin / cos, sinc, erf
Perturbation induced by $\tilde{\ell}_2, \tilde{h}$ (range reduction $\lvert\tilde{r} - r_m\rvert$, cutoff slack)	$< 2^{-100}$	exp, sin / cos, erf cutoff
Primitive $\widetilde{\mathbb{R}}_S$-evaluation error $\lvert\tilde{y} - f(\tilde{x})\rvert$	$< 2^{-s-D-32}$ (and $< 2^{-s-D-23}$)	(i)–(iv) (and (v) erf)
Pre-rounded composite table error $\lvert\tilde{v} - v\rvert$	$< 2^{-s-20} < 2^{-q-10}$ ($s = q + 10$)	all level and leaf tables
Final $\mathcal{Q}_q$ table entry error $\lvert\bar{v} - v\rvert$	$< 2^{-q}$ (real), $< \sqrt{2} \cdot 2^{-q}$ (complex)	1-ulp contract (7)

Finally, each primitive evaluates $N_{\text{terms}} = \Theta(D)$ terms using $O(1)$ schoolbook multiplications and divisions of $O(S) = O(D)$-bit integers per term, taking $O(D^3)$ bit operations. □

A.2 Register width inventory and breadth-first tape schedule

This subsection records the bit widths of all intermediate scratch registers, describes the layout of the shared per-level table tape, and specifies the breadth-first tape traversal across all recursion levels.

Scratch and accumulator register widths. Complementing the uniform record width $w_\ell = 2w_{\text{coord},\ell}$ with $w_{\text{coord},\ell} = q_{\text{child}} + H_{\ell+1} + 3 = \Theta(\log n)$ in (6), all intermediate scratch and accumulator registers across the algorithm have explicitly bounded widths:

- *Unrounded products and shifted dividends in* $\mathrm{RM}_q, \mathrm{RD}_q$*:* In RM_q the machine multiplies two $w_{\text{coord},\ell}$-bit signed integers before rounding from fractional precision $2q$ to q, and in RD_q it shifts a stored numerator $N_z = 2^q z$ by 2^q. These operations use scratch product and dividend registers of respective bit widths

$$2w_{\text{coord},\ell} + 2 = \Theta(\log n), \qquad w_{\text{coord},\ell} + q + 2 = \Theta(\log n).$$

- *Nyquist spatial alternating sum $A_{\mathrm{N}}^{(\ell,i)}$ and `NyquistSolve` scratch sum $\tilde{S}$:* Accumulating the unscaled alternating sum $\sum_{j_2=0}^{r_\ell-1}(-1)^{j_2}x^{(\ell,i)}(t_{1,\ell}/2+t_{1,\ell}j_2)$ uses a scratch accumulator before the exact $k_{2,\ell}$-bit shift r_ℓ^{-1} places $A_{\mathrm{N}}^{(\ell,i)}$ into a standard w_ℓ-bit record on T_{stack}. Similarly, accumulating $\tilde{S}=\sum_{i=0}^{t_{1,\ell}-1}(-1)^i\tilde{P}_i$ inside `NyquistSolve` before dividing by $t_{1,\ell}=2^{k_{1,\ell}}$ and rounding to $\widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ uses a second scratch accumulator, with respective bit widths

$$w_{\mathrm{coord},\ell}+k_{2,\ell}+1=\Theta(\log n),\qquad w_{\mathrm{coord},\ell}+k_{1,\ell}+1=\Theta(\log n).$$

- *Step 1 Kronecker integers and overlap-add accumulators:* Each packed block or filter integer in Theorem 6.3 has at most $\tilde{W}_\ell B_{\mathrm{slot}}+1$ bits. Their integer product and each unpacked overlap-add slot accumulator at fractional precision $2q_{\mathrm{child}}$ have respective bit widths at most

$$2\tilde{W}_\ell B_{\mathrm{slot}}+2=\Theta(\tilde{W}_\ell p_{\ell+1}),\qquad B_{\mathrm{slot}}+1=\Theta(p_{\ell+1}).$$

- *Leaf `BaseRadix2FFT` records and root carry registers:* At depth L with $q_b:=p_L+K_L+4$, leaf records on $\widetilde{\mathbb{C}}_{q_b}$ and signed integer registers for root chunk extraction and radix-2^b carry propagation have respective bit widths

$$w_{\mathrm{leaf}}:=2(q_b+H_L+3)=\Theta(\log n),\qquad 2b+K_0+3=\Theta(\log n).$$

Contents of the shared per-level table tape T_{tab}. Before starting the recursive transforms, the outer algorithm writes once per level $\ell\in\{0,\dots,L\}$ a sequential level-table segment onto T_{tab} containing four components.

(a) For $\ell<L$, the machine first computes the Lemma 3.2 approximation $\tilde{x}\in\widetilde{\mathbb{R}}_S$ of $\alpha_\ell\sqrt{p_{\ell+1}\ln 2/\pi}$ at $s=q_{\mathrm{child}}+10$ and sets the integer window radius

$$m_{1,\ell}:=\lceil\tilde{x}+2^{-s-20}\rceil,\qquad \tilde{W}_\ell:=\min(2m_{1,\ell}+1,r_\ell).$$

Since $|\tilde{x}-\alpha_\ell\sqrt{p_{\ell+1}\ln 2/\pi}|<2^{-s-20}$, this value equals $\lceil\alpha_\ell\sqrt{p_{\ell+1}\ln 2/\pi}\rceil$ or that integer plus one. It lies in the admissible interval of Lemma 6.5 because

$$\alpha_\ell\sqrt{p_{\ell+1}\ln 2/\pi}\le\frac{2\ln 2}{\pi}p_{\ell+1}<0.45\,p_{\ell+1}.$$

The machine then writes the $t_{1,\ell}$ guarded modulo-r_ℓ folded filter tables $\bar{H}^{(j_1)}\in\widetilde{\mathbb{R}}_{q_{\mathrm{child}}}^{\tilde{W}_\ell}$ of length $\tilde{W}_\ell$ and their Kronecker-packed integers $\mathrm{Pack}_{B_{\mathrm{slot}}}(2^{q_{\mathrm{child}}}\bar{H}^{(j_1)})$ of $O(\tilde{W}_\ell p_{\ell+1})$ bits each.

(b) For $\ell<L$, it writes the r_ℓ 1-ulp faithful twist roots $\bar{\zeta}_{k_2}\in\widetilde{\mathbb{C}}_{q_{\mathrm{child}}}$ approximating $e^{\pi ik_2/r_\ell}$ in natural order $k_2=0,\dots,r_\ell-1$. Each root is evaluated via Lemma 3.2 at $s=q_{\mathrm{child}}+10$ and rounded via $\mathcal{Q}_{q_{\mathrm{child}}}$.

(c) For $\ell<L$, it writes the r_ℓ 1-ulp faithful column solver tuples $(\mu(u_2),\hat{\rho}(u_2),\hat{\theta}(u_2))\in\widetilde{\mathbb{R}}_{q_{\mathrm{child}}}^3$ in natural order $u_2=0,\dots,r_\ell-1$. For $0\le u_2<r_\ell/2$, we have $c_0(u_2)>c_1(u_2)$, $M(u_2)=c_0(u_2)>0.49999999$, $\rho(u_2)=-c_1(u_2)/c_0(u_2)\in(-1,0)$, and $1-\rho(u_2)^{t_{1,\ell}}>0.86$. On this half, the pre-rounded approximations $\tilde{c}_0(u_2),\tilde{c}_1(u_2)\in\widetilde{\mathbb{R}}_S$ are evaluated via Lemma 3.2 at $s=q_{\mathrm{child}}+10$ to form $(\mu(u_2),\hat{\rho}(u_2),\hat{\theta}(u_2))$ via $\mathcal{Q}_{q_{\mathrm{child}}}$. These three rounded scalars approximate $(M(u_2)^{-1},\rho(u_2),\theta(u_2))$ within $\frac{1}{4}u$ each, and only those three rounded scalars are written to T_{tab}. For $r_\ell/2<u_2<r_\ell$, the tuple is set by reflection symmetry to

$$(\mu(u_2),\,\hat{\rho}(u_2),\,\hat{\theta}(u_2)):=(\mu(r_\ell-u_2),\,\hat{\rho}(r_\ell-u_2),\,\hat{\theta}(r_\ell-u_2)).$$

At the Nyquist column $u_2=r_\ell/2$, `NyquistSolve` reads only the Nyquist reciprocal $\mu_{\mathrm{N}}:=\mu(r_\ell/2)\approx c^{-1}$, evaluated via Lemma 3.2 at $s=q_{\mathrm{child}}+10$ and rounded via $\mathcal{Q}_{q_{\mathrm{child}}}$, with placeholder values $(\hat{\rho}(r_\ell/2),\hat{\theta}(r_\ell/2)):=(-1,0)$.

(d) At the leaf level $\ell = L$ with $R_L = 2^{K_L} \le 4p_L^2$, for each butterfly stage $s \in \{1, \dots, K_L\}$ of `BaseRadix2FFT`, it writes the contiguous stage-s 1-ulp faithful twiddle list $(\bar{\omega}_{s,j})_{j=0}^{2^{s-1}-1} \in \widetilde{\mathbb{C}}_{q_b}^{2^{s-1}}$ approximating $e^{2\pi i j \cdot 2^{K_L - s}/R_L}$ at precision $q_b := p_L + K_L + 4$. Each twiddle entry is evaluated via Lemma 3.2 at target precision $q_b + 10$ followed by $\mathcal{Q}_{q_b}$, streaming

$$\sum_{s=1}^{K_L} 2^{s-1} = R_L - 1 < R_L$$

total twiddle records sequentially without strided rescans.

Breadth-first downward, leaf, and upward tape sweeps. The machine executes the recursion tree on $O(1)$ tapes without recursion-stack tape growth or head shuttling between parent and child regions. Besides the input and output tapes, it uses two alternating data tapes $T_{\text{data}}^{(0)}, T_{\text{data}}^{(1)}$, four scratch tapes $T_1, \dots, T_4$, the table tape T_{tab}, and a level-segmented stack tape T_{stack}. It also allocates $O(1)$ dedicated work tapes for the nested inner multiplier of Section 7.2:

- *Downward sweep for $\ell = 0, \dots, L-1$:* At the start of level ℓ, $T_{\text{data}}^{(\ell \bmod 2)}$ holds the R_0/R_ℓ node input vectors $x^{(\ell,i)} \in \widetilde{\mathbb{C}}_{p_\ell+2}^{R_\ell}$ for $0 \le i < R_0/R_\ell$ contiguously in natural node order as uniform fixed-width records of width w_ℓ, occupying $R_0 w_\ell = \Theta(n)$ bits. Positioning T_{tab} at the level-ℓ table segment and $T_{\text{data}}^{(\ell+1 \bmod 2)}$ at its start, the machine processes each node $i = 0, \dots, R_0/R_\ell - 1$ sequentially in three steps.

  First, it scans $x^{(\ell,i)}$ on $T_{\text{data}}^{(\ell \bmod 2)}$ using a $\log_2 t_{1,\ell}$-bit counter to accumulate the exact Nyquist spatial alternating sum $A_{\text{N}}^{(\ell,i)} \in \widetilde{\mathbb{C}}_{q_{\text{child}}}$ from the stride-$t_{1,\ell}$ offset-$t_{1,\ell}/2$ entries (25). Appending $A_{\text{N}}^{(\ell,i)}$ to the level-ℓ segment of T_{stack} costs $O(R_\ell w_\ell)$ steps per node and writes $O((R_0/R_\ell)w_\ell) = O(n)$ stack bits per level.

  Second, it de-interleaves $x^{(\ell,i)}$ into $t_{1,\ell}$ rows of length r_ℓ via Lemma 3.3(i) and runs Step 1 row spreading from Theorem 6.3. Row j_1's pre-packed filter integer of $O(\tilde{W}_\ell p_{\ell+1})$ bits is replayed from T_{tab} once for each of the $\lceil r_\ell/\tilde{W}_\ell \rceil$ blocks in row j_1, taking $O(r_\ell p_{\ell+1})$ table-replay steps per row and $O(R_\ell p_{\ell+1})$ per node.

  Third, it runs Step 2a pre-twist by replaying the r_ℓ twist roots from T_{tab} once per row in $O(R_\ell p_{\ell+1})$ steps per node. It then runs the Step 2b polynomial transform $G_{t_{1,\ell}}$ of Lemma 3.5, whose output polynomials $\widehat{Q}_{v_1}$ arrive in natural order $v_1 = 0, \dots, t_{1,\ell} - 1$, followed by Step 2c post-twist. The resulting $t_{1,\ell}$ child input vectors $\widehat{V}_{v_1,\cdot}$, padded to width $w_{\ell+1}$, are appended sequentially to $T_{\text{data}}^{(\ell+1 \bmod 2)}$. At the end of level ℓ, rewinding $T_{\text{data}}^{(\ell+1 \bmod 2)}$ takes $O(R_0 w_{\ell+1}) = O(n)$ steps, and replaying the level-ℓ table across all R_0/R_ℓ nodes takes $O(R_0 p_{\ell+1}) = O(n)$ steps.

- *Leaf evaluation at $\ell = L$:* The machine reads the R_0/R_L contiguous leaf vectors of length $R_L \le 4p_L^2$ sequentially from $T_{\text{data}}^{(L \bmod 2)}$, writes the evaluated leaf outputs sequentially back to $T_{\text{data}}^{(L \bmod 2)}$, and rewinds at the end in $O(n)$ steps. On each leaf vector, `BaseRadix2FFT` first bit-reverses the R_L records in $K_L = \log_2 R_L$ sequential even/odd split passes in $O(R_L(1+K_L)q_b)$ steps. In each stage $s \in \{1, \dots, K_L\}$, it splits each block of 2^s records into its upper and lower halves of length 2^{s-1} on T_1, T_2 while replaying the 2^{s-1}-entry stage-s twiddle list from T_{tab} once per block. Across all K_L stages and all R_0/R_L leaves, leaf table replay and tape movement take

$$\frac{R_0}{R_L} \cdot O\big(R_L(1+K_L)q_b\big) = O(n \log\log n)$$

  Turing machine steps.

- *Upward sweep for $\ell = L-1$ down to 0:* At the start of level ℓ, $T_{\text{data}}^{(\ell+1 \bmod 2)}$ holds the $(R_0/R_\ell)t_{1,\ell}$ child output vectors grouped into R_0/R_ℓ contiguous node groups of $t_{1,\ell}$ rows of length r_ℓ in natural node order $i = 0, \ldots, R_0/R_\ell - 1$. The machine rewinds T_{stack} across the level-ℓ segment in $O(n)$ steps to its first entry $i = 0$ and rewinds $T_{\text{data}}^{(\ell \bmod 2)}$ to its start. Processing the node groups sequentially in natural forward order $i = 0, \ldots, R_0/R_\ell - 1$, it first reads node i's $t_{1,\ell} \times r_\ell$ child outputs from $T_{\text{data}}^{(\ell+1 \bmod 2)}$ and routes them into column-major order (u_2, u_1) via Lemma 3.3(ii). Next, it reads $A_{\text{N}}^{(\ell,i)}$ sequentially from the level-ℓ segment of T_{stack} and replays the r_ℓ solver tuples from T_{tab} sequentially as $u_2 = 0, \ldots, r_\ell - 1$ in $O(r_\ell p_{\ell+1})$ table steps per node to run `CyclicSolve`/`NyquistSolve` and quantize to $\widetilde{\mathbb{C}}_{p_\ell+2}$. Finally, it transposes $(\widehat{Z}_{u_1,u_2})$ from column-major order (u_2, u_1) back to natural frequency order $u_2 + r_\ell u_1$ via Lemma 3.3(i) and appends the solved parent vector, re-padded to width w_ℓ, sequentially to $T_{\text{data}}^{(\ell \bmod 2)}$. At the end of level ℓ, it rewinds $T_{\text{data}}^{(\ell \bmod 2)}$ and erases and rewinds the level-ℓ segment of T_{stack} in $O(n)$ steps.

Summing over all $L = O(\log \log n)$ levels, total table replay and level-sweep coordination take $O(n \log \log n) = o(n \log n)$ Turing machine steps.

B Multitape routing proofs

This appendix proves the multitape Turing machine time bounds of Lemma 3.3 for matrix transposition and odd-stride column routing. These two data-movement subroutines connect the row-wise spreading and polynomial-transform stages to the column-wise deconvolution stage at every recursion level. Readers familiar with multi-pass merge and transpose schedules on multitape Turing machines may skip directly to Appendix C.

Proof idea. Both routing tasks reduce to $\log_2 t_1$ sequential passes over contiguous fixed-width records using two auxiliary tapes and an amortized-$O(1)$ binary counter. Matrix transposition repeatedly splits and interleaves sub-columns of doubling length 2^s, whereas odd-stride column routing first transposes into column-major order, tags each record with its destination index $u_1 = (2u_2 - 1)v_1 \bmod t_1$, and runs bottom-up merge sort across all columns in parallel.

Proof of Lemma 3.3. We first record the amortized cost of binary record counters and then prove parts (i) and (ii) in turn.

Counter amortization. Incrementing or decrementing a binary counter on a dedicated counter tape N times in succession flips bit b at most $\lceil N/2^b \rceil$ times. Across all N operations, this costs $O(N + \log N)$ total head steps, or amortized $O(1)$ steps per increment.

(i) Matrix transposition. A $t_1 \times r$ array of fixed-width w-bit records occupies

$$t_1 r w = \Theta(t_1 r q)$$

tape cells. Here row-major order (j_1, j_2) consists of $t_1 = 2^{k_1}$ contiguous rows of length r, and column-major order (j_2, j_1) consists of r contiguous columns of length t_1 (cf. [HvdH21, Section 2.1]). To transpose from row-major order (j_1, j_2) to column-major order (j_2, j_1), execute $k_1 = \log_2 t_1$ sequential passes $s = 1, \ldots, k_1$. At the start of pass s, the tape holds $t_1/2^{s-1}$ blocks, each consisting of r contiguous sub-columns of 2^{s-1} records. In a single scan over the tape, copy even-indexed blocks to auxiliary tape T_1 and odd-indexed blocks to auxiliary tape T_2. A $(k_2 + s - 1)$-bit record counter tracks the block boundaries during this scan. Next, rewind T_1 and T_2, and scan T_1 and T_2 in

lockstep to interleave each pair of 2^{s-1}-record sub-columns into a 2^s-record sub-column, producing $t_1/2^s$ blocks of r sub-columns of length 2^s. After k_1 passes, the single block holds r contiguous columns of length t_1.

Reversing the k_1 passes transposes column-major order (j_2, j_1), indexed by $j = j_1 + t_1 j_2$, back into row-major order (j_1, j_2). Specifically, each reversed pass splits every length-2^s sub-column into its first and second halves of length 2^{s-1} on T_1, T_2 and concatenates them block-wise. Each pass scans $O(t_1 rw)$ cells, totaling

$$O\big(t_1 rq(1 + \log_2 t_1)\big)$$

Turing machine steps.

(ii) Odd-stride column routing. First transpose $(\widehat{C}_{v_1,u_2})$ from row-major order (v_1, u_2) to column-major order (u_2, v_1) by (i) in $O(t_1 rq(1+\log_2 t_1))$ steps. For each column $u_2 \in \{0, \ldots, r-1\}$, its t_1 records now occupy a contiguous block in natural order $v_1 = 0, \ldots, t_1 - 1$. In one sequential scan over all columns $u_2 = 0, \ldots, r-1$, maintain the column stride and destination tag

$$s(u_2) := (2u_2 - 1) \bmod t_1, \qquad u_1 = (2u_2 - 1)v_1 \bmod t_1.$$

A k_1-bit accumulator is initialized to 0 at the start of each column and adds $s(u_2)$ modulo t_1 as v_1 increments from 0 to $t_1 - 1$. Prepending the k_1-bit destination tag u_1 to each record takes $O(k_1) \le O(q)$ steps per record.

Since each tagged record has fixed width $w + k_1 = \Theta(q)$, execute $k_1 = \log_2 t_1$ passes of standard bottom-up merge sort within all r columns simultaneously. In pass $s \in \{0, \ldots, k_1 - 1\}$, bit s of a k_1-bit record counter detects alternating sorted runs of length 2^s within each column and distributes them to T_1 and T_2. Merging each pair of length-2^s runs on T_1 and T_2 by comparing their k_1-bit tags produces a sorted run of length 2^{s+1}. Each of the k_1 passes scans $O(t_1 rq)$ tape cells, taking

$$O\big(t_1 rq(1 + \log_2 t_1)\big)$$

steps in total. □

C Deferred error and filter calculations

This appendix collects the routine fixed-point rounding and integral-tail calculations deferred from Sections 5 and 6. Together they establish the column-solver rounding bound of Lemma 5.2 and the sinc-Gaussian filter norm, tail, and folding bounds of Lemma 6.1. Readers who accept those two numerical lemmas from the main text may skip this appendix on a first reading.

C.1 Proof of Lemma 5.2

This subsection bounds the fixed-point rounding error accumulated across the two-pass cyclic column solver and the Nyquist column solver.

Proof idea. On non-Nyquist columns, the exact recurrence ratio satisfies $|\rho| < 1$ and its rounded table value satisfies $|\hat{\rho}| \le 1 + \frac{1}{4}u$. Unrolling t_1 steps therefore amplifies local rounding errors by at most $(1 + \frac{1}{4}u)^{t_1} < 1.0001$ and accumulates $O(t_1^2 u \mathcal{M}_Y)$ total error across the prefix and second passes. At the Nyquist column $u_2 = r/2$, the alternating sums are exact on the child grid and only the initial scaling and final division round, giving an $O(t_1 u \mathcal{M}_Y)$ bound.

Proof of Lemma 5.2. By (7), on $\widetilde{\mathbb{C}}_{q_{\text{child}}}$ each real table entry has error $\le \frac{1}{4}u = 0.25u$, while each complex rounded product $\mathrm{RM}_{q_{\text{child}}}$ has error $\le \frac{\sqrt{2}}{8}u < 0.177u$. For $u_2 \neq r/2$, by reflection symmetry between $u_2 < r/2$ and $u_2 > r/2$, it suffices to consider $u_2 < r/2$. Then $M(u_2) > 0.49999999$ implies $M(u_2)^{-1} < 2.0000001$ and $|\mu| \le 2.0001$. Setting $B_i^\dagger := M(u_2)^{-1}\widehat{Y}_i$, we have $|B_i^\dagger| \le 2.0001\mathcal{M}_Y$ and

$$e_i^{(B)} := |\tilde{B}_i - B_i^\dagger| \le |\mu - M(u_2)^{-1}|\,|\widehat{Y}_i| + \frac{\sqrt{2}}{8}u \le \left(\frac{1}{4} + \frac{\sqrt{2}}{8}\right) u\mathcal{M}_Y < 0.427u\mathcal{M}_Y.$$

In the prefix sweep, $P_0^\dagger = B_0^\dagger$ and $P_i^\dagger = B_i^\dagger + \rho P_{i-1}^\dagger$ with $|\rho| = a < 1$, so $|P_i^\dagger| \le 2.0001(i+1)\mathcal{M}_Y$. Since $|\hat{\rho}| \le 1 + \frac{1}{4}u$, the prefix error $e_i^{(P)} := |\tilde{P}_i - P_i^\dagger|$ satisfies $e_0^{(P)} = e_0^{(B)} \le 0.427u\mathcal{M}_Y$ and, for $1 \le i \le t_1 - 1$,

$$e_i^{(P)} \le |\hat{\rho}|e_{i-1}^{(P)} + |\hat{\rho} - \rho|\,|P_{i-1}^\dagger| + e_i^{(B)} + \frac{\sqrt{2}}{8}u \le \left(1 + \frac{1}{4}u\right) e_{i-1}^{(P)} + (0.604 + 0.501i)u\mathcal{M}_Y.$$

Unrolling this linear recurrence from $i = 0$ to $t_1 - 1$, using $(1 + \frac{1}{4}u)^{t_1} \le e^{t_1 u/4} < 1.0001$ since $t_1 u \le 2^{-20}$ and the triangular sum $\sum_{i=1}^{t_1-1} i = \frac{t_1(t_1-1)}{2} < \frac{1}{2}t_1^2$, gives for $t_1 \ge 2$:

$$e_{t_1-1}^{(P)} \le 1.0001\left(0.604t_1 + 0.501\frac{t_1(t_1-1)}{2}\right) u\mathcal{M}_Y < (0.605t_1 + 0.251t_1^2)u\mathcal{M}_Y < 0.56t_1^2 u\mathcal{M}_Y.$$

Next, in the wrap-around closure $Z_{t_1-1}^{\text{comp}} = \mathrm{RM}_{q_{\text{child}}}(\hat{\theta}, \tilde{P}_{t_1-1})$ versus $Z_{t_1-1}^\dagger = \theta P_{t_1-1}^\dagger$, where $\theta := (1 - \rho^{t_1})^{-1} < 1.163$, $|\hat{\theta}| < 1.164$, and $|P_{t_1-1}^\dagger| \le 2.0001t_1\mathcal{M}_Y$, we have

$$\begin{aligned} e_{t_1-1}^{(Z)} := |Z_{t_1-1}^{\text{comp}} - Z_{t_1-1}^\dagger| &\le |\hat{\theta}|e_{t_1-1}^{(P)} + |\hat{\theta} - \theta|\,|P_{t_1-1}^\dagger| + \frac{\sqrt{2}}{8}u \\ &< (0.652t_1^2 + 0.501t_1 + 0.177)u\mathcal{M}_Y < 0.95t_1^2 u\mathcal{M}_Y. \end{aligned}$$

Finally, in the second recurrence pass $Z_i^{\text{comp}} = \tilde{B}_i + \mathrm{RM}_{q_{\text{child}}}(\hat{\rho}, Z_{(i-1) \bmod t_1}^{\text{comp}})$ versus $Z_i^\dagger = B_i^\dagger + \rho Z_{(i-1) \bmod t_1}^\dagger$ for $i = 0, \ldots, t_1 - 2$, Theorem 5.1(2) bounds the exact solution uniformly by $\|Z^\dagger\|_\infty \le \|B^{-1}\|_{\infty\to\infty}\|\widehat{Y}\|_\infty < 2.36t_1\mathcal{M}_Y$. Hence at every step of the second pass, $|\hat{\rho} - \rho|\,|Z_{(i-1) \bmod t_1}^\dagger| \le \frac{1}{4}u(2.36t_1\mathcal{M}_Y) = 0.590t_1 u\mathcal{M}_Y$, so each of the $t_1 - 1$ steps adds local error at most $(0.427 + 0.177 + 0.590t_1)u\mathcal{M}_Y = (0.604 + 0.590t_1)u\mathcal{M}_Y$. Across at most $t_1 - 1$ steps with amplification $(1 + \frac{1}{4}u)^{t_1-1} < 1.0001$, we obtain

$$\|Z^{\text{comp}} - Z^\dagger\|_\infty \le 1.0001\Big(e_{t_1-1}^{(Z)} + (t_1 - 1)(0.604 + 0.590t_1)u\mathcal{M}_Y\Big) < 1.85\,t_1^2\,u\,\mathcal{M}_Y \le 80\,t_1^2\,u\,\mathcal{M}_Y.$$

For $u_2 = r/2$, $|\mu_{\text{N}}| \le 2.0001$, $\|\tilde{D}_i - c^{-1}\widehat{Y}_i\|_\infty \le 0.427u\mathcal{M}_Y$, and the alternating prefix sum $\tilde{P}_i = \tilde{D}_i - \tilde{P}_{i-1}$ is exact on $\widetilde{\mathbb{C}}_{q_{\text{child}}}$, so $\|\tilde{P}_i - P_i^\dagger\|_\infty \le 0.427iu\mathcal{M}_Y < 0.427t_1 u\mathcal{M}_Y$. The alternating sum $\tilde{S}$ is exact on $\widetilde{\mathbb{C}}_{q_{\text{child}}}$, and the single rounding of $(A_{\text{N}} - \tilde{S})/t_1$ adds $\le 0.177u$, yielding

$$\|Z^{\text{comp}} - Z^\dagger\|_\infty \le (0.854t_1 + 0.177)u\mathcal{M}_Y \le 5t_1 u\mathcal{M}_Y \le 80t_1^2 u\mathcal{M}_Y.$$

Since $5t_1 \le 2.5\,t_1^2$ for $t_1 \ge 2$, both cases are below $2.5\,t_1^2\,u\,\mathcal{M}_Y$. We state the generous constant 80, which is the value used in Lemma 6.5. □

C.2 Proof of Lemma 6.1

This subsection proves the ℓ_1 norm bound, the spatial truncation tail bound, and the folded-table quantization bound for the windowed sinc-Gaussian row filters.

Proof idea. Part 1 bounds the two central taps by a convexity–concavity split at the unique inflection point on $[0, 1/2]$ and compares the paired outer taps with an exponential integral to obtain the logarithmic ℓ_1 bound. Part 2 bounds the exterior taps $|d| > m_{1,\ell}$ by separating one negative boundary term and dominating the rest by a Gaussian tail integral. Part 3 applies Poisson summation and Parseval's identity on $\mathbb{Z}_{r_\ell}$ to bound the periodic folded filter, then adds the guard-precision summation and final rounding errors.

Proof of Lemma 6.1. **Part 1.** For $j_1 = 0$, $\kappa_0^{(0)} = 1$ and $\kappa_d^{(0)} = 0$ for all $d \neq 0$, so $\|\kappa^{(0)}\|_1 = 1$. For $1 \le j_1 < t_{1,\ell}$, let $\delta := j_1/t_{1,\ell} \in (0,1)$. For $d \in \{0,-1\}$, $|\kappa_0^{(j_1)}|+|\kappa_{-1}^{(j_1)}| \le \mathrm{sinc}(\delta)+\mathrm{sinc}(1-\delta) = \frac{\sin(\pi\delta)}{\pi\delta(1-\delta)}$. On $[0,1]$, let $h(\delta) := \sin(\pi\delta) - 4\delta(1-\delta)$. By symmetry $h(\delta) = h(1-\delta)$, it suffices to consider $\delta \in [0, 1/2]$, where $h(0) = h(1/2) = 0$, $h'(0) = \pi - 4 < 0$, $h'(1/2) = 0$, and $h''(\delta) = -\pi^2 \sin(\pi\delta) + 8$ decreases from $8 > 0$ at $\delta = 0$ to $8 - \pi^2 < 0$ at $\delta = 1/2$, vanishing at a unique $\delta_* \in (0, 1/2)$. On $[\delta_*, 1/2]$, h is concave with $h(1/2) = h'(1/2) = 0$, so $h(\delta) \le h(1/2) + h'(1/2)(\delta - 1/2) = 0$, and in particular $h(\delta_*) \le 0$. On $[0, \delta_*]$, h is convex, so $h(\delta) \le \max(h(0), h(\delta_*)) \le 0$. Hence $\sin(\pi\delta) \le 4\delta(1-\delta)$ on $[0,1]$ and the two central terms sum to at most $4/\pi$. For $d \ge 1$ and $d \le -2$, pairing $d = k \ge 1$ with $d = -1-k$ gives $|\kappa_k^{(j_1)}| + |\kappa_{-1-k}^{(j_1)}| \le \frac{2}{\pi k} e^{-\pi k^2/\alpha_\ell^2}$. Since $k \mapsto \frac{1}{k} e^{-\pi k^2/\alpha_\ell^2}$ is decreasing on $[1, \infty)$,

$$\sum_{k=1}^{\infty} \frac{2}{\pi k} e^{-\pi k^2/\alpha_\ell^2} \le \frac{2}{\pi} e^{-\pi/\alpha_\ell^2} + \frac{2}{\pi} \int_1^\infty \frac{e^{-\pi t^2/\alpha_\ell^2}}{t}\, dt = \frac{2}{\pi} e^{-\pi/\alpha_\ell^2} + \frac{1}{\pi} \int_{\pi/\alpha_\ell^2}^\infty \frac{e^{-u}}{u}\, du.$$

For the integral over $[\pi/\alpha_\ell^2, 1]$ and $x = \pi/\alpha_\ell^2 \in (0, 0.445)$, we have

$$\int_{\pi/\alpha_\ell^2}^1 \frac{e^{-u}}{u}\, du \le \int_{\pi/\alpha_\ell^2}^1 \frac{1 - u + u^2/2}{u}\, du = \ln(\alpha_\ell^2/\pi) - (1 - \pi/\alpha_\ell^2) + \frac{1}{4}(1 - \pi^2/\alpha_\ell^4),$$

$$2e^{-x} + x - 1 + \frac{1}{4}(1 - x^2) \le 2(1 - x + x^2/2) + x - 1 + \frac{1}{4}(1 - x^2) = \frac{5}{4} - x\left(1 - \frac{3}{4}x\right) < \frac{5}{4} = 1.25.$$

Combining these bounds gives $\sum_{k=1}^\infty \frac{2}{\pi k} e^{-\pi k^2/\alpha_\ell^2} \le \frac{2}{\pi} \ln \alpha_\ell + \frac{1.25 - \ln\pi + E_1(1)}{\pi}$. Since $(1+v)(1 - v/2 + v^2/3) = 1 + \frac{v}{6}(3 - v + 2v^2) \ge 1$ for all $v \ge 0$, we have $\frac{1}{1+v} \le 1 - \frac{v}{2} + \frac{v^2}{3}$ and hence

$$E_1(1) = \int_1^\infty \frac{e^{-u}}{u}\, du = e^{-1} \int_0^\infty \frac{e^{-v}}{1+v}\, dv \le e^{-1} \int_0^\infty e^{-v}\left(1 - \frac{v}{2} + \frac{v^2}{3}\right) dv = \frac{7}{6e} < 0.42920.$$

Using $\ln \pi > 1.14472$, the total constant is at most

$$\frac{4 + 1.25 - 1.14472 + 0.42920}{\pi} = \frac{4.53448}{\pi} < 1.444 < 1.62.$$

Part 2. For $d \ge m_{1,\ell} + 1$, since $\delta \in [0, 1)$, we have $d + \delta \ge d \ge m_{1,\ell} + 1$, so $\sum_{d=m_{1,\ell}+1}^\infty |g(d+\delta)| \le \sum_{d=m_{1,\ell}+1}^\infty \frac{e^{-\pi d^2/\alpha_\ell^2}}{\pi d} \le \int_{m_{1,\ell}}^\infty \frac{e^{-\pi t^2/\alpha_\ell^2}}{\pi t}\, dt$. For $d \le -m_{1,\ell} - 1$, let $k := -d \ge m_{1,\ell} + 1$, so that

$$|d + \delta| = k - \delta > k - 1 \ge m_{1,\ell}.$$

Separating the single negative boundary term $k = m_{1,\ell}+1$, which satisfies $|g(-m_{1,\ell}-1+\delta)| < \frac{e^{-\pi m_{1,\ell}^2/\alpha_\ell^2}}{\pi m_{1,\ell}} \le \frac{2^{-p_{\ell+1}}}{\pi m_{1,\ell}}$, and bounding $\sum_{k=m_{1,\ell}+2}^{\infty} \frac{e^{-\pi(k-1)^2/\alpha_\ell^2}}{\pi(k-1)} \le \int_{m_{1,\ell}}^{\infty} \frac{e^{-\pi t^2/\alpha_\ell^2}}{\pi t}\,dt$ gives

$$\tau \le \frac{2^{-p_{\ell+1}}}{\pi m_{1,\ell}} + 2\int_{m_{1,\ell}}^{\infty} \frac{e^{-\pi t^2/\alpha_\ell^2}}{\pi t}\,dt \le \left(\frac{1}{\pi m_{1,\ell}} + \frac{1}{\pi p_{\ell+1}\ln 2}\right) 2^{-p_{\ell+1}},$$

proving (28). Since $p_{\mathrm{work},\ell} \ge 8$ and $p_{\ell+1} \ge 20$ imply $m_{1,\ell} \ge \lceil \frac{2\ln 2}{\pi}\sqrt{160}\rceil = 6$, we have

$$\frac{1}{\pi m_{1,\ell}} + \frac{1}{\pi p_{\ell+1}\ln 2} \le \frac{1}{6\pi} + \frac{1}{20\pi\ln 2} < 0.077 < 0.10.$$

At all non-leaves, $m_{1,\ell} \ge 7$, giving $\frac{1}{\pi m_{1,\ell}} + \frac{1}{\pi p_{\ell+1}\ln 2} \le \frac{1}{7\pi} + \frac{1}{20\pi\ln 2} < 0.069$.

Part 3. By Poisson summation, substituting $s = t + j_1/t_{1,\ell}$ in $\int_{-\infty}^{\infty} g(t + j_1/t_{1,\ell}) e^{2\pi i t(u_2/r_\ell - m)}\,dt$, the discrete Fourier coefficients of $h^{(j_1),\mathrm{per}} \in \mathbb{C}^{r_\ell}$ are

$$\widehat{h^{(j_1),\mathrm{per}}}_{u_2} = \frac{1}{r_\ell} \sum_{m\in\mathbb{Z}} \hat{g}\left(\frac{u_2}{r_\ell} - m\right) e^{-2\pi i (j_1/t_{1,\ell})(u_2/r_\ell - m)}.$$

In contrast, U° in Theorem 4.2 evaluates $h^{(j_1),\mathrm{per}}$ at reversed index $j_2 - k_2$, flipping the phase sign. Since $\hat{g} \ge 0$ and $\sum_{m\in\mathbb{Z}} \hat{g}(u_2/r_\ell - m) = 1$, we have $|\widehat{h^{(j_1),\mathrm{per}}}_{u_2}| \le \frac{1}{r_\ell}$ for all u_2. By Parseval's identity on $\mathbb{Z}_{r_\ell}$,

$$\|h^{(j_1),\mathrm{per}}\|_2^2 = r_\ell \sum_{u_2=0}^{r_\ell-1} |\widehat{h^{(j_1),\mathrm{per}}}_{u_2}|^2 \le r_\ell \cdot r_\ell \cdot \frac{1}{r_\ell^2} = 1,$$

and Cauchy–Schwarz gives $\|h^{(j_1),\mathrm{per}}\|_1 \le \sqrt{r_\ell}\|h^{(j_1),\mathrm{per}}\|_2 \le \sqrt{r_\ell}$. By the triangle inequality, $\|h^{(j_1),\mathrm{tr}} - h^{(j_1),\mathrm{per}}\|_1 \le \sum_{|e|>m_{1,\ell}} |\kappa_e^{(j_1)}| = \tau$. Finally, at guard precision v_{tap}, summing at most $\lceil W_\ell/r_\ell\rceil \le W_\ell$ terms per residue class incurs guard error $\le W_\ell 2^{-v_{\mathrm{tap}}} \le 2^{-p_{\ell+1}-10}$ before the final rounding to $\widetilde{\mathbb{R}}_{q_{\mathrm{child}}}$, which adds at most $\frac{1}{2}2^{-q_{\mathrm{child}}} = \frac{1}{8}2^{-p_{\ell+1}}$. Thus for each $a \in \{0,\dots,\tilde{W}_\ell - 1\}$, $|\bar{H}_a^{(j_1)} - h^{(j_1),\mathrm{tr}}_{(o-a) \bmod r_\ell}| \le (\frac{1}{8} + 2^{-10})2^{-p_{\ell+1}} < \frac{1}{2}2^{-p_{\ell+1}}$. Since $h_d^{(j_1),\mathrm{tr}} = \bar{h}_d^{(j_1)} = 0$ outside the $\tilde{W}_\ell$ indices $d = (o-a) \bmod r_\ell$, $\|\bar{h}^{(j_1)} - h^{(j_1),\mathrm{tr}}\|_1 = \sum_{a=0}^{\tilde{W}_\ell - 1} |\bar{H}_a^{(j_1)} - h^{(j_1),\mathrm{tr}}_{(o-a) \bmod r_\ell}| \le \tilde{W}_\ell 2^{-p_{\ell+1}-1}$. □